



PUSAT PENDIDIKAN DAN PELATIHAN
BADAN PUSAT STATISTIK

MODUL PELATIHAN FUNGSIONAL PRANATA KOMPUTER TINGKAT AHLI

SISTEM INFORMASI

PUSDIKLAT BPS
2022

Hak Cipta © pada:

Pusat Pendidikan dan Pelatihan Badan Pusat Statistik
Edisi Tahun 2022

**Pusat Pendidikan dan Pelatihan Badan Pusat Statistik
Jl. Raya Jagakarsa NO. 70 Jakarta Selatan 12620**

SISTEM INFORMASI

Modul Pelatihan Fungsional Pranata Komputer Tingkat Ahli

TIM PENGARAH SUBSTANSI:

1. Dr. Eni Lestariningsih, S.Si, MA
2. Dr. Pudji Ismartini M.App.Stat
3. Atas Parlindungan Lubis S.Si, M.Si

PENULIS MODUL:

1. Danuk Cahya Permana S.ST, M.T.I.
2. Dede Trinovie Rawung SSi, MStat
3. Andrian Widihastanto S.Kom
4. Else Huslijah S.Tr.Stat.

EDITOR: -

COVER: Else Huslijah S.Tr.Stat.

Sumber Ilustrasi: freepik.com, unsplash.com

JAKARTA – PUSDIKLAT BPS – 2022

ISBN: -

KATA PENGANTAR



Puji syukur kehadiran Allah SWT yang telah memberikan petunjuk sehingga Modul 8 Sistem Informasi ini dapat disusun. Modul ini membekali peserta dengan pengetahuan dan keahlian tentang sistem informasi khususnya dalam menganalisis kebutuhan, merancang, membangun, mengembangkan, melakukan uji coba, dan memantau kinerja sistem informasi sehingga mampu melakukan

pekerjaan terkait sistem informasi dengan baik sesuai standar yang berlaku

Modul ini merupakan salah satu dari tiga belas modul yang diberikan kepada peserta Pelatihan Fungsional Pranata Komputer (Prakom). Ke-tigabelas modul adalah:

1. Modul 1: *Information Technology Enterprise*
2. Modul 2: Manajemen Layanan Teknologi Informasi
3. Modul 3: Pengelolaan Data
4. Modul 4: Manajemen Risiko Teknologi Informasi
5. Modul 5: Audit Teknologi Informasi
6. Modul 6: Sistem Jaringan Komputer
7. Modul 7: Manajemen Infrastruktur Teknologi Informasi
8. Modul 8: Sistem Informasi
9. Modul 9: Pengolahan Data
10. Modul 10: Area Teknologi Informasi Spesial
11. Modul 11: Dokumentasi dan Laporan
12. Modul 12: Pengembangan Profesi Pranata Komputer
13. Modul 13: Administrasi dan Penilaian Pranata Komputer

Ucapan terima kasih dan apresiasi kami sampaikan kepada seluruh pihak yang telah membantu dan memberikan masukan dalam penyusunan modul ini. Tanggapan dan saran yang konstruktif kami harapkan guna perbaikan dan pengembangan di masa mendatang. Semoga modul ini dapat bermanfaat bagi pengembangan kompetensi bidang prakom para peserta pelatihan.

Jakarta, Februari 2022
Kepala Pusdiklat BPS

Eni Lestariningsih, S.Si, M.A.
NIP. 197003101994012001

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
DAFTAR TABEL	iv
DAFTAR GAMBAR.....	v
BAB I PENDAHULUAN	1
Kompetensi Dasar	1
Indikator Keberhasilan	2
Panduan Penggunaan Modul.....	2
BAB II PENGENALAN SISTEM INFORMASI.....	4
2.1. Uraian Materi	4
a. Penjelasan Umum Sistem Informasi	4
b. Jenis Sistem Informasi	6
c. Daur Hidup Sistem Informasi	13
d. Penyebab Kegagalan Proyek Sistem Informasi.....	20
2.2. Rangkuman	20
2.3. Soal Latihan.....	21
BAB III ANALISIS KEBUTUHAN SISTEM INFORMASI	22
3.1. Uraian Materi	22
a. Analisis Permasalahan.....	22
b. Identifikasi Kebutuhan Pengguna	23
c. Pendefinisian Solusi.....	27
d. Alat Bantu Analisis	30
e. Studi Kelayakan.....	41
3.2. Rangkuman	45
3.3. Soal Latihan.....	46
3.4. Contoh Kasus.....	46
BAB IV PERANCANGAN SISTEM INFORMASI	47
4.1. Uraian Materi	47
a. Pemodelan Proses	47

b. Perancangan Antarmuka Pengguna	76
c. Perancangan Arsitektur	83
4.2. Rangkuman	97
4.3. Soal Latihan.....	98
4.4. Contoh Kasus.....	98
BAB V PEMBANGUNAN, PENGEMBANGAN, DAN UJI COBA SISTEM INFORMASI	99
5.1. Uraian Materi	99
a. Algoritma Pemrograman	99
b. Pemrograman Berorientasi Objek (PBO)	110
c. Pemrograman <i>Database</i>	114
d. Strategi dan Teknis Pengujian Program	119
5.2. Rangkuman	132
5.3. Soal Latihan.....	133
5.4. Contoh Kasus.....	133
BAB VI PEMANTAUAN KINERJA SISTEM INFORMASI.....	135
6.1. Uraian Materi	135
a. Tujuan Pemantauan.....	135
b. Hasil Pemantauan Kinerja Sistem	136
c. Prediksi Pemeliharaan.....	137
6.2. Rangkuman	139
6.3. Soal Latihan.....	140
BAB VII KESIMPULAN.....	141
DAFTAR PUSTAKA	142

DAFTAR TABEL

Tabel 3.1. Contoh Profil Pengguna.....	24
Tabel 3.2. Sindrom Pengguna dan Pengembang Sistem.....	25
Tabel 3.3. Contoh Tabel Penilaian Masalah.....	26
Tabel 3.4. Matriks Solusi Kandidat	42
Tabel 3.5. Matriks Analisis Kelayakan.....	45
Tabel 4.1. Pola <i>Model-View-Controller</i> (MVC).....	87
Tabel 4.2. Pola Arsitektur Berlapis.....	89
Tabel 4.3. Pola <i>Repository</i>	91
Tabel 4.4. Pola <i>Client-Server</i>	93
Tabel 4.5. Pola Pipa dan Filter	94
Tabel 5.1. Contoh <i>Property</i> Objek.....	111
Tabel 6.1. Contoh Hasil Pemantauan Kinerja Sistem.....	137

DAFTAR GAMBAR

Gambar 2.1. Jenis Sistem Informasi berdasarkan Alur Informasi	7
Gambar 2.2. Arsitektur Sistem Tersentralisasi	9
Gambar 2.3. Arsitektur Sistem Desentralisasi.....	10
Gambar 2.4. Arsitektur Sistem <i>Client-Server</i>	12
Gambar 2.5. Contoh Arsitektur Sistem <i>Client-Server</i> Kompleks.....	13
Gambar 2.6. Siklus Hidup Pengembangan Sistem <i>Waterfall</i>	15
Gambar 2.7. Contoh Pembangunan Sistem Dengan Metode <i>Waterfall</i>	16
Gambar 2.8. Siklus Hidup Pengembangan Sistem <i>Iterative</i>	16
Gambar 2.9. Contoh Pembangunan Sistem Dengan Metode <i>Iterative</i>	17
Gambar 2.10. Siklus Hidup Pembangunan Sistem <i>Incremental</i>	17
Gambar 2.11. Contoh Pembangunan Sistem Dengan Metode <i>Incremental</i>	18
Gambar 2.12. Siklus Hidup Pembangunan Sistem <i>Agile</i>	18
Gambar 2.13. Contoh Pembangunan Sistem Dengan Metode <i>Agile</i>	19
Gambar 3.1. Diagram Pohon <i>System Requirement</i>	27
Gambar 3.2. Pertimbangan Batasan Sistem	29
Gambar 3.3. Diagram Fishbone	35
Gambar 3.4. Contoh Diagram Fishbone	35
Gambar 3.5. Contoh Matriks RACI.....	36
Gambar 3.6. Simbol-simbol dalam <i>Flowchart</i>	38
Gambar 3.7. Contoh <i>Flowchart</i> Entri Data	39
Gambar 3.8. Simbol-simbol dalam BPMN	40
Gambar 3.9. Contoh BPMN Proses	40
Gambar 3.10. Contoh BPMN Proses Kolaborasi	41
Gambar 4.1. Notasi DFD	52
Gambar 4.2. Contoh Diagram Konteks	53
Gambar 4.3. Contoh Diagram Zero (<i>Overview Diagram</i>).....	53
Gambar 4.4. Contoh Diagram Rinci (DFD Level n)	53
Gambar 4.5. Notasi <i>Use Case Diagram</i>	56
Gambar 4.6. Contoh <i>Use Case Diagram</i>	56
Gambar 4.7. Notasi <i>Class Diagram</i>	57

Gambar 4.8. Struktur <i>Class</i> pada Notasi <i>Class Diagram</i>	58
Gambar 4.9. Contoh <i>Class Diagram</i>	58
Gambar 4.10. Notasi <i>State Diagram</i>	59
Gambar 4.11. Contoh <i>State Diagram</i>	59
Gambar 4.12. Notasi <i>Sequence Diagram</i>	60
Gambar 4.13. Contoh <i>Sequence Diagram</i>	60
Gambar 4.14. Notasi <i>Communication Diagram</i>	62
Gambar 4.15. Contoh <i>Communication Diagram</i>	62
Gambar 4.16. Notasi <i>Activity Diagram</i>	63
Gambar 4.17. Contoh <i>Activity Diagram</i>	63
Gambar 4.18. Notasi <i>Component Diagram</i>	64
Gambar 4.19. Contoh <i>Component Diagram</i>	64
Gambar 4.20. Notasi <i>Deployment Diagram</i>	65
Gambar 4.21. Contoh <i>Deployment Diagram</i>	66
Gambar 4.22. Notasi <i>Service Interface Diagram</i>	69
Gambar 4.23. Contoh <i>Service Interface Diagram</i>	69
Gambar 4.24. Notasi <i>Service Participant Diagram</i>	71
Gambar 4.25. Contoh <i>Service Participant Diagram</i>	71
Gambar 4.26. Notasi <i>Service Architecture Diagram</i>	72
Gambar 4.27. Contoh <i>Service Architecture Diagram</i>	73
Gambar 4.28. Notasi <i>Service Contract Diagram</i>	74
Gambar 4.29. Contoh <i>Service Contract Diagram</i>	75
Gambar 4.30. Notasi <i>Service Categorization Diagram</i>	75
Gambar 4.31. Contoh <i>Service Categorization Diagram</i>	75
Gambar 4.32. Contoh Rancangan Masukan	78
Gambar 4.33. Contoh Rancangan Keluaran.....	83
Gambar 4.34. Organisasi MVC	88
Gambar 4.35. Arsitektur Aplikasi Web Menggunakan Pola MVC.....	88
Gambar 4.36. Arsitektur Berlapis Generik.....	90
Gambar 4.37. Arsitektur Sistem LIBSYS	90
Gambar 4.38. Arsitektur <i>Repository</i> untuk IDE.....	92
Gambar 4.39. Arsitektur <i>Client-Server</i> untuk Perpustakaan Film	93

Gambar 4.40. Contoh Arsitektur Pipa dan Filter	95
Gambar 4.41. Arsitektur Perangkat Lunak dari Sistem ATM	96
Gambar 4.42. Arsitektur Sistem Pemrosesan Bahasa	97
Gambar 5.1. Contoh Paradigma Prosedural atau Imperatif.....	100
Gambar 5.2. Contoh Paradigma <i>Object Oriented</i>	102
Gambar 5.3. Notasi <i>Flowchart</i>	105
Gambar 5.4. Contoh <i>Flowchart</i>	106
Gambar 5.5. Struktur Alur Algoritma.....	107
Gambar 5.6. Contoh Struktur Dasar Berurutan	108
Gambar 5.7. Contoh Struktur Dasar Pemilihan.....	109
Gambar 5.8. Contoh Struktur Dasar Pengulangan	110
Gambar 5.9. Contoh Kelas	111
Gambar 5.10. Contoh Enkapsulasi	113
Gambar 5.11. Contoh Polimorfisme	114
Gambar 5.12. Langkah Pengujian Program	121
Gambar 5.13. Contoh Penyusunan Definisi <i>Rule Validation</i> Program.....	123
Gambar 5.14. Contoh <i>Requirement Traceability Matrix</i>	124
Gambar 5.15. Contoh <i>Test Case</i>	125
Gambar 5.16. <i>Black Box Testing</i>	125
Gambar 5.17. <i>White Box Testing</i>	126
Gambar 5.18. Kategori Pengujian berdasarkan Kronologis.....	127
Gambar 5.19. Contoh Dokumen Hasil Uji Coba (<i>Test Execution</i>)	131
Gambar 5.20. Contoh <i>Requirement Traceability Matrix</i> setelah Pengujian....	132
Gambar 6.1. Prediksi Pemeliharaan	137

BAB I PENDAHULUAN

Informasi merupakan sesuatu yang sangat penting dalam kehidupan manusia. Karena begitu pentingnya, maka informasi yang datang tidak boleh terlambat, tidak boleh bias, harus bebas dari kesalahan-kesalahan, dan relevan dengan penggunaannya, sehingga informasi tersebut menjadi informasi yang berkualitas dan berguna bagi pemakainya. Untuk mendapatkan informasi yang berkualitas, perlu dibangun sebuah sistem informasi sebagai media pembangkitnya. Sistem informasi merupakan cara untuk menghasilkan informasi yang berguna. Informasi yang berguna akan mendukung sebuah pengambilan keputusan bagi pemakainya.

Pada dasarnya sistem informasi merupakan suatu sistem yang dibuat oleh manusia yang terdiri dari komponen-komponen dalam organisasi untuk mencapai suatu tujuan yaitu menyajikan informasi. Sistem informasi di dalam suatu organisasi yang mempertemukan kebutuhan pengolahan transaksi, mendukung operasi, bersifat manajerial, dan kegiatan strategi dari suatu organisasi dan menyediakan pihak luar tertentu dengan laporan-laporan yang diperlukan.

Dalam modul ini, peserta akan diajak untuk berpikir secara kritis terkait pemahaman dan penerapan sistem informasi sehingga peserta diharapkan dapat melakukan pekerjaan terkait sistem informasi dengan baik sesuai standar yang berlaku dalam pelaksanaan tugas dan fungsinya sebagai pranata komputer. Oleh karena itu, pahamiilah setiap dasar kompetensi yang harus peserta kuasai, beserta indikator keberhasilan dan sejumlah capaian belajar untuk mengukur pemahaman peserta tentang modul ini.

Kompetensi Dasar

Kompetensi dasar yang ingin dicapai melalui modul ini adalah peningkatan pengetahuan dan keahlian tentang sistem informasi khususnya dalam menganalisis kebutuhan, merancang, membangun, mengembangkan,

melakukan uji coba, dan memantau kinerja sistem informasi sehingga mampu melakukan pekerjaan terkait sistem informasi dengan baik sesuai standar yang berlaku.

Indikator Keberhasilan

Setelah mempelajari isi modul dan mengikuti kegiatan pembelajaran di dalamnya, peserta diharapkan dapat:

- a. Menjelaskan tentang sistem informasi;
- b. Menganalisis kebutuhan sistem informasi;
- c. Merancang sistem informasi;
- d. Membangun, mengembangkan, dan melakukan uji coba sistem informasi;
- e. Memantau kinerja sistem informasi.

Panduan Penggunaan Modul

Untuk mengoptimalkan pencapaian tujuan Modul Sistem Informasi, Modul ini dilengkapi dengan bahan pendukung berupa: 1) Bahan bacaan; 2) Kasus; 3) Data; dan 4) Grafik. Untuk memperoleh hasil belajar yang optimal, peserta perlu mengikuti serangkaian pengalaman belajar, yaitu: membaca materi Sistem Informasi secara *e-learning*; melakukan kegiatan yang mengandung unsur pembelajaran tentang substansi Sistem Informasi; mendengar, berdiskusi, dan bersimulasi dalam membahas kasus, serta mengimplementasikan dalam pekerjaan sesuai tugas dan fungsi sebagai pranata komputer khususnya terkait sistem informasi.

Mata Diklat ini terdiri dari lima kegiatan belajar, yakni sebagai berikut:

1. Pengenalan Sistem Informasi
2. Analisis Kebutuhan Sistem Informasi
3. Perancangan Sistem Informasi
4. Pembangunan, Pengembangan, dan Uji Coba Sistem Informasi
5. Pemantauan Kinerja Sistem Informasi

Untuk membantu Saudara dalam mempelajari modul ini, ada baiknya diperhatikan beberapa petunjuk belajar berikut ini:

- a. Bacalah dengan cermat bagian pendahuluan modul ini sampai Saudara memahami secara tuntas tentang apa, untuk apa, dan bagaimana mempelajari modul ini.
- b. Baca sepintas bagian demi bagian dan temukan kata-kata kunci dari kata-kata yang dianggap baru. Carilah dan baca pengertian kata-kata kunci tersebut.
- c. Tangkaplah pengertian demi pengertian dari isi modul ini melalui pemahaman sendiri dan tukar pikiran dengan peserta diklat lain atau dengan narasumber/fasilitator Saudara.
- d. Untuk memperluas wawasan, baca dan pelajari sumber-sumber lain yang relevan Saudara dapat menemukan bacaan dari berbagai sumber, termasuk dari internet.
- e. Mantapkan pemahaman Saudara dengan mengerjakan latihan dalam modul serta mengikuti kegiatan diskusi dan praktik studi kasus dengan peserta diklat lain.
- f. Jangan dilewatkan untuk mencoba menjawab soal-soal yang dituliskan pada setiap akhir kegiatan belajar.

Hal ini berguna untuk mengetahui apakah Saudara sudah memahami dengan benar kandungan modul ini. Selamat belajar!

BAB II PENGENALAN SISTEM INFORMASI

2.1. Uraian Materi

Sebuah sistem informasi pada hakikatnya merupakan suatu sistem yang memiliki komponen-komponen atau subsistem-subsistem untuk menghasilkan informasi. Pada bagian ini akan dibahas mengenai pengertian-pengertian mendasar yang menuju pada pemahaman Sistem Informasi secara menyeluruh.

a. Penjelasan Umum Sistem Informasi

Sistem Informasi adalah kesatuan subsistem/komponen yang terdiri dari komputer, basis data (*database*), sumber daya manusia, sistem jaringan, dan prosedur yang dioperasikan secara terpadu untuk menghasilkan informasi (Badan Pusat Statistik, 2021:5). Definisi sistem informasi menurut para ahli:

1. Menurut J.L. Whitten sistem informasi adalah pengaturan orang, data, proses, dan informasi teknologi/teknologi informasi yang berinteraksi untuk mengumpulkan, memproses, menyimpan, dan menyediakan sebagai *output* informasi yang diperlukan untuk mendukung organisasi (Whitten, 2004)
2. O'Brien mengatakan bahwa sistem informasi merupakan suatu kombinasi dari setiap unit yang dikelola oleh *user* atau manusia, *hardware* (perangkat keras komputer), *software* (perangkat lunak), jaringan komputer dan jaringan komunikasi data (komunikasi), dan juga *database* (basis data) yang mengumpulkan, mengubah, dan menyebarkan informasi tentang suatu organisasi. Jadi, pada dasarnya, sistem informasi memang harus memiliki elemen – elemen tersebut agar dapat berguna dan juga bekerja dengan optimal (O'Brien, 2011).
3. Leitch & Davis juga mengemukakan pendapat mereka mengenai sistem informasi. Sistem informasi adalah sebuah sistem yang terdapat di dalam

suatu organisasi yang berfungsi untuk mempertemukan kebutuhan pengolahan transaksi harian, mendukung operasi, kegiatan manajerial dan strategis dari suatu organisasi dan memberikan hasilnya dalam bentuk laporan bagi pihak – pihak luar (Leitch, 1983).

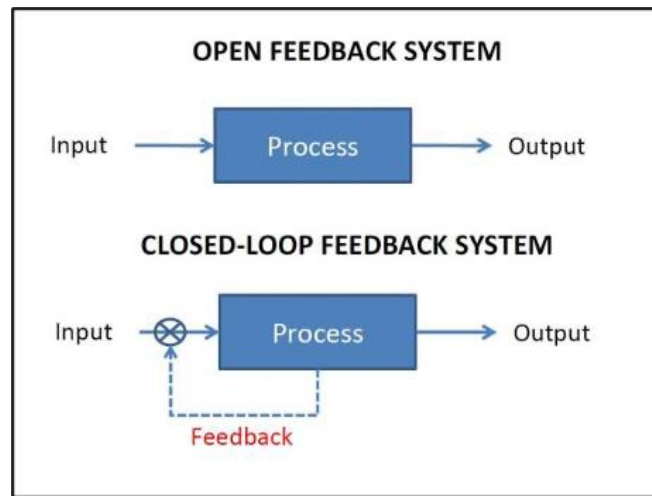
4. Davis (1991) mengatakan bahwa suatu sistem informasi adalah sebuah sistem yang menerima input data dan instruksi, mengolah data sesuai dengan instruksi dan mengeluarkan hasilnya. Dengan begitu, maka bisa disimpulkan bahwa suatu sistem informasi memiliki alur tertentu, mulai dari input hingga menjadi *output* yang bermanfaat (Davis, 2012).
5. Stair & Reynolds (2010) mengatakan bahwa sistem informasi merupakan suatu perangkat elemen atau komponen yang saling terkait satu sama lain, yang dapat mengumpulkan, mengolah, menyimpan dan juga menyebarkan data dan juga informasi, serta mampu untuk memberikan *feedback* untuk memenuhi tujuan suatu organisasi (Stair, 2017).
6. Kenneth & Laudon (2010) Laudon mengatakan bahwa yang dimaksud dengan sistem informasi adalah suatu komponen yang saling bekerja satu sama lain untuk mengumpulkan, mengolah, menyimpan dan juga menyebarkan informasi untuk mendukung kegiatan suatu organisasi, seperti pengambilan keputusan, koordinasi, pengendalian, analisis masalah, dan juga visualisasi dari organisasi (Laudon dan Laudon, 2015).

Tujuan dari sistem informasi adalah menghasilkan informasi. Sistem informasi mengolah data menjadi bentuk yang berguna bagi para pemakainya. Data yang diolah saja tidak cukup dapat dikatakan sebagai suatu informasi. Untuk dapat berguna, maka informasi harus didukung oleh tiga pilar sebagai berikut: (1) tepat kepada orangnya atau relevan (*relevance*), (2) tepat waktu (*timeliness*), dan (3) tepat nilainya atau akurat (*accurate*). Keluaran yang tidak didukung oleh tiga pilar ini tidak dapat dikatakan sebagai informasi yang berguna, tetapi merupakan “sampah” (*garbage*).

b. Jenis Sistem Informasi

Dari konfigurasi alur informasi yang terdapat di dalamnya, sistem informasi terdiri dari sistem informasi terbuka (*Open Loop Information System*) dan sistem informasi tertutup (*Closed Loop Information System*). Sistem informasi tertutup merupakan sistem yang menyediakan informasi umpan balik yang digunakan oleh kontrol untuk melakukan penyempurnaan terhadap proses atau pun input sistem, sedangkan sistem informasi terbuka tidak menyediakan umpan balik pada prosesnya.

Pada kenyataannya sebagian besar sistem informasi merupakan sistem informasi tertutup. Hal ini didukung melalui Sommerville, pengembangan perangkat lunak tidak berhenti ketika sistem dikirimkan tetapi berlanjut sepanjang masa pakai sistem. Setelah sistem dikerahkan, mau tidak mau harus berubah jika ingin tetap berguna. Perubahan bisnis dan perubahan harapan pengguna menghasilkan persyaratan baru untuk perangkat lunak yang ada. Bagian dari perangkat lunak mungkin harus dimodifikasi untuk memperbaiki kesalahan yang ditemukan dalam operasi, untuk menyesuaikannya dengan perubahan pada platform perangkat keras dan perangkat lunaknya, dan untuk meningkatkan kinerjanya atau karakteristik non-fungsional lainnya. Berdasarkan jajak pendapat industri informal, Erlich (2000) menunjukkan bahwa 85-90% dari biaya perangkat lunak organisasi adalah biaya evolusi. Survei lain menunjukkan bahwa sekitar dua pertiga dari biaya perangkat lunak adalah biaya evolusi. Yang pasti, biaya perubahan perangkat lunak adalah bagian besar dari anggaran TI untuk semua perusahaan.



Gambar 2.1. Jenis Sistem Informasi berdasarkan Alur Informasi

Terdapat beberapa klasifikasi yang membagi jenis-jenis sistem informasi yaitu:

1. Level Organisasi

- a) Sistem Informasi Departemen adalah sistem informasi yang hanya digunakan dalam sebuah departemen. Contoh, Aplikasi pemantau kinerja pegawai yang digunakan departemen SDM.
- b) Sistem Informasi Perusahaan adalah sebuah sistem terpadu yang dapat dipakai sejumlah departemen bersama-sama Contoh, sistem informasi perguruan tinggi.
- c) Sistem Informasi Antar Organisasi adalah sistem informasi yang menggabungkan dua organisasi atau lebih. Contoh, sistem informasi reservasi pesawat terbang.

2. Dukungan Kepada Pemakai

Berdasarkan fungsi yang diberikan kepada pemakai, sistem informasi yang digunakan pada semua area fungsional dalam organisasi dapat diklasifikasikan sebagai berikut:

- a) Sistem Pemrosesan Transaksi (*Transaction Processing System*/TPS) adalah sistem yang dapat memproses jumlah data yang besar dan sumber data umumnya internal dan keluaran terutama dimaksudkan

untuk pihak internal. Pemrosesan informasi dilakukan secara teratur dalam harian, bulanan, dsb. Kapasitas penyimpanan besar dan kecepatan pemrosesan yang diperlukan tinggi karena volume yang besar.

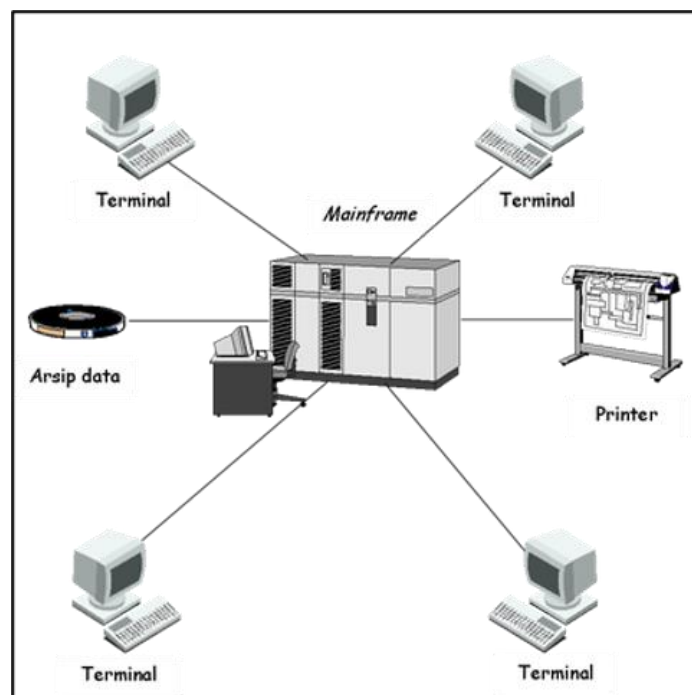
- b) Sistem Informasi Manajemen (*Management Information System/MIS*) adalah sistem yang beroperasi pada tugas-tugas yang terstruktur, yakni pada lingkungan yang telah mendefinisikan pekerjaan secara tegas dan jelas dan dapat meningkatkan efisiensi dengan mengurangi biaya, serta menyediakan laporan dan kemudahan akses yang berguna untuk pengambilan keputusan tetapi tidak secara langsung.
- c) Sistem Otomasi Perkantoran (*Office Automation System/OAS*) adalah sistem yang memberikan fasilitas tugas-tugas pemrosesan informasi sehari-hari di dalam perkantoran dan organisasi bisnis. Sistem ini sering kali dikatakan dapat mendukung kantor tanpa kertas (*paperless office*).
- d) Sistem Pendukung Keputusan (*Decision Support System/DSS*) adalah sistem berbasis komputer yang interaktif dan membantu mengambil keputusan dengan menggunakan data dan model untuk memecahkan persoalan-persoalan tak terstruktur.
- e) Sistem Informasi Eksekutif (*Executive Information System/EIS*) adalah sistem yang menyediakan fasilitas yang fleksibel bagi manajer dan eksekutif dalam mengakses informasi eksternal dan internal untuk mengidentifikasi masalah atau mengenali peluang.
- f) Sistem Pendukung Kelompok (*Group Support System/GSS*) adalah sistem informasi yang mendukung sejumlah orang yang bekerja dalam suatu kelompok. Sistem ini mencakup penggunaan teknologi presentasi, pengaksesan basis data pada komputer, dan kemampuan yang memungkinkan peserta pertemuan dapat berkomunikasi secara elektronik.

g) Sistem Pendukung Cerdas (*Intelligent Support System/ISS*) adalah sistem cerdas yang biasa dipakai dalam aplikasi bisnis atau biasa disebut sistem pakar yang mampu memberikan tanggapan yang cepat dan memuaskan terhadap situasi-situasi baru dan menangani masalah yang kompleks berdasarkan penalaran dan pengetahuan.

3. Arsitektur Sistem

a) Arsitektur Tersentralisasi

Arsitektur tersentralisasi (terpusat) sudah dikenal semenjak tahun 1960-an, dengan *mainframe* sebagai aktor utama. *Mainframe* adalah komputer yang berukuran relatif besar yang ditujukan untuk menangani data yang berukuran besar, dengan ribuan terminal untuk mengakses data dengan tanggapan yang sangat cepat, dan melibatkan jutaan transaksi.



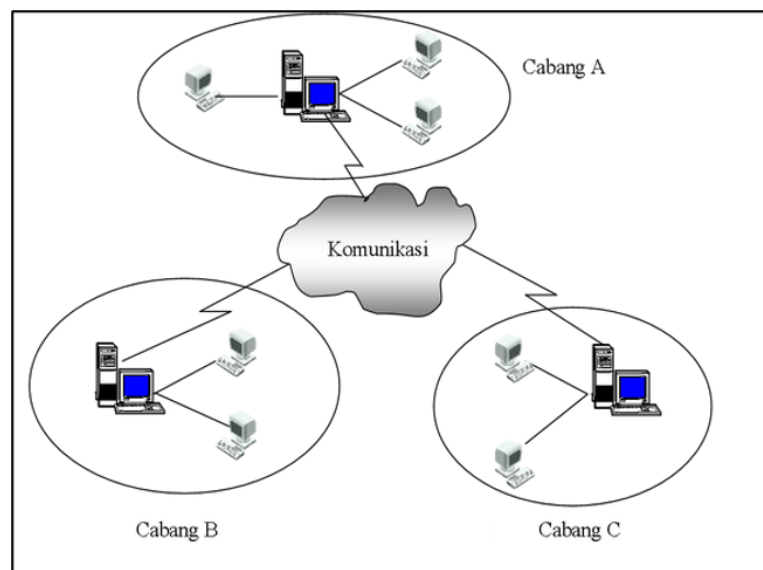
Gambar 2.2. Arsitektur Sistem Tersentralisasi

Implementasi dari arsitektur terpusat adalah pemrosesan data yang terpusat (biasa disebut komputasi terpusat). Semua pemrosesan data dilakukan oleh komputer yang ditempatkan di dalam suatu lokasi yang

ditujukan untuk melayani semua pemakai dalam organisasi. Kebanyakan perusahaan yang tidak memiliki cabang menggunakan model seperti ini.

b) **Arsitektur Desentralisasi**

Arsitektur desentralisasi merupakan konsep dari pemrosesan data tersebar (atau terdistribusi). Sistem pemrosesan data terdistribusi (atau biasa disebut sebagai komputasi tersebar) sebagai sistem yang terdiri atas sejumlah komputer yang tersebar padu di berbagai lokasi yang dihubungkan dengan sarana telekomunikasi dengan masing-masing komputer mampu melakukan pemrosesan yang serupa secara mandiri tetapi bisa saling berinteraksi dalam pertukaran data. Dengan kata lain sistem pemrosesan data distribusi membagi sistem pemrosesan dan terpusat ke dalam subsistem-subsistem yang lebih kecil, yang pada hakikatnya masing-masing subsistem tetap berlaku sebagai sistem pemrosesan data yang terpusat.



Gambar 2.3. Arsitektur Sistem Desentralisasi

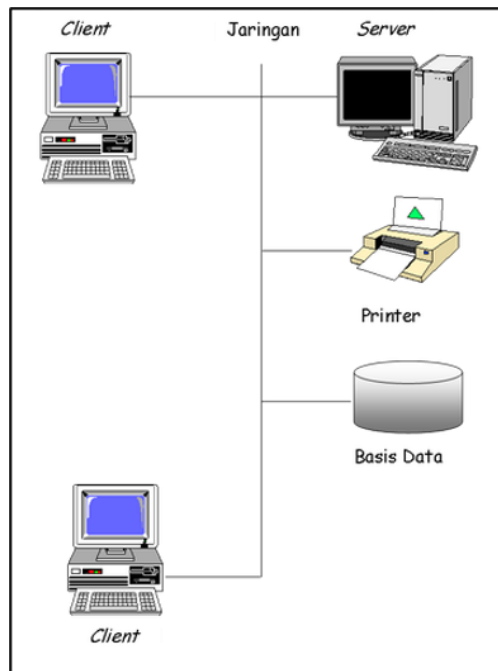
Model sederhana sistem pemrosesan terdistribusi terdapat pada sejumlah komputer yang terhubung dalam jaringan yang menggunakan arsitektur *peer-to-peer* pada model ini komputer memiliki kontrol terhadap *resource* misalnya data, printer atau CD-ROM, tetap

memungkinkan komputer lain menggunakan sumber tersebut. Sistem seperti ini menjadi pemandangan umum semenjak kehadiran PC yang mendominasi perkantoran. Penerapan sistem terdistribusi biasa dilakukan pada dunia perbankan. Setiap kantor cabang memiliki pemrosesan data tersendiri. Namun, jika dilihat pada operasional seluruh bank bersangkutan, sistem pemrosesannya berupa sistem pemrosesan data yang terdistribusi.

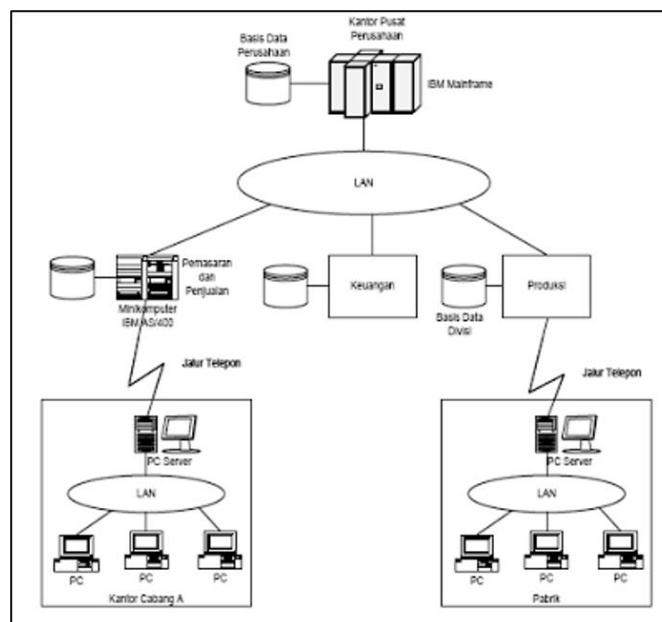
c) *Arsitektur Client Server*

Pada Arsitektur ini terbagi dua yakni *client* dan *server*. *Client* adalah sistem atau proses yang melakukan suatu permintaan data atau layanan ke *server*, sedangkan *Server* merupakan suatu sistem yang menjadi pusat data yang menyediakan data/layanan yang diminta oleh *client*.

Client mempunyai kemampuan untuk melakukan proses sendiri, ketika sebuah *client* meminta suatu data ke *server* maka *server* akan segera menanggapinya dengan memberikan data yang diminta ke *client* bersangkutan dan setelah diterima oleh *client* segera melakukan pemrosesan. Model komputasi yang berbasis *client/server* mulai banyak diterapkan pada sistem informasi. Dengan menggunakan arsitektur ini, sistem informasi dapat dibangun dengan menggunakan perangkat lunak yang berbeda-beda.

Gambar 2.4. Arsitektur Sistem *Client-Server*

Arsitektur informasi mencakup area fungsional (keuangan, akuntansi, dan sebagainya) serta juga sistem-sistem yang mendukungnya (seperti TPS dan MIS).



Gambar 2.5. Contoh Arsitektur Sistem *Client-Server* Kompleks

c. Daur Hidup Sistem Informasi

Siklus hidup sistem (*system life cycle*) adalah proses evolusioner yang diikuti dalam penerapan sistem atau subsistem informasi berbasis komputer. Siklus hidup sistem terdiri dari serangkaian tugas yang mengikuti langkah-langkah pendekatan sistem, karena tugas-tugas tersebut mengikuti pola yang teratur dan dilakukan secara *top down*. Siklus hidup sistem sering disebut sebagai pendekatan air terjun (*waterfall approach*) bagi pembangunan dan pengembangan sistem. Pembangunan sistem hanyalah salah satu dari rangkaian daur hidup suatu sistem. Meskipun demikian proses ini merupakan aspek yang sangat penting. Kita akan melihat beberapa fase atau tahapan daur hidup suatu sistem.

1. *Requirements Analysis* (Analisis Kebutuhan)

Tahap ini menganalisis masalah dan kebutuhan yang harus diselesaikan dengan sistem komputer yang akan dibuat. Tahap ini berakhir dengan pembuatan laporan kelayakan yang mengidentifikasi kebutuhan sistem yang baru dan merekomendasikan apakah kebutuhan atau masalah tersebut dapat diselesaikan dengan sistem komputer yang ada.

2. *System and Software Design* (Perancangan Sistem)

Tahap ini melakukan rancangan desain sistem. Tahap ini memberikan rincian kinerja program dan interaksi antara pengguna dengan program tersebut.

3. *Implementation* (Implementasi)

Tahap ini adalah spesifikasi desain yang telah dibuat untuk diterjemahkan ke dalam program/instruksi yang ditulis dalam bahasa pemrograman.

4. *System Testing* (Pengujian Sistem)

Tahap ini semua program digabungkan dan diuji sebagai satu sistem yang lengkap untuk menjamin semua bekerja dan memenuhi kebutuhan penanganan masalah yang dihadapi.

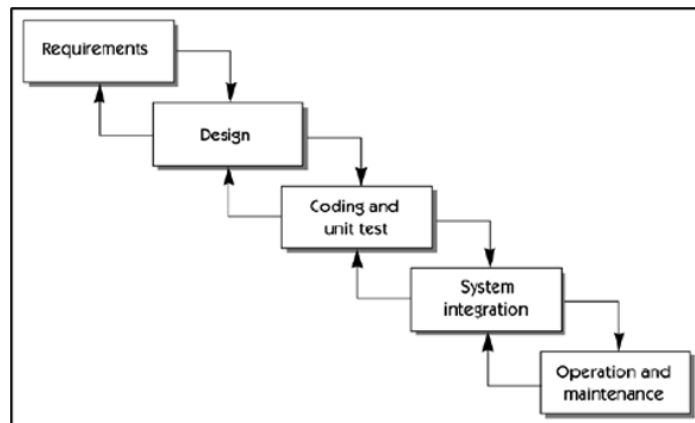
5. *Operation and Maintenance* (Pengoperasian dan Pemeliharaan)

Tahap ini merupakan penerapan program yang telah dibuat untuk digunakan secara utuh dan pengelolaan masalah baru yang muncul sebagai bahan masukan untuk memperbaiki sistem program yang baru.

Sistem informasi kemudian akan melanjutkan daur hidupnya. Sistem dibangun untuk memenuhi kebutuhan. Sistem beradaptasi terhadap aneka perubahan lingkungan yang dinamis hingga kemudian sampai pada kondisi di mana sistem tidak dapat lagi beradaptasi. Sistem baru kemudian dibangun untuk menggantikannya. Terdapat beberapa metode yang lazim digunakan pada daur hidup pengembangan sistem informasi seperti *waterfall*, *iterative*, *incremental*, dan *agile*.

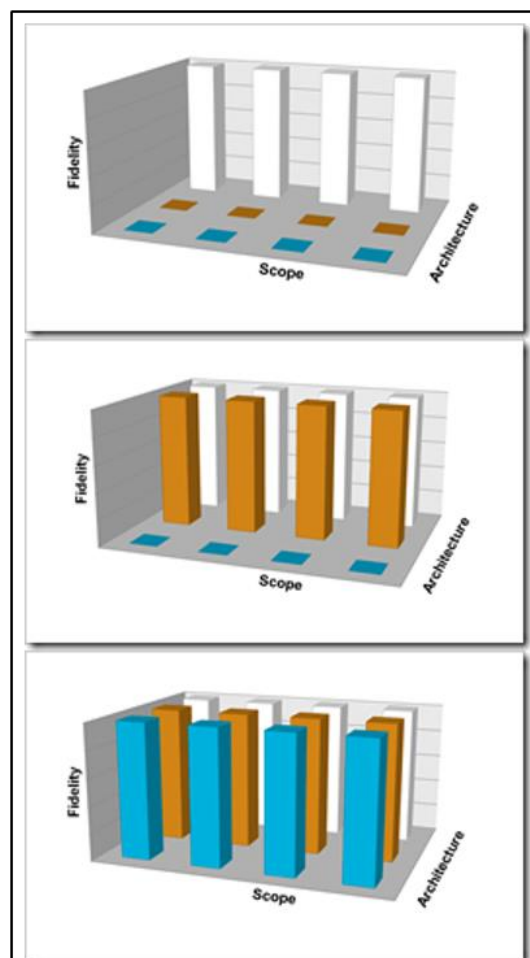
1. *Waterfall*

Menurut Pressman (2010), model waterfall adalah model klasik yang bersifat sistematis, berurutan dalam membangun *software*. Model ini juga dikenal dengan nama "*Linear Sequential Model*" dan sering disebut dengan "*classic life cycle*". Model ini sering dianggap kuno, tetapi merupakan model yang paling banyak dipakai didalam *Software Engineering* (SE). Model ini melakukan pendekatan secara sistematis dan urut mulai dari level kebutuhan sistem lalu menuju ke tahap analisis, desain, *coding*, *testing/verification*, dan *maintenance*. Disebut dengan waterfall karena tahap demi tahap yang dilalui harus menunggu selesainya tahap sebelumnya dan berjalan berurutan. Sebagai contoh tahap desain harus menunggu selesainya tahap sebelumnya yaitu tahap *requirement*.



Gambar 2.6. Siklus Hidup Pengembangan Sistem *Waterfall*

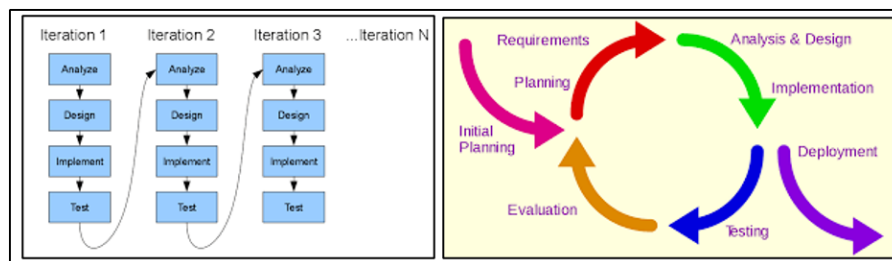
Pembangunan sistem dengan metode *waterfall* dapat dianalogikan dengan gambaran sebagai berikut:



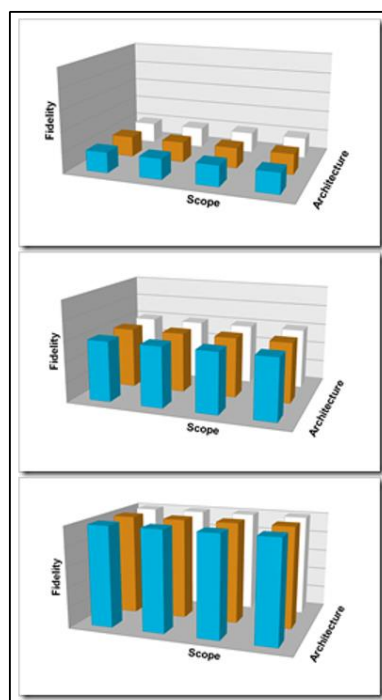
Gambar 2.7. Contoh Pembangunan Sistem Dengan Metode *Waterfall*

2. *Iterative*

Model pengembangan sistem yang bersifat dinamis dalam artian setiap tahapan proses pengembangan sistem dapat diulang jika terdapat kekurangan atau kesalahan. Setiap tahapan pengembangan sistem dapat dikerjakan berupa ringkasan dan tidak lengkap, namun pada akhir pengembangan akan didapatkan sistem yang lengkap pada pengembangan sistem.

Gambar 2.8. Siklus Hidup Pengembangan Sistem *Iterative*

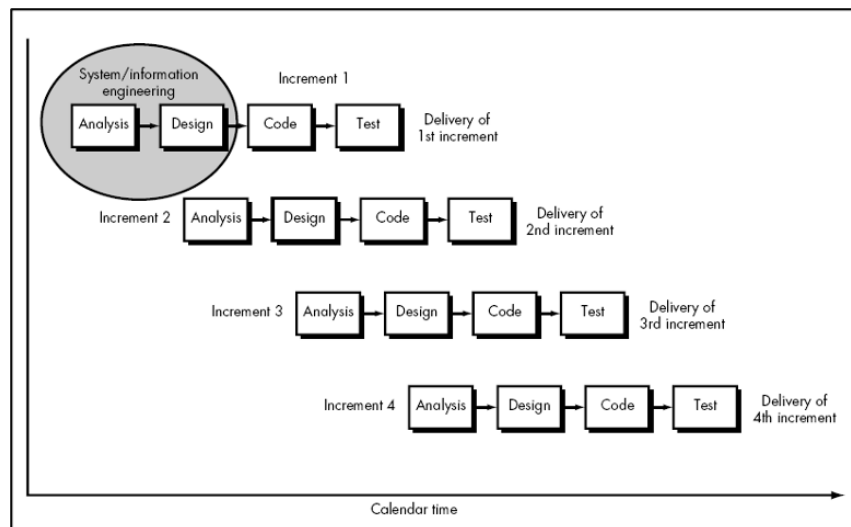
Progres pembangunan sistem dengan metode *iterative* juga dapat dianalogikan dengan gambaran sebagai berikut:



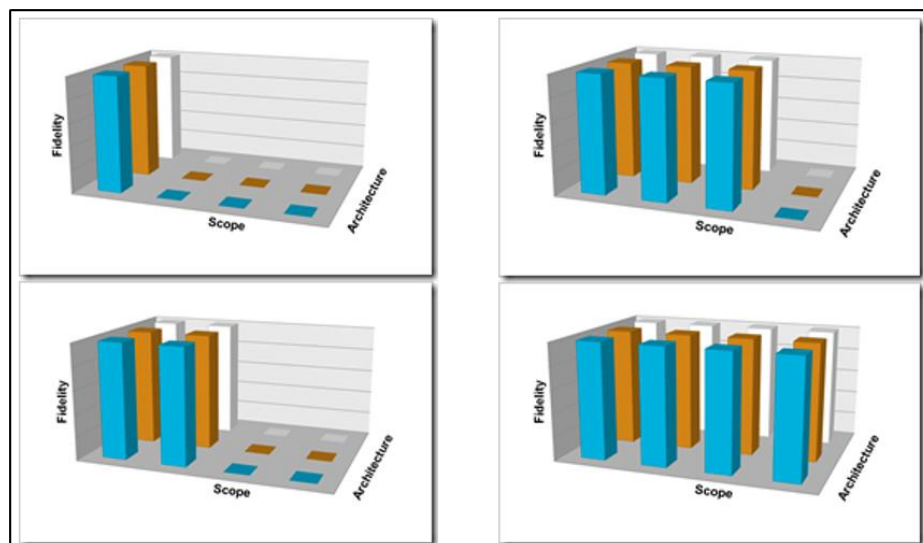
Gambar 2.9. Contoh Pembangunan Sistem Dengan Metode *Iterative*

3. *Incremental*

Model pengembangan sistem yang dipecah sehingga model pengembangannya secara *increment*/bertahap. Kebutuhan pengguna diprioritaskan dan prioritas tertinggi dimasukkan dalam awal *increment*. Model *Incremental* digambarkan sebagai berikut.

Gambar 2.10. Siklus Hidup Pembangunan Sistem *Incremental*

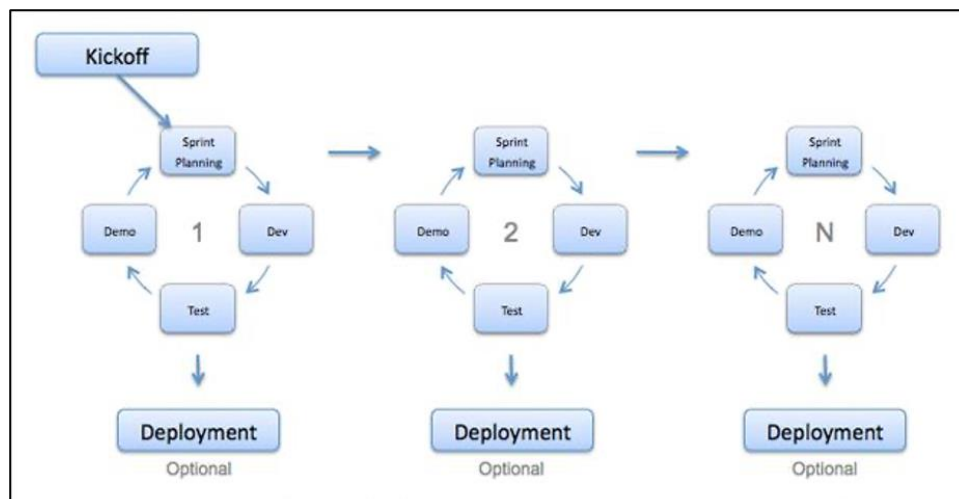
Pembangunan sistem dengan metode *incremental* dapat dianalogikan dengan gambaran sebagai berikut:



Gambar 2.11. Contoh Pembangunan Sistem Dengan Metode *Incremental*

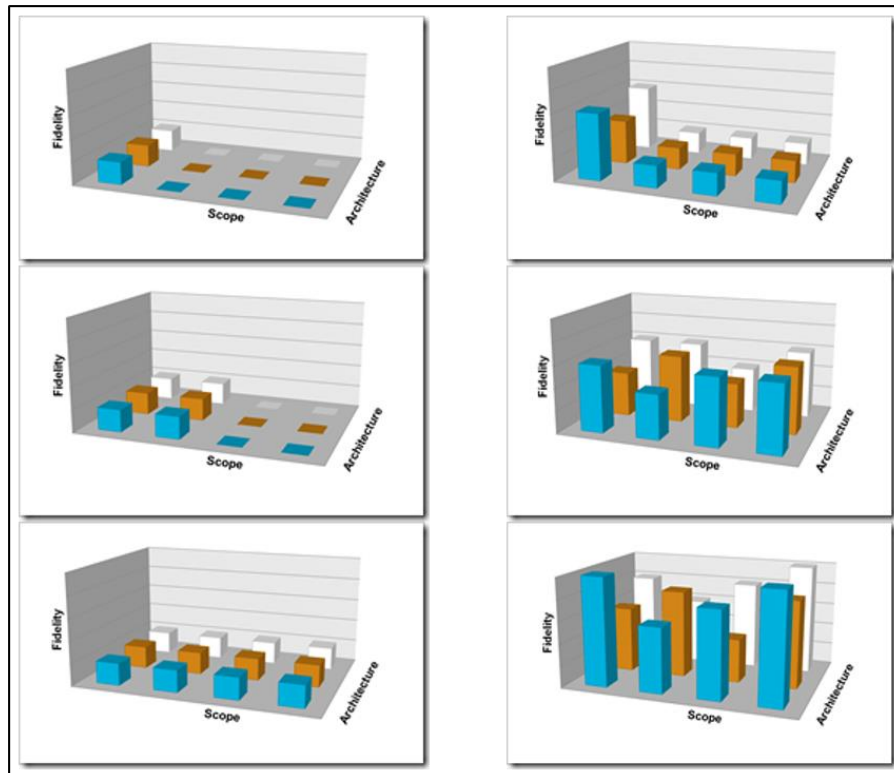
4. Agile

Sekelompok metodologi pengembangan perangkat lunak yang didasarkan pada prinsip-prinsip yang sama atau pengembangan sistem jangka pendek yang memerlukan adaptasi cepat dari pengembang terhadap perubahan dalam bentuk apapun. *Agile* memiliki pengertian bersifat cepat, ringan, bebas bergerak, dan waspada sehingga saat membuat perangkat lunak dengan menggunakan *agile* diperlukan inovasi dan *responsibility* yang baik antara tim pengembang dan klien agar kualitas dari perangkat lunak yang dihasilkan bagus dan kelincihan dari tim seimbang. Secara umum gambaran metode adalah sebagai berikut.

Gambar 2.12. Siklus Hidup Pembangunan Sistem *Agile*

Sekilas pendekatan *agile* ini mirip dengan metode *incremental*. Namun pada *agile*, setiap *increment* akan menghasilkan *working system* yang bisa langsung disebarkan untuk digunakan oleh pengguna dengan mempertimbangkan modularitas dari fitur-fitur yang dirilis. Hal ini berbeda dengan metode *increment* yang mana bisa jadi satu *increment* yang dilakukan belum berupa sistem utuh yang bisa digunakan. Hal ini memudahkan untuk penyesuaian kebutuhan pengguna yang sering berubah-ubah karena sistem sudah dapat disampaikan terlebih dahulu dan

pengembangan selanjutnya bisa lebih independen atau modular terhadap fungsi/fitur yang sudah berjalan. Berdasarkan tahapan pemenuhan kebutuhannya, pembangunan sistem dengan metode *agile* juga dapat dianalogikan dengan gambaran sebagai berikut:



Gambar 2.13. Contoh Pembangunan Sistem Dengan Metode *Agile*

Tidak seperti pada metode-metode sebelumnya yang apabila terjadi perubahan kebutuhan pengguna, maka otomatis akan mempengaruhi anggaran dan waktu pengerjaan sistem, metode *agile* ini lebih memudahkan manajer proyek untuk mengunci anggaran dan waktu agar tidak berubah dengan menyesuaikan cakupan yang ingin diberikan kepada pengguna sistem. Dengan demikian kualitas yang ingin diberikan pada setiap fiturnya akan lebih terjaga.

d. Penyebab Kegagalan Proyek Sistem Informasi

Dari suatu studi yang dilakukan oleh *Project Management Consulting Company, PM Solutions*, ada beberapa penyebab utama kegagalan proyek pengembangan TI. Dan diantara penyebab utama tersebut, *poor requirements management* adalah nomor satu.

Pada tahun 2013, salah satu perusahaan penyedia manajemen portfolio berbasis *cloud* “Innotas” mengungkapkan hasil penelitiannya bahwa sekitar 50% dari proyek TI mengalami kegagalan. Pada penelitian lanjutan tahun 2016 angka ini meningkat menjadi sekitar 55%. Angka yang tidak jauh berbeda dikeluarkan oleh IBM yang mengungkapkan bahwa hanya 40% proyek TI yang sesuai jadwal, anggaran, dan kualitasnya.

Namun demikian pada tahun 2017, laporan Project Management Institute (PMI) menunjukkan peningkatan yang cukup signifikan yaitu dengan rentang sekitar 60-80% proyek TI yang sukses. Penjelasan lebih spesifik pada laporan tersebut menyebutkan penyebab peningkatan performa ini dikarenakan beberapa hal yaitu: manajemen proyek yang semakin matang, era digital yang membuat TI dan bisnis menjadi semakin sejalan, peningkatan kapabilitas baik secara teknis maupun kepemimpinan, penggunaan PMO dan EPMO, dukungan pimpinan, dan yang terakhir adalah *agile*.

2.2. Rangkuman

Sistem informasi mengolah data menjadi bentuk yang berguna bagi para pemakainya dan harus didukung oleh tiga pilar (1) tepat kepada orangnya atau relevan (*relevance*), (2) tepat waktu (*timeliness*), dan (3) tepat nilainya atau akurat (*accurate*).

Sistem informasi dapat diklasifikasikan berdasarkan level organisasi, dukungan kepada pemakai, dan arsitektur sistem.

Siklus hidup sistem terdiri dari serangkaian tugas yang mengikuti langkah-langkah pendekatan sistem dimulai dari analisis kebutuhan, perancangan, implementasi, pengujian, sampai dengan pengoperasian dan pemeliharaan sistem.

Penyebab utama kegagalan proyek pengembangan TI adalah *poor requirements management*. Namun persentase kegagalan ini sudah semakin menurun salah satunya karena penggunaan metode agile yang menerapkan adaptasi cepat dari pengembang terhadap perubahan dalam bentuk apapun.

2.3. Soal Latihan

- 1) Jelaskan secara singkat daur hidup suatu sistem!
- 2) Jelaskan perbedaan metode *Incremental* dengan *Agile*!
- 3) Jelaskan pendapat Anda apakah metode *Agile* harus selalu mengakomodir perubahan *requirement* oleh pengguna atau *stakeholder*!

BAB III ANALISIS KEBUTUHAN SISTEM INFORMASI

3.1. Uraian Materi

Kebutuhan untuk suatu sistem adalah deskripsi tentang apa yang harus dilakukan sistem, layanan yang disediakan, dan batasan operasinya. Kebutuhan ini mencerminkan kebutuhan pelanggan untuk sistem yang melayani tujuan tertentu seperti mengendalikan perangkat, menempatkan pesanan, atau mencari informasi. Proses menemukan, menganalisis, mendokumentasikan, dan memeriksa layanan dan kendala ini disebut analisis kebutuhan (Sommerville, 2011).

Analisis kebutuhan sistem merupakan proses awal dalam tahapan pembangunan atau perbaikan sistem. Dan inti dari proses ini adalah bagaimana mengelola kebutuhan secara efektif akan pembangunan atau perbaikan sistem yang diharapkan (*effective requirements management*). Jika pengelolaan *requirements* ini tidak efektif maka risiko kegagalan pembangunan atau perbaikan sistem sangat tinggi.

a. Analisis Permasalahan

Ketika lingkungan pengembangan aplikasi atau sistem informasi semakin produktif, tentunya semakin lebih mudah untuk mengembangkan sistem yang dapat memenuhi kebutuhan bisnis *user*. Namun, dalam fakta yang telah disampaikan ternyata kita masih berhadapan dengan tantangan untuk dapat memahami dan memenuhi kebutuhan sistem sebenarnya. “*Problem behind the problem*” atau bisa juga disebut akar permasalahan adalah permasalahan kita yang sebenarnya. Kebanyakan pengembang hanya menyisihkan waktu sangat pendek untuk memahami masalah bisnis sebenarnya, kebutuhan *user* dan *stakeholder*, dan lingkungan dimana aplikasi dikembangkan. Sebaliknya, *developers* lebih cenderung fokus pada penyediaan solusi teknologi dengan pemahaman yang minim terhadap masalah yang akan dipecahkan.

Oleh karena itu, pemahaman terhadap akar permasalahan adalah kunci sukses seorang analis dalam menganalisis masalah, yakni memahami masalah bisnis sebenarnya, kebutuhan *user* dan *stakeholder*, dan mengusulkan solusi yang sesuai dengan kebutuhan tersebut. Setelah analis melakukan analisis masalah dengan baik, kemungkinan solusi sederhana dapat diterapkan misal dengan *bypass* masalah yang disebabkan oleh suatu proses tanpa harus memperbaiki proses tersebut atau dengan memperbaiki proses bisnis saja dan bukan membangun sistem yang baru.

Ketika analis berhadapan dengan *problem solving* baik sederhana maupun kompleks, analis harus memahami tujuan terlebih dahulu. Tujuan dalam analisis masalah adalah memahami masalah yang akan diselesaikan sebelum memulai pengembangan.

Salah satu cara sederhana untuk mendapatkan persetujuan adalah menulis masalah dan mendapatkan persetujuan masalah tersebut. Akan sangat membantu jika ada pemahaman terhadap manfaat dari solusi yang diusulkan bagi *user*. Melihat manfaat dari kacamata *user* akan sangat membantu memahami pandangan *stakeholder* terhadap masalah itu sendiri. Setelah memahami masalah besar sistem, maka analis harus memahami penyebab dari masalah itu dengan berbagai macam teknik. Salah satu teknik yang bisa dilakukan adalah analisis akar masalah (*root cause analysis*), yaitu cara sistematis dalam mengurai akar masalah dari masalah yang sudah teridentifikasi atau gejala masalah. Jika semua akar masalah berhasil diidentifikasi dan kita mendapatkan seberapa sering akar masalah tersebut muncul, maka cukup bagi analis untuk fokus pada akar masalah yang sering ditemukan tersebut.

b. Identifikasi Kebutuhan Pengguna

Memahami kebutuhan *user* dan *stakeholder* adalah kunci sukses pengembangan solusi yang efektif. Langkah awal sebelum melakukan identifikasi kebutuhan adalah melakukan identifikasi *user* dan *stakeholder*.

Identifikasi *user* maupun *stakeholder* ini akan sangat membantu analis dalam mendapatkan berbagai macam perspektif terhadap masalah dan kebutuhan akan solusinya. Jika ingin lebih maksimal lagi solusi yang diharapkan, diperlukan identifikasi *non-user stakeholder*, yaitu *stakeholder* yang hanya terpengaruh oleh *outcome* dari sistem.

Hasil identifikasi pengguna berupa daftar profil masing-masing pengguna seperti pada Tabel 3.1.

Tabel 3.1. Contoh Profil Pengguna

<i>Roles</i>	IT Manager
<i>Type</i>	<i>Techninal User</i>
<i>Representative</i>	Mr. Lukman Hadiwijaya
<i>Responsibilities</i>	Bertanggungjawab atas <i>development</i> dan implementasi <i>project</i>
<i>Success Criteria</i>	Proyek selesai dengan tidak melebihi <i>budget</i> yang ditetapkan
<i>Involvement</i>	<i>Steering Committee</i>
<i>Deliverables</i>	<i>Project completion</i>
<i>Comment/Issue</i>	Proyek selesai tepat waktu dan sesuai dengan kebutuhan

Ada beberapa teknik yang bisa digunakan untuk mengidentifikasi kebutuhan pengguna, baik yang sederhana atau sulit, mahal dan sangat teknis, antara lain *Direct Observation* (DIL0: *day in the life of*), wawancara, *brainstorming*/curah pendapat, *Focus Group Discussion* (FGD), kuesioner, dan sebagainya. Tidak ada satu teknik yang sempurna di setiap kasus. Oleh karena itu, penggunaan berbagai macam teknik akan dapat memperkaya pemahaman seorang analis terhadap masalah yang dihadapi. Beberapa sindrom yang sering ditemui pada penggalan kebutuhan dari *stakeholder* dapat dilihat pada Tabel 3.2.

Tabel 3.2. Sindrom Pengguna dan Pengembang Sistem

<i>The "User and the Developer" Syndrome</i>	
<i>Problem</i>	<i>Solution</i>
<i>Users do not know what they want, or they know what they want but cannot articulate it.</i>	<i>Recognize and appreciate the user as domain expert; try alternative communication and elicitation techniques.</i>
<i>Users think they know what they want until developers give them what they said they wanted.</i>	<i>Provide alternative elicitation techniques earlier: storyboarding, role playing, throwaway prototypes, and so on.</i>
<i>Analysts think they understand user problems better than users do.</i>	<i>Put the analyst in the user's place. Try role playing for an hour or a day.</i>
<i>Everybody believes everybody else is politically motivated.</i>	<i>Yes, its part of human nature, so let's get on with the program.</i>

Pemahaman yang lebih baik tentang sistem dapat dicapai jika dan hanya jika analis memahami dengan benar *"needs" user* atau *stakeholder*. Pemahaman ini akan menjadi rujukan untuk dapat mengambil keputusan yang lebih baik dalam definisi dan implementasi sistem. Meskipun, dalam kebanyakan hal *"needs"* dari *user* tidak jelas atau samar atau bahkan rancu, tetapi pernyataan tersebut menjadi sangat penting bagi langkah berikutnya. Sebagai contoh "Saya ingin cara mudah untuk mengetahui inventori saya", "Saya ingin ada peningkatan produktivitas lebih pada layanan penjualan saya", dll. Jadi, *"needs"* adalah refleksi bisnis, personal, atau masalah/peleuang operasional yang ditujukan untuk pertimbangan, pembelian atau penggunaan sistem yang baru.

Menariknya, ketika kita mau melangkah ke proses berikutnya untuk identifikasi fitur dari sistem yang dapat menjawab *"needs" user*, sering dijumpai bahwa kita sudah mendapat jawaban fitur sistem tersebut dari *user*. *User* atau *stakeholder* sudah menerjemahkan kebutuhannya (*needs*) ke perilaku sistem yang diyakini dapat memecahkan kebutuhan riil mereka (*real needs*). "Saya ingin ..." telah diterjemahkan menjadi "Yang saya pikirkan adalah sistem yang dapat menjawab ... atau sistem yang bisa ..." Hal ini tidak menjadi masalah, karena *user* selalu berbicara pengalaman praktisnya dalam bahasa

sehari-hari. Artinya bahwa hubungan antara kebutuhan dan fitur sangat erat sekali.

Fitur adalah cara mudah dan praktis untuk menggambarkan fungsionalitas sistem tanpa harus menyampaikan secara rinci. Fitur adalah layanan yang sistem sediakan untuk memenuhi satu atau lebih kebutuhan *stakeholder* (*stakeholder needs*). Beberapa contoh lain fitur suatu sistem adalah, kontrol pintu manual ketika terjadi kebakaran pada sistem kontrol *lift*, penyediaan status yang *up to date* dari seluruh barang di inventori pada sistem kontrol inventori, dll.

Hasil identifikasi kebutuhan pengguna berupa penjelasan lingkungan pengguna dan tabel penilaian terhadap masalah berdasarkan kriteria disertai bukti metode pengumpulannya seperti hasil observasi, wawancara, *brainstorming*, FGD, atau kuesioner. Beberapa pertanyaan dapat digunakan untuk membantu menjelaskan lingkungan pengguna seperti Siapa saja penggunanya? Apa latar belakang pendidikan dan bagaimana pemahaman tentang TI pengguna? Apakah pengguna berpengalaman dengan jenis aplikasi ini? Platform mana yang digunakan? Apa rencana untuk platform masa depan? Aplikasi tambahan apa yang digunakan yang perlu dihubungkan? Apa harapan untuk kegunaan sistem? Apa harapan untuk waktu pelatihan? Apa jenis dokumentasi *online* atau *hardcopy* yang dibutuhkan?

Contoh tabel penilaian masalah dapat dilihat pada Tabel 3.3.

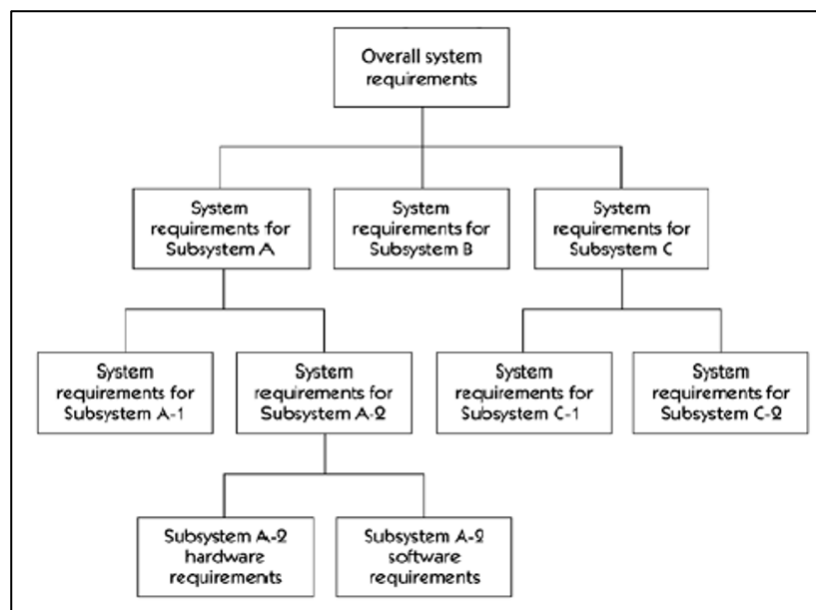
Tabel 3.3. Contoh Tabel Penilaian Masalah

Permasalahan	Penyebab	Dampak Masalah	Mempengaruhi	Solusi Saat Ini	Solusi yang Diusulkan	Prioritas
Perkembangan proyek sulit dimonitor	Belum ada pemberitahuan status setiap task dalam proyek	Proyek terlambat jika ada task yang terlambat	Project Manager	Menanyakan status task secara manual melalui WhatsApp	Membangun Sistem Manajemen Proyek yang disertai fitur notifikasi	Tinggi
...	...	...	...	...	...	...

c. Pendefinisian Solusi

Pemodelan solusi sistem ini merujuk pada *requirement* yang didapatkan dari hasil analisis menggunakan alat/*tools* yang dipakai, seluruh *requirement* kemudian dijelaskan dalam model yang lebih spesifik.

Semakin banyaknya *requirements* yang harus didokumentasikan akan semakin susah untuk mengelolanya. Oleh karena itu, kemampuan dalam mengorganisasi *requirements* yang sudah diidentifikasi menjadi mutlak diperlukan. Salah satu cara yang dapat digunakan untuk organisasi sistem/*software requirements* adalah dengan visualisasi seperti diagram pohon pada Gambar 3.1.



Gambar 3.1. Diagram Pohon *System Requirement*

Setelah tahapan menetapkan strategi mengelola *requirements* yang telah diidentifikasi, maka tahapan terpenting selanjutnya adalah mengawal dokumen ini terhadap perubahan yang akan terjadi saat pengembangan.

1. Cakupan Sistem

Ketika pernyataan masalah telah disetujui dan *user/stakeholder* sudah teridentifikasi, langkah selanjutnya adalah mendefinisikan sistem dimana

masalah itu berada. Dua hal penting yang tidak boleh dilupakan adalah setelah memahami masalah maka langkah berikutnya adalah mempertimbangkan solusi potensial terhadap masalah tersebut. Sekarang kita memasuki masa transisi di antara dua hal tersebut, yakni menentukan cakupan dari sistem solusi.

Dalam kebanyakan kasus, cakupan sistem sangat jelas. Namun, jika sistem semakin kompleks (seperti misalnya banyak terkait dengan sistem yang lain) maka cakupan sistem menjadi tidak begitu jelas. Analisis sistem harus menentukan apakah data dibagikan pada beberapa aplikasi, apakah aplikasi baru terdistribusi ke beberapa komputer klien yang lain, dan siapa pengguna.

Pendekatan *systemic thinking* juga sangat membantu dalam menentukan cakupan sistem solusi. Bagaimana menentukan cakupan? Berikut pertanyaan yang dapat membantu untuk ini:

- Siapa yang akan memasok, menggunakan, atau menghapus informasi dari sistem?
- Siapa yang akan mengoperasikan sistem?
- Siapa yang akan melakukan pemeliharaan sistem?
- Di mana sistem akan digunakan?
- Dari mana sistem mendapatkan informasinya?
- Sistem eksternal apa lagi yang akan berinteraksi dengan sistem?

2. Batasan Sistem

Sebelum masuk ke tahapan berikutnya dalam pengembangan sistem, analisis perlu mempertimbangkan *constraint* atau batasan dari solusi yang akan dibangun. Segala macam sumber batasan harus dipertimbangkan. Diantaranya, jadwal, ROI, anggaran untuk peralatan dan SDM, isu lingkungan, OS, *database*, sistem komputer yang ada, isu teknis, isu politik, dan sebagainya. Beberapa contoh hal yang harus diperhatikan terkait batasan sistem adalah sebagai berikut.

Source	Sample Considerations
Economics	• What financial or budgetary constraints apply? • Are there costs of goods sold or any product pricing considerations? • Are there any licensing issues?
Politics	• Do internal or external political issues affect potential solutions? • Are there any interdepartmental problems or issues?
Technology	• Are we restricted in our choice of technologies? • Are we constrained to work within existing platforms or technologies? • Are we prohibited from using any new technologies? • Are we expected to use any purchased software packages?
Systems	• Is the solution to be built on our existing systems? • Must we maintain compatibility with existing solutions? • What operating systems and environments must be supported?
Environment	• Are there environmental or regulatory constraints? • Are there legal constraints? • What are the security requirements? • What other standards might restrict us?
Schedule and resources	• Is the schedule defined? • Are we restricted to existing resources? • Can we use outside labor? • Can we expand resources? Temporarily? Permanently?

Gambar 3.2. Pertimbangan Batasan Sistem

Jika *requirements* yang nantinya harus dijawab sudah ditentukan, maka seberapa banyak sumber daya yang dikerahkan akan mempengaruhi seberapa pendek waktu yang diperlukan. Semakin banyak sumber daya yang dikerahkan untuk menjawab user *needs/requirements* maka semakin pendek waktu yang diperlukan untuk menyelesaikannya. Namun, yang perlu dicatat disini adalah bahwa ketika anda bertemu pada masalah keterlambatan proyek, penambahan sumber daya justru akan semakin memperlambat tercapainya tujuan proyek (Brooks' law).

Langkah awal untuk menentukan ruang lingkup proyek adalah dengan menetapkan *high-level requirements baseline*. Salah satu strategi untuk menetapkan *high-level requirements baseline* adalah dengan menggunakan teknik penentuan prioritas *requirements*. Dengan menggunakan skala *critical-important-useful*, analis dapat menentukan prioritas *requirements* yang harus dipenuhi.

Selain atribut untuk prioritas, analis juga bisa memberikan atribut yang terkait dengan *effort* dan risiko yang berhubungan dengan *features/requirements* sistem. Jika, analis mampu memberikan penilaian pada atribut *effort* dan risiko, maka untuk *requirements* yang *critical* diperlukan strategi yang sesuai dengan nilai atribut *effort* dan risiko.

Setelah ruang lingkup berhasil didefinisikan, tantangan terberat berikutnya adalah bagaimana mengelolanya. Kemampuan seorang analis dalam merangkul *user/stakeholder* menjadi kunci di sini. Selain itu, kemampuan negosiasi, penerapan strategi komunikasi yang efektif, dan kecerdasan politis juga menjadi komoditas utama bagi seorang analis.

d. Alat Bantu Analisis

1. Analisis Permasalahan

Kegiatan identifikasi masalah adalah bagian dari proses pemecahan masalah yang idealnya harus jelas prosesnya mulai dari definisi masalah serta membedakan antara akar penyebab masalah, gejala dan efek. Definisi masalah adalah titik awal dari identifikasi masalah.

Definisi masalah terdiri dari dua bagian, yaitu definisi awal masalah yang merupakan bagian awal dari kegiatan dengan menggunakan *tool* kerangka PIECES, kemudian masuk ke “pembedahan” masalah secara lebih rinci, membahas submasalah dan aspek-aspek yang relevan dengan masalah. Penggunaan Pohon Masalah, Pohon Hipotesis, dan Pohon Isu dapat digunakan untuk membantu proses kegiatan “pembedahan” masalah. Bahan utama

dalam mengidentifikasi masalah bisa berupa *Business Process Diagram* atau SOP (*Standard Operating Procedure*) di satuan organisasi.

a) Kerangka PIECES

Berikut adalah kerangka untuk identifikasi masalah, kesempatan atau peluang, dan perintah, yang dikenal dengan kerangka PIECES *Wetherbe*.

- P** kebutuhan untuk mengoreksi atau memperbaiki *performance*
- I** kebutuhan untuk mengoreksi atau memperbaiki *information*
- E** kebutuhan untuk mengoreksi atau memperbaiki *economics*, mengendalikan biaya, atau meningkatkan keuntungan
- C** kebutuhan untuk mengoreksi atau memperbaiki *Control* atau keamanan
- E** kebutuhan untuk mengoreksi atau memperbaiki *Efficiency* orang, mesin (komputer) dan proses
- S** kebutuhan untuk mengoreksi atau memperbaiki *Services*/layanan ke pelanggan, pemasok, rekan kerja, karyawan, dan lain-lain.

Lebih lengkapnya dapat diuraikan sebagai berikut:

- 1) ***Performance*** (kinerja), peningkatan terhadap kinerja (hasil kerja) sistem yang baru sehingga menjadi lebih efektif. Kinerja dapat diukur dari *throughput* dan *response time*. *Throughput* adalah jumlah dari pekerjaan yang dapat dilakukan suatu saat tertentu. *Response time* adalah waktu yang dibutuhkan untuk menanggapi suatu pekerjaan.
 - i. Produksi – jumlah kerja selama periode waktu tertentu
 - ii. Waktu respons – penundaan rata-rata antara transaksi atau permintaan dengan respons ke transaksi atau permintaan tersebut.
- 2) ***Information*** (informasi), peningkatan terhadap kualitas informasi yang disajikan
 - i. *Output*

- Kurangnya informasi yang relevan
- Terlalu banyak informasi – “kelebihan informasi”
- Informasi yang tidak dalam format yang berguna
- Informasi yang tidak akurat
- Informasi yang tidak tepat waktunya untuk penggunaan selanjutnya

ii. Input

- Data tidak ditangkap pada waktunya
- Data tidak ditangkap secara akurat – terdapat *error*
- Data sulit ditangkap
- Data ditangkap secara berlebihan – data yang sama ditangkap lebih dari sekali

iii. Data Tersimpan

- Data disimpan secara berlebihan dalam banyak *file* dan/atau *database*
- Integrasi data yang jelek
- Data tersimpan tidak akurat
- Data tidak aman dari kecelakaan atau vandalisme
- Data tidak diorganisasikan dengan baik
- Data tidak fleksibel – tidak mudah untuk memenuhi kebutuhan informasi baru dari data tersimpan.
- Data tidak dapat diakses

3) ***Economy*** (ekonomis), peningkatan terhadap manfaat-manfaat atau keuntungan-keuntungan atau penurunan-penurunan biaya yang terjadi

i. Biaya

- Biaya tidak diketahui
- Biaya tidak dapat dilacak ke sumber
- Biaya terlalu tinggi

ii. Keuntungan

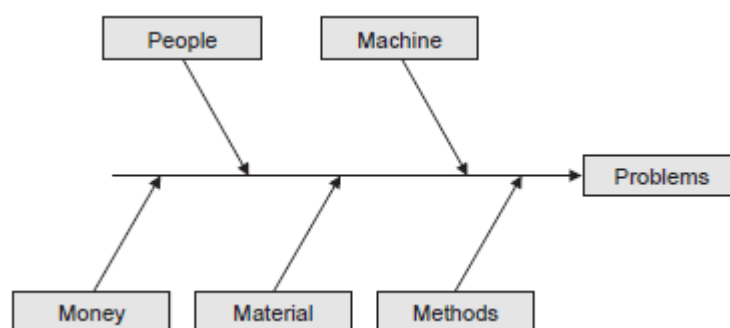
- Profit yang didapat
 - Pangsa pasar baru
- 4) **Control** (pengendalian), peningkatan terhadap pengendalian untuk mendeteksi dan memperbaiki kesalahan-kesalahan serta kecurangan-kecurangan yang akan terjadi
- i. Keamanan atau kontrol terlalu lemah
- Input data tidak diedit dengan cukup
 - Kejahatan (misalnya, penggelapan atau pencurian) terhadap data
 - Etika dilanggar pada data atau informasi – mengacu pada data atau informasi yang mencapai orang-orang yang tidak mempunyai wewenang
 - Data tersimpan secara berlebihan tidak konsisten dalam *file* atau *database* yang berbeda
 - Peraturan atau panduan privasi data dilanggar (atau dapat dilanggar)
 - *Error* pemrosesan terjadi (oleh manusia, mesin atau perangkat lunak)
 - *Error* pembuatan keputusan
- ii. Kontrol atau keamanan berlebihan
- Prosedur birokratis memperlambat sistem
 - Pengendalian mengganggu para pelanggan atau karyawan
 - Pengendalian berlebihan menyebabkan penundaan pemrosesan
- 5) **Efficiency** (efisiensi), peningkatan terhadap efisiensi operasi. Efisiensi berbeda dengan ekonomis. Bila ekonomis berhubungan dengan jumlah sumber daya yang digunakan, efisiensi berhubungan dengan bagaimana sumber daya tersebut digunakan dengan pemborosan yang paling minimum. Efisiensi dapat diukur dari *output* dibagi dengan input.

- i. Orang, mesin atau komputer
 - ii. Usaha yang dibutuhkan untuk tugas-tugas terlalu berlebihan
 - iii. Material yang dibutuhkan untuk tugas-tugas terlalu berlebihan
- 6) **Services** (pelayanan), peningkatan terhadap pelayanan yang diberikan oleh sistem
- i. Sistem menghasilkan produk yang tidak akurat
 - ii. Sistem menghasilkan produk yang tidak konsisten
 - iii. Sistem menghasilkan produk yang tidak dapat dipercaya
 - iv. Sistem tidak mudah dipelajari
 - v. Sistem tidak mudah digunakan
 - vi. Sistem tidak kompatibel

Semua komponen dari PIECES harus teridentifikasi dengan jelas sehingga definisi permasalahan awal sudah *clear* dan telah disepakati bersama. Hasil dari analisis berupa pernyataan masalah yang narasinya merupakan pernyataan negatif, misalnya: kurangnya, lambatnya, mahalnyanya, belum optimalnya, dan lain-lain.

b) Diagram *Fishbone*

Diagram *Fishbone* atau Diagram Rantai Sebab-Akibat (disebut juga Ishikawa atau analisis akar penyebab) menyediakan peta visual dari faktor penyebab yang berkontribusi atau berdampak terhadap masalah tertentu. Bentuknya seperti pada Gambar 3.3 berikut ini.

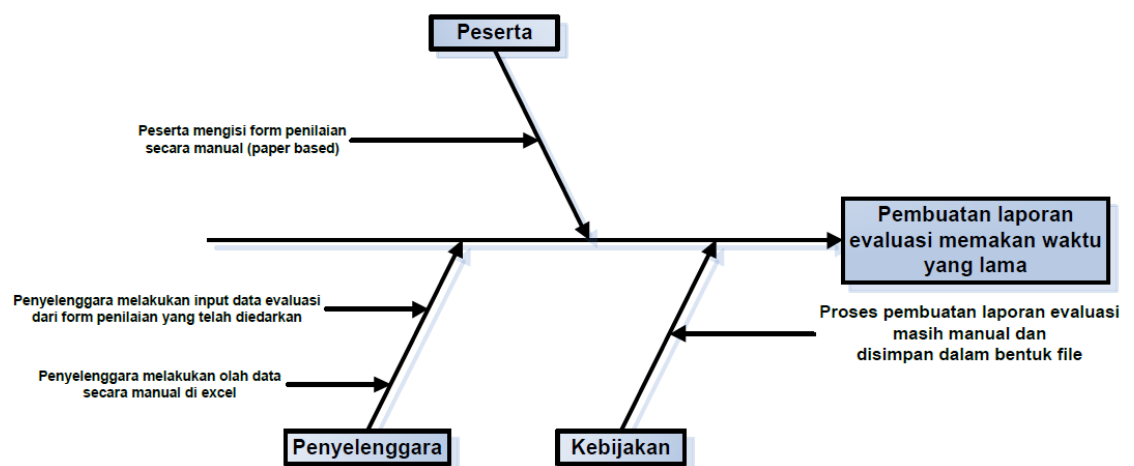


Gambar 3.3. Diagram *Fishbone*

Langkah-langkah dalam membuat Diagram *Fishbone*:

1. Masukkan pernyataan masalah hasil dari analisis menggunakan kerangka PIECES ke dalam kepala ikan (kotak "*problem*"). Dalam hal ini pernyataan masalah disebut sebagai "*akibat*".
2. Cari akar permasalahan yang bersumber dari faktor-faktor manajemen (6M: *Manpower/people*, *Money*, *Machine*, *Material*, *Method*, dan *Market*), bisa juga menambahkan faktor lain seperti *Information* dan *Environment*. Dapat juga menggunakan faktor-faktor lain sesuai dengan referensi yang dipakai.
3. Masukkan narasi sebab (*causes*) yang muncul dari masing masing faktor.
4. Tuliskan narasi sebab yang lebih mendalam (*subcauses*) jika ada ke dalam *causes*.

Contoh Diagram *Fishbone* dapat dilihat pada Gambar 3.4.

Gambar 3.4. Contoh Diagram *Fishbone*

2. Analisis *Stakeholders* (RACI)

Stakeholder proyek merupakan individu-individu dan organisasi yang terlibat secara aktif dalam proyek, atau kepentingannya dapat terpengaruh sebagai akibat dari proses eksekusi dan keberhasilan sebuah proyek. Tim

manajemen proyek harus mengidentifikasi para *stakeholder*, menentukan apa saja kebutuhan-kebutuhan dan ekspektasi mereka, kemudian mengelola dan mempengaruhi ekspektasi tersebut untuk memastikan keberhasilan sebuah proyek.

Salah satu alat yang dapat digunakan dalam analisis *stakeholders* adalah Matriks RACI (*Responsible, Accountable, Consulted, dan Informed*).

Matriks RACI digunakan untuk memperjelas peran *stakeholder* yang bertanggung jawab dalam masing-masing tugas (*tasks*), *milestone* dan keputusan yang diambil sepanjang berjalannya sebuah proyek.

Project tasks	Senior Analyst	Project Manager	Head of Design	SVP Finance	SEO Lead	Sales Director	Senior Management
Phase 1: Research							
Econometric model	R	I	I	A	C	I	I
Strategic framework	A	I	I	R	I	I	C
Risk factors	R	I	I	A	I	I	I
Phase 2: Structure							
Product specs	I	A	R	I	C	C	C
Design wireframe	I	C	R	I	C	I	C
User journey	I	C	R	I	C	C	C
User experience testing	I	C	R	I	C	C	C
Evaluation framework	I	R	C	I	C	I	C
Development backlog	I	R	C	I	C	I	C
Delivery roadmap	C	R	A	C	C	C	I

Gambar 3.5. Contoh Matriks RACI

R, A, C, I adalah peran dari *stakeholders* yaitu:

Responsible

Responsible merujuk pada orang yang ditugaskan langsung untuk menyelesaikan sebuah tugas (*tasks*) atau membuat hasil dari tugas tersebut. Orang ini biasanya sebagai pengembang/pembuat (*developers*) program aplikasi. Orang yang sebagai *responsible* biasanya ada dalam tim proyek, bisa satu atau beberapa orang.

Accountable

Accountable merujuk pada orang yang mendelegasikan tugas dan mereviu semua pekerjaan dalam proyek. *Accountable* memastikan tim (orang *responsible*) bekerja sesuai tujuan dan menyelesaikan pekerjaan tepat waktu. Setiap tugas (*tasks*) hanya memiliki satu orang sebagai *accountable*.

Consulted

Consulted merujuk pada orang yang memberikan input dan *feedback* pada pekerjaan yang sedang berjalan. Mereka punya kepentingan karena hasil dari proyek dapat berpengaruh pada pekerjaan mereka.

Informed

Informed merujuk pada orang yang perlu dimasukkan dalam lingkaran sebuah proyek tetapi bukan sebagai *Consulted*, bukan pula yang berkecimpung secara detail dalam tugas. Mereka perlu tau apa yang terjadi yang dapat berefek pada pekerjaan mereka. Orang-orang ini biasanya di luar tim bahkan berada pada departemen yang berbeda. Mereka biasanya pimpinan atau pejabat-pejabat dalam organisasi.

Contoh sederhana dalam memahami RACI sebagai berikut:

Andi sedang membuat program aplikasi A yang akan diintegrasikan dengan program aplikasi B yang telah dibuat oleh Indri. Deni adalah manajer proyek dan Susi adalah Manajer Pemasaran Produk. Dalam hal ini untuk pembuatan program aplikasi A, Andi berstatus *Responsible*, Deni adalah *Accountable*, dan Indri dibutuhkan sebagai *Consulted*, karena program aplikasinya akan diintegrasikan oleh Andi. Terakhir Susi sebagai *Informed* pada saat program aplikasi sudah siap digunakan.

3. Solusi Sistem

Untuk menggambarkan solusi sistem yang diajukan dengan ringkas dan jelas diperlukan beberapa alat pemodelan. Pemodelan yang bisa digunakan antara lain adalah Pemodelan dengan *Flowchart* dan BPMN.

a) *Flowchart*

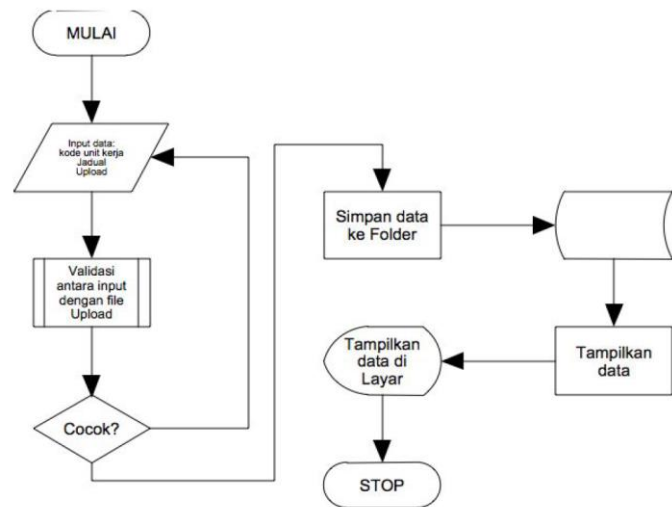
Flowchart atau bagan alur adalah diagram yang menampilkan langkah-langkah dan keputusan untuk melakukan sebuah proses dari suatu program. *Flowchart* ini menggambarkan secara rinci prosedur dari proses program. *Flowchart* program terdiri dari dua macam, antara lain: *flowchart* logika program (*program logic flowchart*) dan *flowchart* program komputer terinci (*detailed computer program flowchart*). Untuk menampilkan solusi sistem kita gunakan *flowchart* program komputer terinci.

Simbol-simbol *flowchart* adalah sebagai berikut:

Simbol	Maksud	Simbol	Maksud
	Terminal (START, END)		Titik sambungan pada halaman yang sama
	Input/Output (READ, WRITE)		Titik konektor yang berada pada halaman lain
	Proses (menyatakan assignment statement)		Call (Memanggil subprogram)
	Decision (YES, NO)		Dokumen
	Display		Stored Data
	Alur proses		Preparation (Pemberian nilai awal suatu variabel)

Gambar 3.6. Simbol-simbol dalam *Flowchart*

Berikut ini, pada Gambar 3.7, diberikan contoh pembuatan *flowchart* untuk program aplikasi entri data.

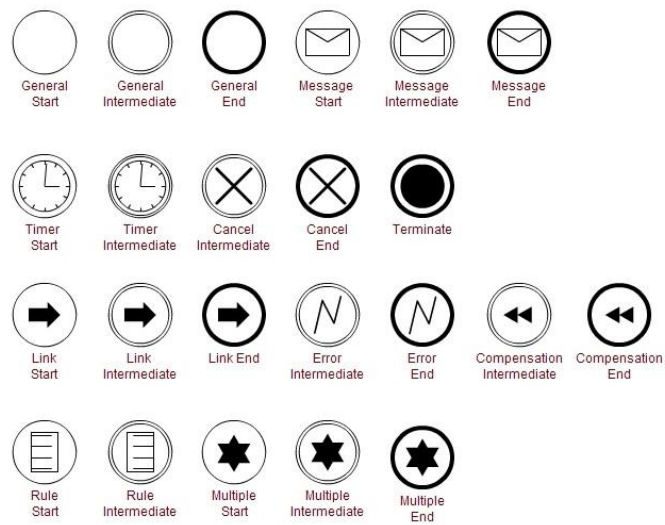
Gambar 3.7. Contoh *Flowchart* Entri Data

b) Pemodelan dengan BPMN

BPMN adalah singkatan dari *Business Process Modeling Notation*. Tujuan pemodelan BPMN adalah memberikan gambaran yang jelas tentang proses dari awal hingga akhir. pemodelan BPMN memuat jalur visual yang menunjukkan urutan aktivitas proyek yang diperlukan untuk berpindah dari akhir sebuah proses ke proses lainnya.

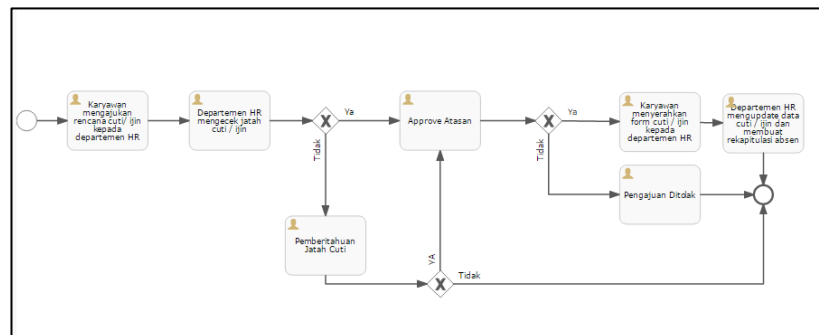
Simbol-simbol BPMN adalah sebagai berikut:

Shape	Element/Object
	Event
	Task/Activity
	Gateway
	Sequence Flow



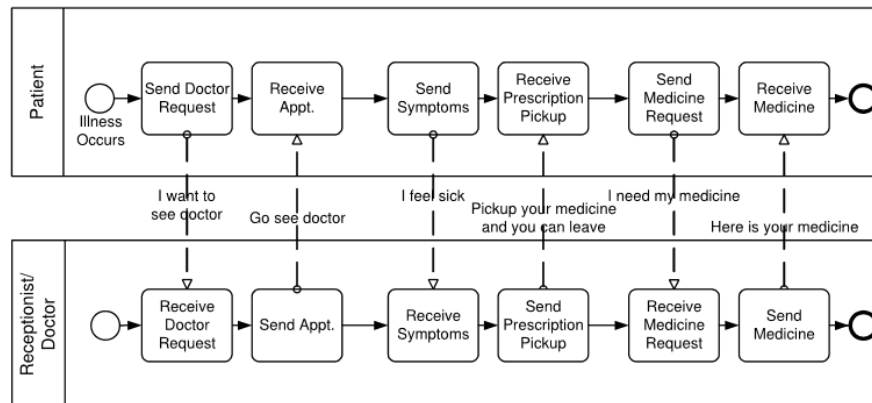
Gambar 3.8. Simbol-simbol dalam BPMN

BPMN dapat menjelaskan suatu sistem/proses secara utuh ataupun sebuah proses kolaborasi dengan beberapa entitas (*user*). Contoh BPMN untuk suatu proses secara utuh dapat dilihat pada Gambar 3.9.



Gambar 3.9. Contoh BPMN Proses

Contoh BPMN untuk suatu proses kolaborasi dengan beberapa entitas dapat dilihat pada Gambar 3.10.



Gambar 3.10. Contoh BPMN Proses Kolaborasi

e. Studi Kelayakan

Kelayakan adalah ukuran akan seberapa menguntungkan atau seberapa praktis pengembangan sistem informasi terhadap organisasi. Analisis kelayakan adalah proses pengukuran kelayakan dengan beberapa variabel. Kelayakan sebaiknya diukur sepanjang tahapan proses pengembangan sistem. Lingkup dan kompleksitas proyek yang tampaknya dapat dikerjakan dengan mudah, dapat berubah setelah persoalan dan kesempatan awal dianalisis secara lengkap atau setelah sistem didesain. Jadi, proyek yang layak dikerjakan di saat tahapan awal (studi kelayakan awal), dapat menjadi tidak layak pada tahapan selanjutnya.

1. Identifikasi solusi kandidat

Untuk mendapatkan solusi sistem yang layak dan tepat, maka diperlukan beberapa kandidat sistem sebagai bahan untuk perbandingan dan mengukur kelayakan sistem.

Alat yang digunakan adalah Matriks Solusi Kandidat. Matriks ini berisi minimal 3 (tiga) kandidat sistem dengan beberapa karakteristik yang telah ditetapkan.

Bentuk Matriks Solusi Kandidat dapat dilihat pada Tabel 3.4.

Tabel 3.4. Matriks Solusi Kandidat

Karakteristik	Kandidat 1	Kandidat 2	Kandidat 3
Bagian sistem yang dikomputerisasi			
Keuntungan			
Kebutuhan Perangkat Keras			
Kebutuhan Perangkat Lunak			
Metode Pemrosesan Data			
Alat Output dan implikasinya			
Alat Input dan implikasinya			
Alat penyimpanan dan implikasinya			

2. Analisis Solusi Kandidat

Setiap solusi kandidat harus dianalisis kelayakannya. Sebagai kriteria penentu layak atau tidaknya suatu solusi, kita dapat menggunakan faktor kelayakan TELOS.

a) Kelayakan Teknis

Pertanyaan berikut dapat menjadi pertimbangan dalam menentukan kelayakan teknis:

- 1) Apakah teknologi atau solusi yang diajukan cukup praktis?
- 2) Apakah saat ini kita telah mempunyai teknologi yang memadai?
- 3) Apakah kita mempunyai pakar teknis yang memadai?

b) Kelayakan Ekonomis

Hal mendasar dalam banyak proyek adalah kelayakan ekonomis. Selama fase awal proyek (analisis kelayakan umum), analisis kelayakan ekonomis hanyalah menentukan apakah manfaat yang diperoleh dari menyelesaikan persoalan tersebut cukup berharga. Biaya secara praktis tidak mungkin diperkirakan pada tahap itu, karena pertimbangan persyaratan atau kebutuhan sistem belum dimasukkan. Akan tetapi, segera setelah persyaratan atau kebutuhan sistem didapat dan dimasukkan dalam

pertimbangan, analisis dapat memperkirakan biaya dan keuntungan dari suatu solusi dengan menggunakan analisis *cost-benefit*.

c) Kelayakan Legal

Kelayakan legal menyangkut rentang yang luas dari kepentingan yang menyangkut kontrak, liabilitas, pelanggaran, dan banyak jebakan lain yang sering tidak diketahui oleh staf teknis.

d) Kelayakan Operasional

Kriteria kelayakan operasional mengukur tingkat kepentingan masalah atau tingkat penerimaan solusi. Ada dua aspek kelayakan operasional yang harus dipertimbangkan:

- 1) Apakah masalah itu cukup berharga untuk diselesaikan, atau akankah solusi itu bermanfaat untuk menyelesaikan suatu masalah?
- 2) Bagaimana pendapat pengguna akhir dan manajemen mengenai masalah atau solusi itu?

e) Kelayakan Jadwal

Tenggat waktu yang dibutuhkan dalam suatu proyek pengembangan sistem informasi, terkadang sangat menentukan sukses atau gagalnya suatu proyek. Banyak proyek pengembangan sistem informasi yang gagal karena masalah jadwal. Jadwal yang tidak tepat akhirnya dapat mempengaruhi penyelesaian proyek yang kurang baik. Lebih baik penyelesaian proyek dengan baik dan benar, meskipun dari sisi jadwal mengalami keterlambatan, daripada penyelesaian proyek yang tepat waktu tapi sistem tidak dapat berfungsi dengan baik dan benar. Melewati tenggat waktu merupakan hal yang problematis, namun sistem yang tidak berfungsi dengan baik dan benar dapat menjadi malapetaka. Ini merupakan pilihan mana yang lebih baik dari dua hal yang buruk.

3. Membandingkan solusi kandidat

Kita telah membahas kriteria kelayakan suatu alternatif solusi yang dapat digunakan untuk menentukan alternatif solusi mana yang paling layak untuk dipilih. Berikutnya adalah teknik untuk melakukan perbandingan secara cepat di antara solusi-solusi kandidat berdasarkan faktor kelayakan TELOS di atas dengan menggunakan matriks, yaitu Matriks Analisis Kelayakan (*Feasibility Analysis Matrix*). Matriks ini melengkapi matriks sistem kandidat dengan sebuah analisis dan peringkat sistem kandidat. Kolom matriks yang berhubungan dengan solusi kandidat sama dengan yang ditunjukkan dalam matriks sistem kandidat. Satu yang membedakan matriks ini dengan matriks sistem kandidat adalah adanya kolom *weight*, yang merupakan pembobotan nilai pada setiap kriteria kelayakan. Baris matriks berhubungan dengan kriteria kelayakan, ditambahkan satu baris dalam setiap kriteria berupa penilaian kelayakan untuk tiap kandidat. Setelah penetapan penilaian untuk tiap kriteria pada setiap kandidat, berikutnya adalah penetapan peringkat atau nilai akhir dicatat pada baris terakhir. Matriks Analisis Kelayakan dapat dilihat pada Tabel 3.5.

Tabel 3.5. Matriks Analisis Kelayakan

Kriteria Kelayakan	Weight	Kandidat 1	Kandidat 2	Kandidat 3
Teknis	20%			
		Skor:	Skor:	Skor:
Ekonomis	25%			
		Skor:	Skor:	Skor:
Legal	10%			
		Skor:	Skor:	Skor:
Operasional	30%			
		Skor:	Skor:	Skor:
Jadwal	15%			
		Skor:	Skor:	Skor:
Skor Total	100%			

Kita telah mendiskusikan fakta bahwa setiap alternatif solusi dapat dievaluasi menurut lima kriteria: kelayakan teknis, ekonomis, legal, operasional dan jadwal. Bagaimana seorang analis mengambil solusi terbaik? Tidak mudah. Masalah operasional dan ekonomis sangat sering bertentangan. Misalnya, solusi yang menyajikan hasil operasional terbaik bagi pengguna akhir bisa saja yang paling mahal, dan oleh karena itu, paling tidak layak secara ekonomis.

3.2. Rangkuman

Memahami kebutuhan *user* dan *stakeholder* adalah kunci sukses pengembangan solusi sistem informasi yang efektif.

Analisis kebutuhan sistem informasi dilakukan menggunakan beberapa alat yaitu Kerangka PIECES dan Diagram *Fishbone* untuk analisis permasalahan dan Matriks RACI untuk analisis *stakeholder*.

Untuk menampilkan solusi sistem informasi yang akan dikembangkan menggunakan alat *Flowchart* atau BPMN.

Solusi sistem yang akan dikembangkan perlu diuji kelayakannya. Untuk itu perlu melakukan *Feasibility Study* yaitu memberikan 3 (tiga) alternatif kandidat solusi sebagai perbandingan untuk dipilih satu yang paling layak. Alat yang digunakan adalah Matriks Solusi Kandidat dan Matriks Analisis Kelayakan yang mempertimbangkan faktor TELOS.

3.3. Soal Latihan

Buatlah Diagram *Fishbone* untuk menganalisis masalah “Belum optimalnya pelaksanaan SAKIP di unit kerja BPS Provinsi/Kabupaten/Kota” tempat Anda bekerja.

3.4. Contoh Kasus

Rudi sedang membuat program aplikasi SAKIP BPS Kab. X yang akan diintegrasikan dengan program aplikasi SKP yang telah dibuat oleh Andri. Amir adalah Kepala BPS sebagai manajer proyek dibantu oleh para pejabat setingkat eselon IV. Untuk menyelesaikan program aplikasi ini diperlukan konsultasi kepada Sinta, pejabat di BPS Provinsi yang menangani SAKIP. Dari informasi di atas, Buatlah Matrik RACI dengan 4 (empat) *tasks* dan berikan keterangan untuk peran masing-masing orang dalam R, A, C, dan I.

BAB IV PERANCANGAN SISTEM INFORMASI

4.1. Uraian Materi

Perancangan sistem adalah proses mengembangkan model abstrak dari suatu sistem, dengan masing-masing model menyajikan pandangan atau perspektif yang berbeda dari sistem itu (Sommerville, 2011). Pemodelan sistem umumnya berarti mewakili sistem menggunakan beberapa jenis notasi grafis, yang sekarang hampir selalu digunakan adalah notasi dalam *Unified Modeling Language* (UML). Namun, juga dimungkinkan untuk mengembangkan model formal lainnya dari suatu sistem, biasanya sebagai spesifikasi sistem yang terperinci.

Model digunakan selama proses *requirements engineering* untuk membantu memperoleh persyaratan untuk suatu sistem, selama proses desain untuk menggambarkan sistem kepada *developer* yang mengimplementasikan sistem, dan setelah implementasi untuk mendokumentasikan struktur dan operasi sistem. Pemodelan dapat dikembangkan dari sistem yang ada dan sistem yang akan dikembangkan.

a. Pemodelan Proses

Pemodelan proses merupakan kegiatan membuat dokumentasi yang menjelaskan suatu proses sistem informasi yang kompleks ke dalam bagan, diagram, atau bentuk lainnya dengan tujuan agar lebih mudah dipahami. Pemodelan proses dapat dikembangkan dengan model yang berbeda untuk mewakili sistem dari perspektif yang berbeda. Sebagai contoh:

- Perspektif eksternal, dimana proses dimodelkan dari konteks atau lingkungan sistem.
- Perspektif interaksi, dimana proses dimodelkan dari interaksi antara sistem dan lingkungannya atau antara komponen sistem.
- Perspektif struktural, dimana proses dimodelkan dari organisasi sistem atau struktur data yang diproses oleh sistem.

- Perspektif perilaku, dimana proses dimodelkan dari perilaku dinamis sistem dan bagaimana responsnya terhadap suatu.

Pemodelan proses sistem informasi dapat mengadopsi pendekatan berorientasi pada prosedur, berorientasi pada objek, atau berorientasi pada layanan.

1. Pemodelan Berorientasi Prosedur (*Data Flow Diagram/DFD*)

a) Pengertian DFD

DFD adalah model dari sebuah sistem untuk menggambarkan pembagian sistem ke modul yang lebih kecil. Salah satu keuntungan menggunakan DFD adalah memudahkan pengguna yang kurang terbiasa dengan komputer untuk mengerti sistem yang akan dikerjakan. DFD dapat dibagi menjadi tiga bagian.

1) Diagram Konteks

Diagram konteks adalah diagram yang terdiri dari suatu proses dan menggambarkan ruang lingkup suatu sistem. Diagram konteks merupakan level tertinggi dari DFD yang menggambarkan keseluruhan sistem. Sistem dibatasi oleh *boundary*. Dalam diagram konteks hanya boleh ada satu proses dan tidak boleh ada *data store*.

2) Diagram Zero (*Overview Diagram*)

Diagram zero adalah diagram yang menggambarkan proses dari DFD. Diagram ini memberikan pandangan menyeluruh tentang sistem yang akan ditangani, menunjukkan fungsi-fungsi utama atau proses yang ada, aliran data dan entitas eksternal. Pada level ini sudah dimungkinkan adanya *data store*. Untuk proses yang tidak dirinci lagi pada level selanjutnya, symbol '*' atau 'P' (*functional primitive*) dapat digunakan pada akhir nomor proses. Keseimbangan (*balancing*) *input* dan *output* antara diagram zero dengan diagram konteks harus dijaga.

3) Diagram Rinci

Diagram rinci adalah diagram yang menguraikan proses apa yang ada dalam diagram zero atau diagram yang berada di level atasnya.

b) *Balancing* dalam DFD

Aliran data yang masuk dan keluar dari suatu proses harus sama dengan aliran data yang masuk atau keluar dari rincian proses pada level di bawahnya. Hal-hal yang perlu diperhatikan adalah sebagai berikut:

- Harus ada keseimbangan input dan *output* antara satu level dan berikutnya
- Keseimbangan antara level 0 dan 1 dilihat pada input/output dari aliran data ke atau dari terminal pada level 0 sedangkan keseimbangan antara level 1 dan 2 dilihat pada input/output dari aliran data ke/dari proses yang bersangkutan
- Nama aliran data, *data store*, dan terminal pada setiap level harus sama jika objeknya sama

c) Spesifikasi Proses

Setiap proses di DFD harus memiliki spesifikasi proses. Tanpa ini kita tidak akan mengetahui apa yang terjadi di proses tersebut. Ada banyak cara untuk menggambarkan suatu proses. Hal ini tergantung dari level mana proses tersebut berada. Metode penggambaran proses di *top level* berbeda dengan yang berada di level paling bawah. Ada beberapa istilah yang digunakan untuk spesifikasi suatu proses:

- *Mini_spec* (spesifikasi singkat)
- *Job_spec* (spesifikasi tugas)
- *Process description* (deskripsi proses), dan lain-lain.

Spesifikasi proses untuk level atas dapat menggunakan kalimat deskriptif namun pada level yang lebih rinci, yakni pada proses yang paling bawah (*functional primitive*) memerlukan spesifikasi yang lebih

terstruktur dengan menggunakan kaidah-kaidah tertentu. Spesifikasi proses akan menjadi pedoman bagi *programmer* dalam membuat program (*coding*). Metode yang digunakan untuk spesifikasi proses dapat berupa:

- Uraian proses dalam bentuk cerita
- Bahasa Indonesia/Inggris yang terstruktur
- *Decision table*
- *Decision tree*

d) Elemen Dasar DFD

1) Kesatuan Luar (*External Entity*)

Sesuatu yang berada di luar sistem, tetapi ia memberikan data ke atau dari dalam sistem disimbolkan dengan notasi kotak. *Eksternal entity* bukan termasuk bagian dari sistem. Bila sistem informasi dirancang untuk satu bagian maka bagian lainnya yang masih terkait menjadi *external entity*.

2) Arus Data (*Data Flow*)

Arus data merupakan tempat mengalirnya informasi dan digambarkan dengan garis yang menghubungkan komponen dari sistem. Arus data ditunjukkan dengan arah panah dan garis dan diberi nama atas arus data yang mengalir. Arus data ini mengalir di antara proses, *data store*, dan menunjukkan arus data dari data yang berupa masukan untuk sistem atau hasil proses sistem.

3) Proses

Proses adalah apa yang dikerjakan oleh sistem. Proses dapat mengolah data atau aliran data masuk menjadi aliran data keluar. Proses berfungsi mentransformasikan satu atau beberapa *data output* sesuai dengan spesifikasi yang diinginkan. Setiap proses memiliki satu atau beberapa masukan serta menghasilkan satu atau beberapa *data output*. Proses tersebut sering pula disebut *bubble*.

4) Simpanan Data (*Data Store*)

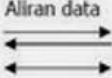
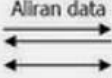
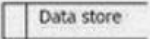
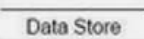
Simpanan data adalah tempat penyimpanan data yang ada di dalam sistem. *Data store* dapat disimbolkan dengan sepasang dua garis dengan salah satu sisi samping terbuka. Proses dapat mengambil data dari atau memberikan data ke *database*.

5) Kamus Data (*Data Dictionary*)

Kamus data berfungsi membantu pengguna sistem untuk mengartikan aplikasi secara detail dan mengorganisasi semua elemen data yang digunakan dalam sistem secara akurat sehingga pemakai dan analis sistem mempunyai dasar pengertian yang sama tentang input, *output*, penyimpanan, dan proses. Kamus data sering disebut juga katalog fakta tentang data dan kebutuhan-kebutuhan informasi dari suatu sistem informasi. Dengan menggunakan kamus data seorang analis dapat mendefinisikan data yang mengalir di sistem, yakni tentang data yang masuk ke sistem dan tentang informasi yang dibutuhkan oleh pengguna sistem.

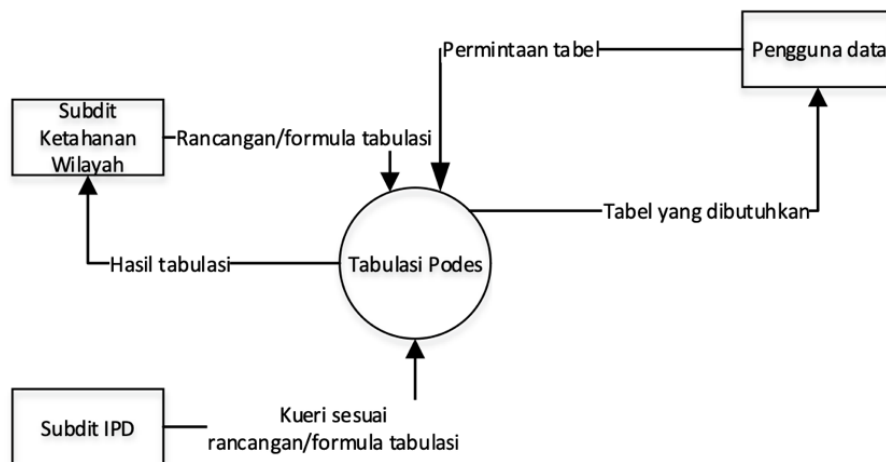
Pada tahap perancangan sistem, kamus data dibuat untuk merancang input, laporan-laporan, dan basis data. Kamus data dibuat berdasarkan arus data yang ada di DFD. Arus datanya bersifat global dan hanya ditunjukkan nama arus datanya saja.

Notasi yang digunakan pada DFD antara lain sebagai berikut.

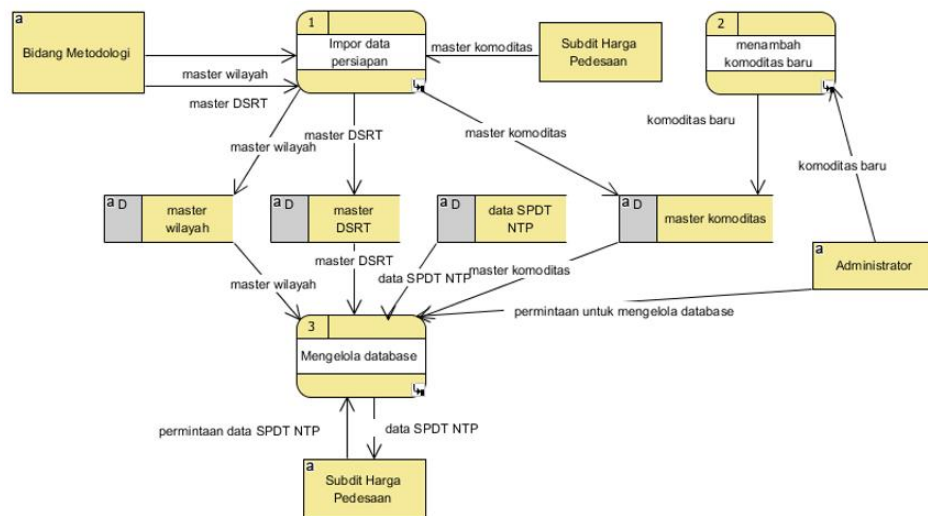
Gane/Sarson	Yourdon/De Marco	Keterangan
 Entitas Eksternal	 Entitas Eksternal	Entitas eksternal, dapat berupa orang/unit terkait yang berinteraksi dengan sistem tetapi diluar sistem
 Proses	 Proses	Orang, unit yang mempergunakan atau melakukan transformasi data. Komponen fisik tidak diidentifikasi.
 Aliran data	 Aliran data	Aliran data dengan arah khusus dari sumber ke tujuan
 Data store	 Data Store	Penyimpanan data atau tempat data direfer oleh proses.

Gambar 4.1. Notasi DFD

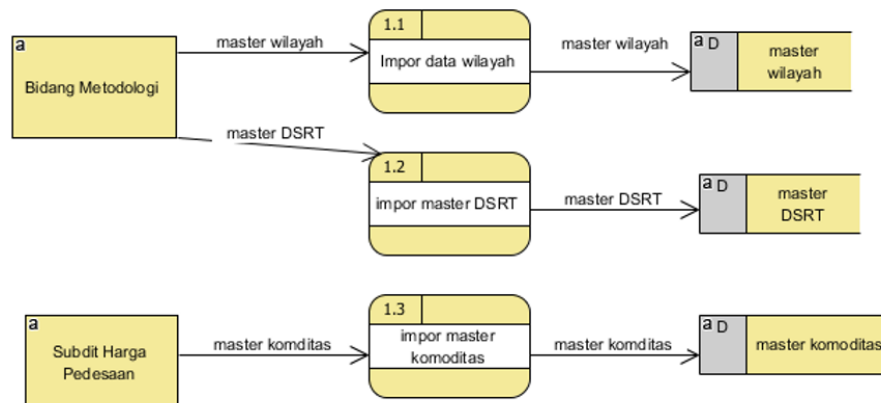
e) Contoh Hasil Pemodelan DFD



Gambar 4.2. Contoh Diagram Konteks



Gambar 4.3. Contoh Diagram Zero (Overview Diagram)



Gambar 4.4. Contoh Diagram Rinci (DFD Level n)

2. Pemodelan Berorientasi Objek (*Unified Modeling Language/UML*)

UML merupakan sekumpulan alat yang digunakan untuk melakukan abstraksi terhadap sebuah sistem atau perangkat lunak berbasis objek. UML juga menjadi salah satu cara untuk mempermudah pengembangan aplikasi yang berkelanjutan. Aplikasi atau sistem yang tidak terdokumentasi biasanya dapat menghambat pengembangan karena *developer* harus melakukan penelusuran dan mempelajari kode program. UML juga dapat menjadi alat bantu untuk mentransfer ilmu tentang sistem atau aplikasi yang akan

dikembangkan dari satu *developer* ke *developer* lainnya. Tidak hanya antar *developer*, *stakeholder* dan siapapun dapat memahami sebuah sistem dengan adanya UML.

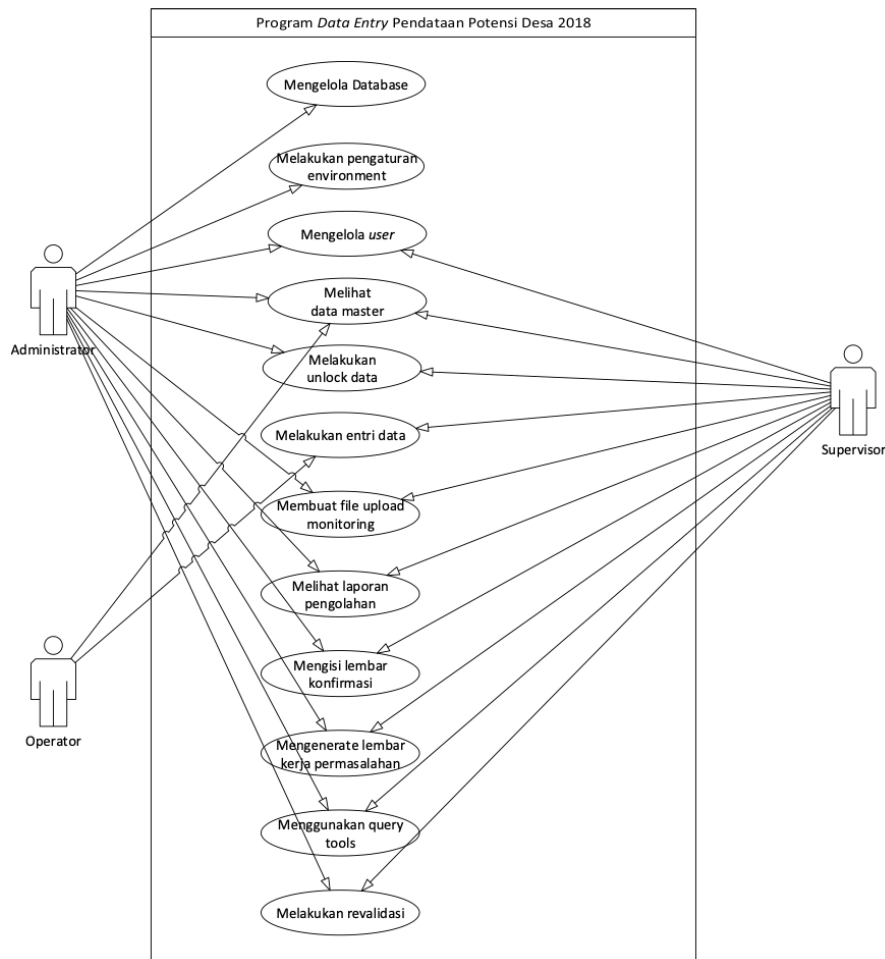
UML adalah bahasa untuk membuat spesifikasi, memvisualisasi, membangun, dan mendokumentasikan *artifacts* (bagian dari informasi yang dihasilkan oleh proses pembuatan perangkat lunak, *artifact* tersebut dapat berupa model, deskripsi, atau perangkat lunak) dari sistem perangkat lunak, seperti pada pemodelan bisnis dan sistem non perangkat lunak lainnya. Selain itu UML adalah bahasa pemodelan yang menggunakan konsep berorientasi objek. UML dibuat oleh Grady Booch, James Rumbaugh, dan Ivar Jacobson di bawah bendera Rational Software Corps. UML menyediakan notasi-notasi yang membantu memodelkan sistem dari berbagai perspektif. UML tidak hanya digunakan dalam pemodelan perangkat lunak, namun hampir dalam semua bidang yang membutuhkan pemodelan.

Beberapa jenis diagram yang digunakan dalam UML antara lain:

a) *Use Case Diagram*

Diagram *use case* menggambarkan sejumlah aktor eksternal dan hubungannya ke *use case* yang diberikan oleh sistem. *Use case* adalah deskripsi fungsi yang disediakan oleh sistem dalam bentuk teks sebagai dokumentasi dari simbol *use case* namun dapat juga dilakukan dalam *activity diagram*. *Use case* digambarkan hanya yang dilihat dari luar oleh aktor (keadaan lingkungan sistem yang dilihat user) dan bukan bagaimana fungsi yang ada di dalam sistem. Notasi yang digunakan pada diagram *use case* adalah sebagai berikut:

SIMBOL	NAMA	KETERANGAN
	<i>Actor</i>	Menspesifikasikan himpunan peran yang pengguna mainkan ketika berinteraksi dengan <i>use case</i> .
	<i>Dependency</i>	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri (<i>independent</i>) akan mempengaruhi elemen yang bergantung padanya elemen yang tidak mandiri.
	<i>Generalization</i>	Hubungan dimana objek anak (<i>descendent</i>) berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk (<i>ancestor</i>).
	<i>Include</i>	Menspesifikasikan bahwa <i>use case</i> sumber secara eksplisit.
	<i>Extend</i>	Menspesifikasikan bahwa <i>use case</i> target memperluas perilaku dari <i>use case</i> sumber pada suatu titik yang diberikan.
	<i>Association</i>	Apa yang menghubungkan antara objek satu dengan objek lainnya.
	<i>System</i>	Menspesifikasikan paket yang menampilkan sistem secara terbatas.
	<i>Use Case</i>	Deskripsi dari urutan aksi-aksi yang ditampilkan sistem yang menghasilkan suatu hasil yang terukur bagi suatu aktor.
	<i>Collaboration</i>	Interaksi aturan-aturan dan elemen lain yang bekerja sama untuk menyediakan perilaku yang lebih besar dari jumlah dan elemen-elemennya (<i>sinergi</i>).
	<i>Note</i>	Elemen fisik yang eksis saat aplikasi dijalankan dan mencerminkan suatu sumber daya komputasi.

Gambar 4.5. Notasi *Use Case Diagram*Contoh *Use Case Diagram*:Gambar 4.6. Contoh *Use Case Diagram*b) *Class Diagram*

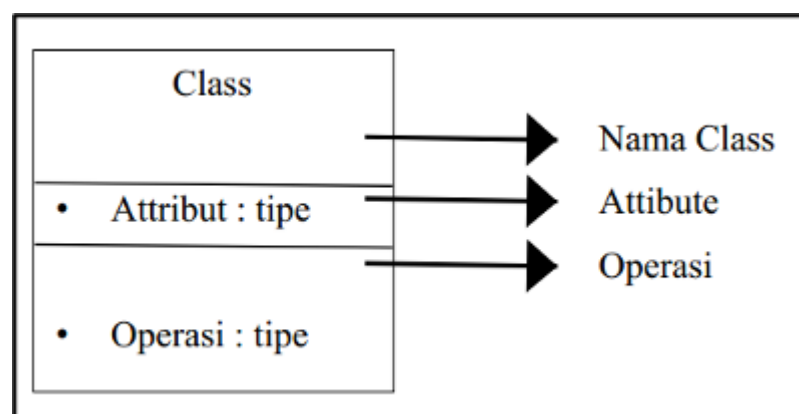
Class Diagram menggambarkan struktur statis *class* di dalam sistem. *Class* merepresentasikan sesuatu yang ditangani oleh sistem. *Class* dapat berhubungan dengan yang lain melalui berbagai cara: *associated* (terhubung satu sama lain), *dependent* (satu *class* tergantung/menggunakan *class* yang lain), *specialized* (satu *class* merupakan spesialisasi dari *class* lainnya), atau *package* (grup bersama sebagai satu unit). Sebuah sistem biasanya mempunyai beberapa *class*

diagram. Notasi yang digunakan pada *class diagram* adalah sebagai berikut:

NO	GAMBAR	NAMA	KETERANGAN
1		<i>Generalization</i>	Hubungan dimana objek anak (<i>descendant</i>) berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk (<i>ancestor</i>)
2		<i>Class</i>	Himpunan dari objek-objek yang berbagi atribut serta operasi yang sama
3		<i>Collaboration</i>	Deskripsi dari urutan aksi-aksi yang ditampilkan sistem yang menghasilkan suatu hasil yang terukur bagi suatu aktor
4		<i>Realization</i>	Operasi yang benar-benar dilakukan oleh suatu objek
5		<i>Dependency</i>	Hubungan dimana perubahan yang terjadi pada suatu element mandiri (<i>independent</i>) akan mempengaruhi elemen yang bergantung padanya elemen yang tidak mandiri
6		<i>Association</i>	Apa yang menghubungkan antara objek suatu dengan objek yang lain

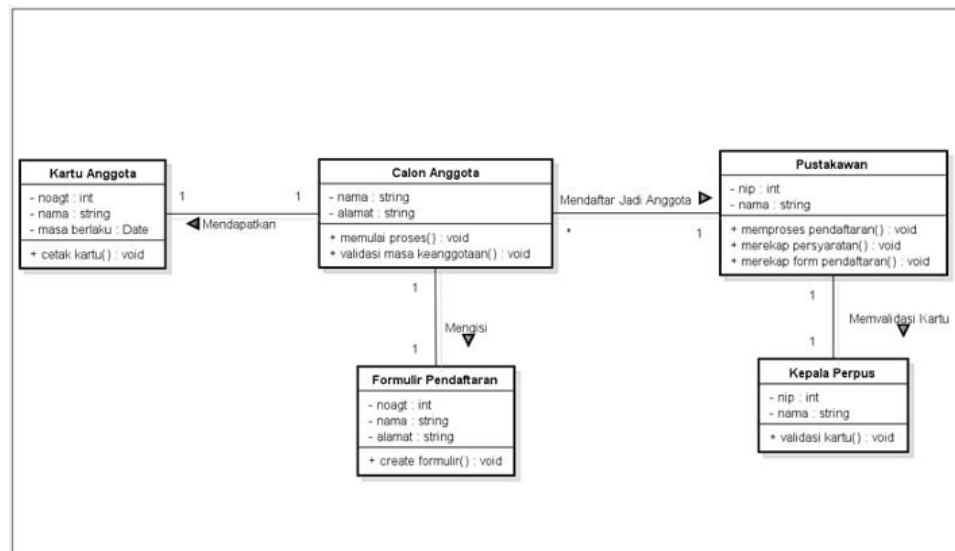
Gambar 4.7. Notasi *Class Diagram*

Struktur pada notasi *class* adalah:



Gambar 4.8. Struktur *Class* pada Notasi *Class Diagram*

Contoh *class diagram* adalah sebagai berikut:

Gambar 4.9. Contoh *Class Diagram*

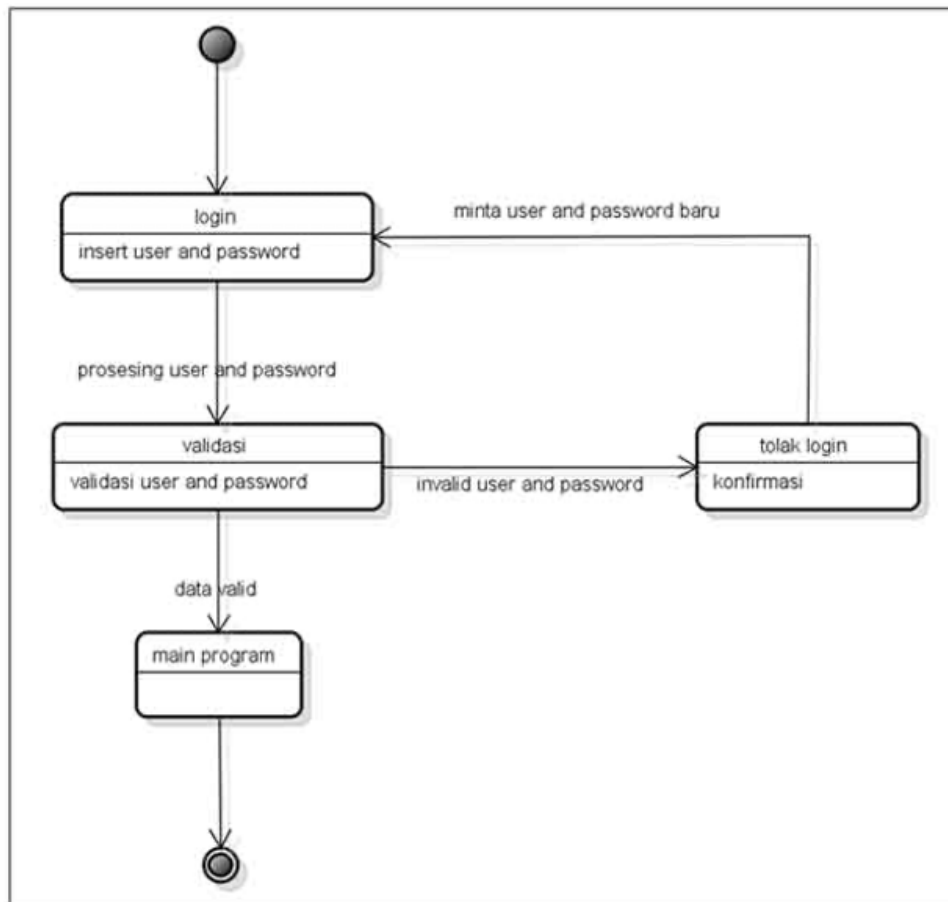
c) *State Diagram*

State Diagram menggambarkan semua *state* (kondisi) yang dimiliki oleh suatu objek dari suatu *class* dan keadaan yang menyebabkan *state* berubah. *State* dapat berupa objek lain yang mengirim pesan. *Class State* tidak digambarkan untuk semua *class*, hanya yang mempunyai sejumlah *state* yang terdefinisi dengan baik dan kondisi *class* berubah oleh *state* yang berbeda. Adapun notasi yang digunakan pada *state diagram* adalah:

Notasi	Penjelasan
	State, digambarkan berbentuk segi empat dengan sudut membulat dan memiliki nama sesuai kondisinya saat itu
	awal (start), digunakan untuk menggambarkan awal dari kejadian dalam suatu diagram statechart
	Titik akhir (end), digunakan untuk menggambarkan akhir dari kejadian dalam suatu diagram statechart
[guard]	Guard, yang merupakan syarat terjadinya transisi yang bersangkutan
	Point, digunakan untuk menggambarkan apakah akan masuk (entry point) ke dalam state atau akan keluar (exit point)
event	Event, digunakan untuk mendeskripsikan kondisi yang menyebabkan sesuatu pada state

Gambar 4.10. Notasi *State Diagram*

Contoh *state diagram*:

Gambar 4.11. Contoh *State Diagram*

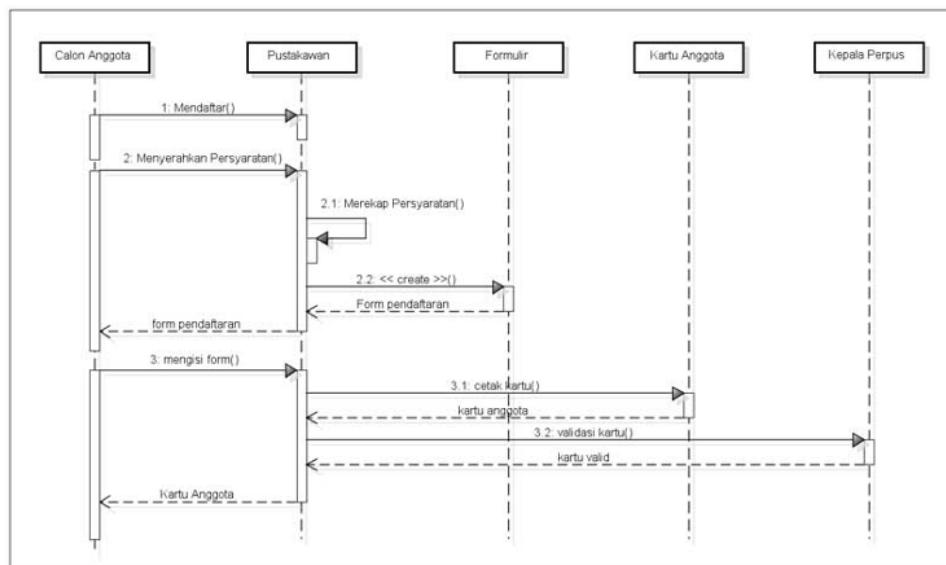
d) *Sequence Diagram*

Sequence Diagram menggambarkan kolaborasi dinamis antara sejumlah objek. Kegunaannya untuk menunjukkan rangkaian pesan yang dikirim antar objek dan juga interaksi antar objek, sesuatu yang terjadi pada titik tertentu dalam eksekusi sistem. Contoh notasi *sequence diagram* adalah sebagai berikut:

NO	GAMBAR	NAMA	KETERANGAN
1		<i>Actor</i>	Menggambarkan seseorang atau sesuatu (seperti perangkat, sistem lain) yang berinteraksi dengan sistem.
2		<i>Life Line</i>	Objek <i>entity</i> , antarmuka yang saling berinteraksi.
3		<i>Object Message</i>	Menggambarkan pesan/hubungan antar objek yang menunjukkan urutan kejadian yang terjadi.
4		<i>Message to Self</i>	Menggambarkan pesan/hubungan objek itu sendiri yang menunjukkan urutan kejadian yang terjadi.

Gambar 4.12. Notasi *Sequence Diagram*

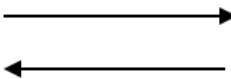
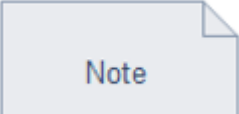
Contoh *sequence diagram*:

Gambar 4.13. Contoh *Sequence Diagram*

e) *Communication Diagram*

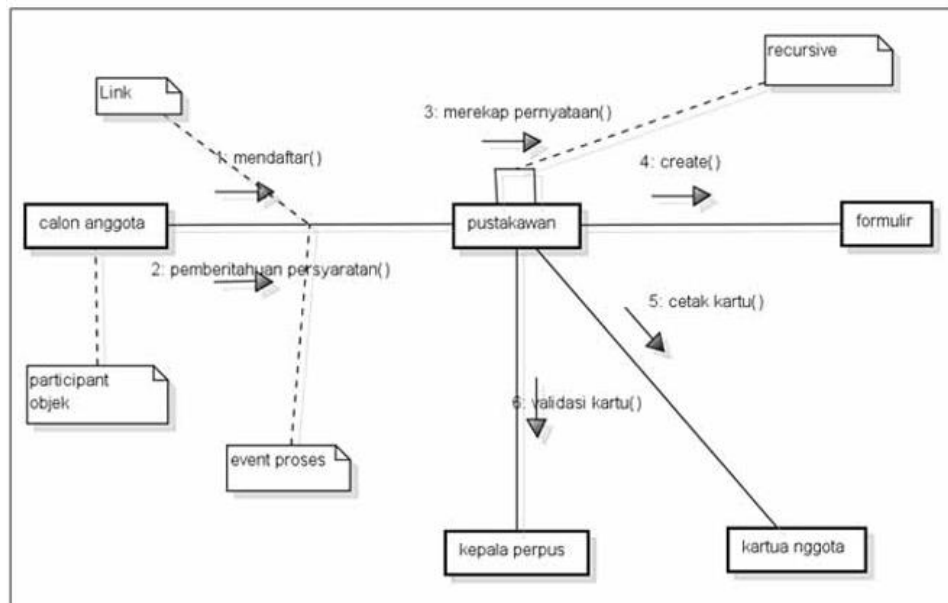
Communication Diagram menggambarkan kolaborasi dinamis seperti *sequence diagram*. Dalam menunjukkan pertukaran pesan, *collaboration diagram* menggambarkan objek dan hubungannya (mengacu ke konteks). Jika penekanannya pada waktu atau urutan maka gunakan

sequence diagrams, tapi jika penekanannya pada konteks gunakan *collaboration diagram*. Notasi pada *collaboration diagram* antara lain sebagai berikut:

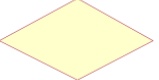
Simbol	Keterangan
	Actor
	Object instance Objek yang dibuat, melakukan tindakan, dan/atau dimusnahkan selama lifeline
	Interaksi link Merupakan indikasi bahwa objek kejadian dan berkolaborasi aktor dan pertukaran pesan.
	Sinkronis pesan Seketika sebuah komunikasi antara objek-objek yang menyampaikan informasi, dengan harapan bahwa tindakan akan dimulai sebagai hasil.
	Berisi catatan, komentar, atau informasi tertulis

Gambar 4.14. Notasi *Communication Diagram*

Contoh *communication diagram* adalah sebagai berikut:

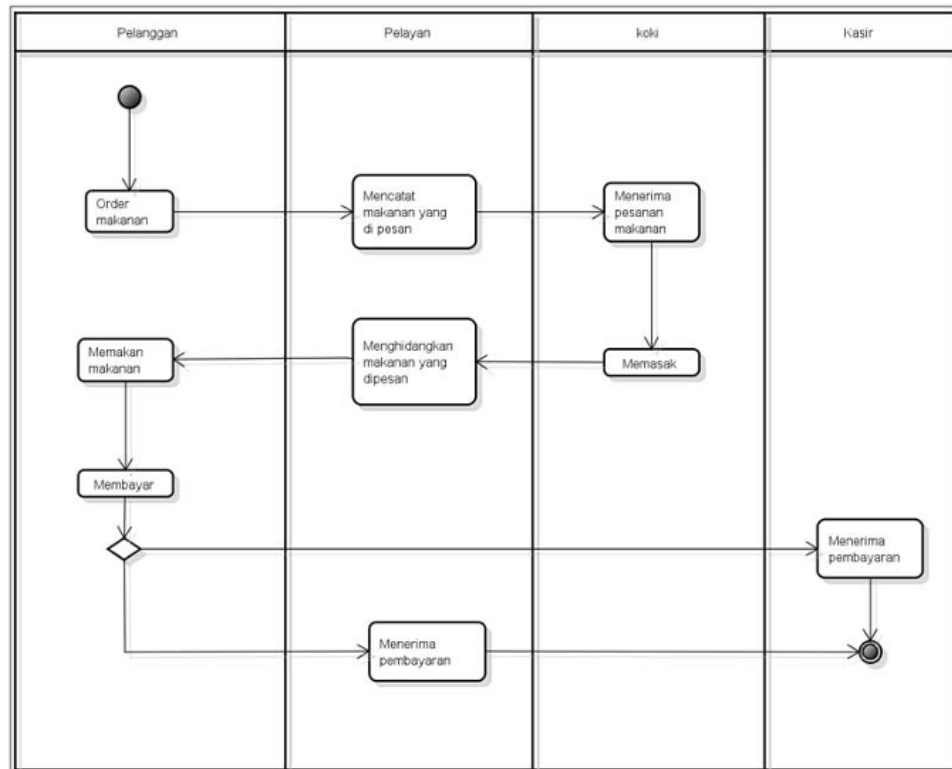
Gambar 4.15. Contoh *Communication Diagram*f) *Activity Diagram*

Activity Diagram menggambarkan rangkaian aliran dari aktivitas yang digunakan untuk mendeskripsikan aktivitas yang dibentuk dalam suatu operasi sehingga dapat juga digunakan untuk aktivitas lainnya seperti *use case* atau interaksi. Notasi pada *activity diagram* adalah:

Simbol	Keterangan
	Titik awal atau permulaan
	Titik akhir atau akhir dari aktivitas
	Aktivitas yang dilakukan oleh aktor
	<i>Decision</i> atau pilihan untuk mengambil keputusan
	Arah tanda panah alur proses

Gambar 4.16. Notasi Activity Diagram

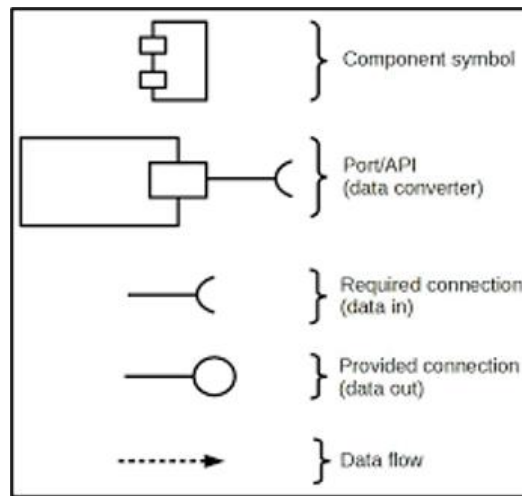
Contoh activity diagram:



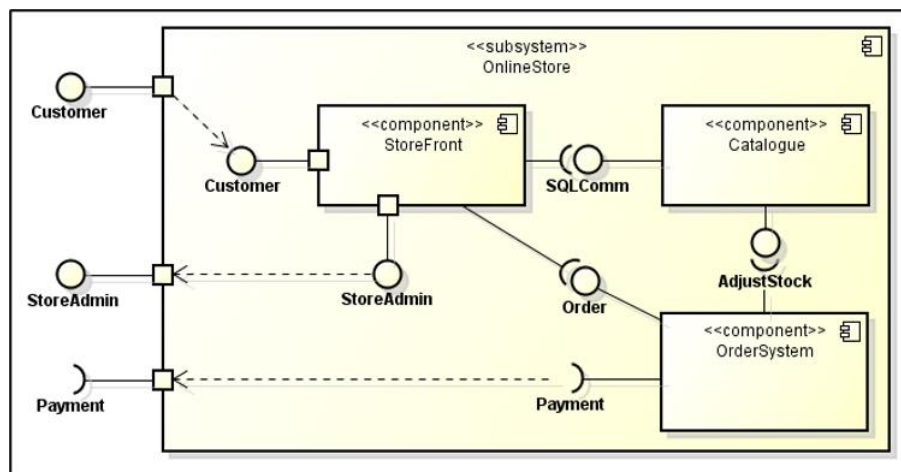
Gambar 4.17. Contoh Activity Diagram

g) Component Diagram

Component Diagram menggambarkan struktur fisik kode dari komponen. Komponen dapat berupa *source code*, komponen biner, atau *executable component*. Sebuah komponen berisi informasi tentang *logic class* atau *class* yang diimplementasikan sehingga membuat pemetaan dari *logical view* ke *component view*. Notasi yang digunakan pada *component diagram* adalah sebagai berikut:

Gambar 4.18. Notasi *Component Diagram*

Contoh *component diagram*:

Gambar 4.19. Contoh *Component Diagram*

h) *Deployment Diagram*

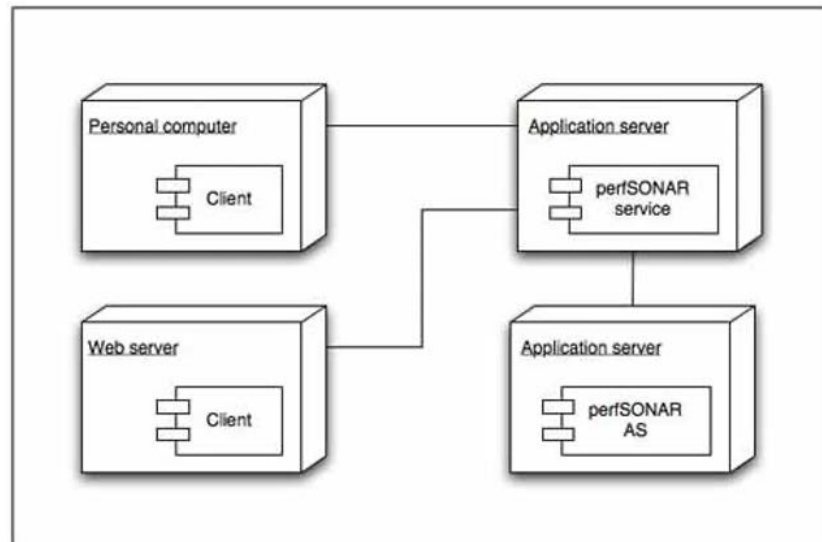
Deployment Diagram menggambarkan arsitektur fisik dari perangkat keras dan perangkat lunak sistem, menunjukkan hubungan komputer dengan perangkat (*nodes*) satu sama lain dan jenis hubungannya. Di dalam *nodes*, *executable component* dan *object* dialokasikan untuk memperlihatkan unit perangkat lunak yang dieksekusi

oleh *node* tertentu. Notasi pada *deployment diagram* adalah sebagai berikut:

Nama Komponen	Keterangan	Simbol
<i>Component</i>	Pada <i>deployment diagram</i> , komponen-komponen yang ada diletakkan didalam node untuk memastikan keberadaan posisi mereka.	
<i>Node</i>	Node menggunakan bagian-bagian <i>hardware</i> dalam sebuah sistem. Notasi untuk node digambarkan sebagai sebuah kubus 3 dimensi.	
Interface	Interface merupakan kumpulan operasi tanpa implementasi dari suatu class. Implementasi operasi dalam interface dijabarkan oleh operasi didalam class.	
Dependency	Dependency merupakan relasi yang menunjukkan bahwa perubahan pada salah satu elemen memberi pengaruh pada elemen lain.	
Generalization	Generalization menunjukkan hubungan antara elemen yang lebih umum ke elemen yang lebih spesifik. Dengan generalization, class yang lebih spesifik (subclass) akan menurunkan atribut dan operasi dari class yang lebih umum (superclass)	
Note	Note digunakan untuk memberikan keterangan atau komentar tambahan dari suatu elemen sehingga bisa langsung terlampir dalam model.	
<i>Association</i>	Sebuah <i>association</i> digambarkan sebagai sebuah garis yang menghubungkan dua node yang mengindikasikan jalur komunikasi antara komponen-komponen <i>hardware</i> .	

Gambar 4.20. Notasi *Deployment Diagram*

Contoh *deployment diagram*:



Gambar 4.21. Contoh *Deployment Diagram*

3. Pemodelan Berorientasi Layanan (*Service oriented architecture Modeling Language/ SoaML*)

Service Oriented Architecture (SOA) adalah paradigma arsitektur untuk mendefinisikan bagaimana orang, organisasi, dan sistem menyediakan dan menggunakan layanan untuk mencapai hasil. SoaML menyediakan cara standar untuk merancang dan memodelkan solusi SOA menggunakan UML. SoaML memungkinkan arsitektur layanan berorientasi bisnis dan berorientasi sistem untuk saling mendukung dan berkolaborasi dalam misi organisasi.

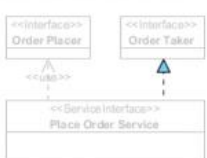
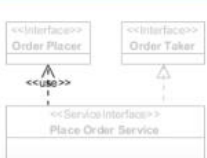
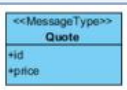
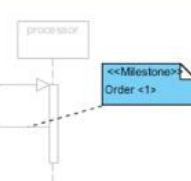
SoaML mengintegrasikan kemampuan pemodelan untuk mendukung penggunaan SOA pada tingkat yang berbeda dan dengan metodologi yang berbeda. Khususnya dukungan untuk pendekatan "berbasis kontrak" dan "berbasis antarmuka" yang secara umum masing-masing mengikuti elemen "*Service Contract*" dan "*Service Interface*" dari profil SoaML. Pendekatan kontrak layanan mendefinisikan kontrak yang menentukan bagaimana penyedia dan konsumen bekerja sama untuk mencapai tujuan melalui penggunaan layanan. Kontrak layanan merupakan kesepakatan antara pihak-

pihak tentang bagaimana layanan akan disediakan dan dikonsumsi. Kesepakatan tersebut termasuk antarmuka, koreografi, syarat, dan ketentuan lainnya. Pendekatan berbasis antarmuka melibatkan penggunaan antarmuka sederhana dan antarmuka layanan. Antarmuka yang sederhana berfokus terutama pada penyampaian layanan satu arah yang tidak memerlukan protokol antar pihak. Antarmuka layanan memungkinkan untuk layanan dua arah. Penyedia (*provider*) dan konsumen (*consumer*) bekerja sama untuk menyelesaikan suatu layanan.

Beberapa jenis diagram yang digunakan dalam UML antara lain:

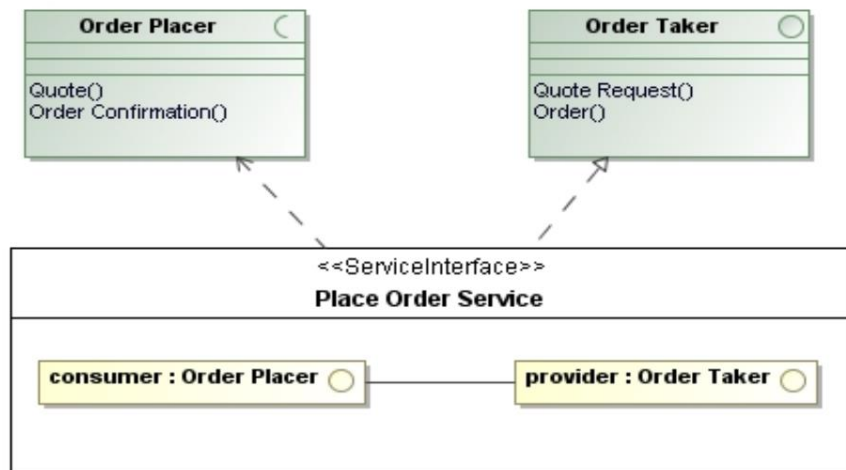
a) *Service Interface Diagram*

Diagram antarmuka layanan adalah jenis diagram SoaML yang dikhususkan untuk definisi dan spesifikasi antarmuka sederhana dan antarmuka layanan. Notasi pada diagram antarmuka layanan adalah sebagai berikut:

Nama	Perwakilan	Keterangan
Service Interface		ServiceInterface mendefinisikan antarmuka dan tanggung jawab peserta untuk menyediakan atau menggunakan layanan. Ini digunakan sebagai jenis Layanan atau Port Permintaan. ServiceInterface adalah sarana untuk menentukan bagaimana peserta berinteraksi untuk menyediakan atau menggunakan Layanan. ServiceInterface dapat mencakup protokol, perintah, dan pertukaran informasi tertentu yang dengannya tindakan dimulai dan hasil dari efek dunia nyata tersedia sebagaimana ditentukan melalui bagian fungsionalitas layanan. ServiceInterface dapat menangani konsep yang terkait dengan kepemilikan, domain kepemilikan, tindakan yang dikomunikasikan antara rekan hukum, kepercayaan, transaksi bisnis, otoritas, delegasi, dll.
Interface		Antarmuka sederhana mendefinisikan layanan satu arah yang tidak memerlukan protokol. Layanan tersebut dapat didefinisikan hanya dengan satu antarmuka UML dan kemudian disediakan pada port "Layanan" dan digunakan pada port "Permintaan".
Role		ServiceInterface adalah Kelas UML. Ini mendefinisikan peran khusus untuk setiap peserta bermain dalam interaksi layanan. Peran ini memiliki nama dan tipe antarmuka. Antarmuka penyedia (yang harus menjadi jenis salah satu bagian di kelas) direalisasikan (disediakan) oleh kelas ServiceInterface. Antarmuka konsumen (jika ada) harus digunakan oleh kelas.
Connector		Hubungkan peran dalam antarmuka layanan.
Capability		A Capability memodelkan kemampuan untuk bertindak dan menghasilkan hasil yang mencapai hasil yang dapat memberikan layanan yang ditentukan oleh ServiceContract atau ServiceInterface terlepas dari Peserta yang mungkin menyediakan layanan tersebut. ServiceContract saja, tidak memiliki ketergantungan atau harapan tentang bagaimana kapabilitas direalisasikan – dengan demikian memisahkan kekhawatiran tentang 'apa' vs. Kemampuan ditampilkan dalam konteks menggunakan diagram dependensi layanan.
Expose		Ketergantungan Expose memberikan kemampuan untuk menunjukkan Kemampuan apa yang dibutuhkan oleh atau disediakan oleh peserta yang harus diekspos melalui Antarmuka Layanan.
Dependency		Penyedia mungkin juga memiliki ketergantungan penggunaan pada antarmuka konsumen, yang menunjukkan fakta bahwa penyedia dapat memanggil konsumen sebagai bagian dari interaksi dua arah. Ini juga dikenal sebagai "panggilan balik" di banyak teknologi.
Realization		ServiceInterface menentukan penerimaan dan operasi yang diterimanya melalui InterfaceRealisasi. ServiceInterface dapat mewujudkan sejumlah Antarmuka. Beberapa model platform tertentu dapat membatasi jumlah antarmuka yang direalisasikan paling banyak satu.
Usage		ServiceInterface menentukan kebutuhan yang diperlukan melalui ketergantungan Penggunaan ke Antarmuka.
Message Type		MessageType adalah jenis objek nilai yang mewakili pertukaran informasi antara permintaan peserta dan layanan. Informasi ini terdiri dari data yang diteruskan ke, dan/atau dikembalikan dari, pemanggilan operasi atau sinyal peristiwa yang didefinisikan dalam antarmuka layanan. MessageType ada dalam domain atau konten khusus layanan dan tidak menyertakan header atau implementasi lain atau informasi khusus protokol.
Milestone		Milestone menggambarkan kemajuan dengan mendefinisikan sinyal yang dikirim ke pengamat abstrak. Sinyal berisi nilai integer yang secara intuitif mewakili jumlah kemajuan yang telah dicapai ketika melewati titik yang melekat pada Milestone ini.

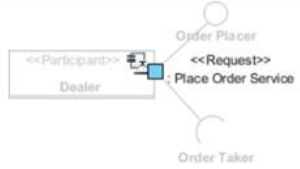
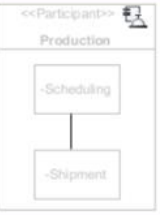
Gambar 4.22. Notasi *Service Interface Diagram*

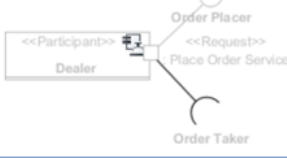
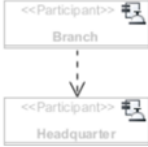
Contoh *service interface diagram*:

Gambar 4.23. Contoh *Service Interface Diagram*

b) *Service Participant Diagram*

Service participant diagram berfokus pada orang, organisasi, sistem atau siapa saja yang mengambil bagian dalam arsitektur layanan. Pemodel menggunakan *service participant diagram* untuk mewakili *participant* ini serta antarmuka yang mereka butuhkan atau disediakan dalam menyelesaikan layanan. Perhatikan bahwa cara *participant* berinteraksi tidak dimodelkan dalam *service participant diagram*. Notasi pada *service participant diagram* adalah sebagai berikut:

Nama	Perwakilan	Keterangan
Participant		Peserta mewakili beberapa (mungkin konkret) pihak atau komponen yang menyediakan dan/atau menggunakan layanan (peserta dapat mewakili orang, organisasi, atau sistem yang menyediakan dan/atau menggunakan layanan). Peserta adalah penyedia layanan jika menawarkan layanan. Peserta adalah konsumen jasa jika menggunakan jasa. Seorang peserta dapat menyediakan atau mengkonsumsi sejumlah layanan. Konsumen dan penyedia layanan adalah peran Peserta bermain: peran penyedia di beberapa layanan dan konsumen di lain, tergantung pada kemampuan yang mereka berikan dan kebutuhan yang mereka miliki untuk melaksanakan kemampuan mereka. Karena sebagian besar konsumen dan penyedia memiliki layanan dan permintaan, Partisipan digunakan untuk memodelkan keduanya.
Agent		Secara umum, agen dapat berupa agen perangkat lunak, agen perangkat keras, agen firmware, agen robot, agen manusia, dan sebagainya. Sementara pengembangan perangkat lunak secara alami menganggap sistem TI dibangun hanya dari agen perangkat lunak, kombinasi mekanisme agen sebenarnya dapat digunakan dari manufaktur lantai toko hingga sistem peperangan.
Part		Digunakan sebagai komponen komposit untuk peserta.
Property		Properti adalah fitur struktural. Ini menghubungkan instance kelas dengan nilai atau kumpulan nilai dari tipe fitur. Sebuah properti dapat ditunjuk sebagai properti pengenalan, properti yang dapat digunakan untuk membedakan atau mengidentifikasi instance dari classifier yang mengandung dalam sistem terdistribusi.
Service Port		Port layanan adalah fitur yang mewakili titik interaksi pada Peserta di mana layanan benar-benar disediakan. Pada penyedia layanan, ini dapat dianggap sebagai "penawaran" layanan (berdasarkan antarmuka layanan). Dengan kata lain, port layanan adalah titik interaksi untuk melibatkan peserta dalam suatu layanan melalui antarmuka layanannya.
Request Port		Konsumen layanan menentukan antarmuka layanan yang mereka butuhkan dengan menggunakan port permintaan.
Port		Port diperluas dengan properti connectorRequired untuk menunjukkan apakah konektor diperlukan pada port ini atau pengklasifikasi yang berisi mungkin dapat berfungsi tanpa terhubung apa pun.
Service Channel		ServiceChannel menyediakan jalur komunikasi antara Permintaan konsumen dan layanan penyedia.
Connector		Hubungkan bagian-bagian dalam peserta, jika ada.

Capability	 <pre> classDiagram class Shipping["<<Capability>> Shipping"] { +requestShipping() } </pre>	A Capability memodelkan kemampuan untuk bertindak dan menghasilkan hasil yang mencapai hasil yang dapat memberikan layanan yang ditentukan oleh ServiceContract atau ServiceInterface terlepas dari Peserta yang mungkin menyediakan layanan tersebut. ServiceContract saja, tidak memiliki ketergantungan atau harapan tentang bagaimana kapabilitas direalisasikan – sehingga memisahkan kekhawatiran tentang "apa" vs. "bagaimana". Kemampuan dapat menentukan dependensi atau proses internal untuk merinci bagaimana kemampuan itu disediakan termasuk dependensi pada Kemampuan lain. Kemampuan ditampilkan dalam konteks menggunakan diagram dependensi layanan.
Realization	 <pre> classDiagram class Dealer["<<Participant>> Dealer"] class OrderPlacer["Order Placer"] class OrderTaker["Order Taker"] class PlaceOrderService["<<Request>> Place Order Service"] Dealer -- > OrderPlacer Dealer -- > OrderTaker Dealer --> PlaceOrderService </pre>	Menghubungkan port dan antarmuka layanan/permintaan untuk mewakili antarmuka yang disediakan oleh peserta pada port tertentu.
Usage	 <pre> classDiagram class Dealer["<<Participant>> Dealer"] class OrderPlacer["Order Placer"] class OrderTaker["Order Taker"] class PlaceOrderService["<<Request>> Place Order Service"] Dealer --> OrderPlacer Dealer --> OrderTaker Dealer --> PlaceOrderService </pre>	Menghubungkan port dan antarmuka layanan/permintaan untuk mewakili antarmuka yang dibutuhkan oleh peserta, pada port tertentu.
Dependency	 <pre> classDiagram class Branch["<<Participant>> Branch"] class Headquarter["<<Participant>> Headquarter"] Branch -.-> Headquarter </pre>	Menunjukkan bahwa peserta/antarmuka/port bergantung pada peserta/antarmuka/port.

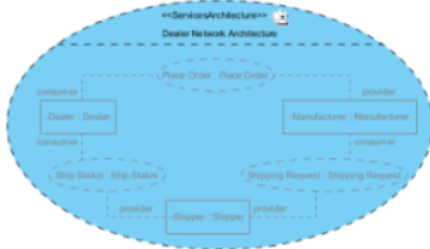
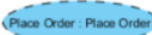
Gambar 4.24. Notasi *Service Participant Diagram*

Contoh *service interface diagram*:

Gambar 4.25. Contoh *Service Participant Diagram*

c) *Service Architecture Diagram*

SoaML memungkinkan pemodel untuk membangun model arsitektur layanan yang menyatukan spesifikasi layanan dan peserta dan menunjukkan bagaimana orang, tim, dan organisasi bekerja bersama untuk mencapai tujuan. Notasi pada *service architecture diagram* adalah sebagai berikut:

Nama	Perwakilan	Keterangan
Services Architecture		Arsitektur Layanan (SOA) menjelaskan bagaimana peserta bekerja sama untuk suatu tujuan dengan menyediakan dan menggunakan layanan yang dinyatakan sebagai kontrak layanan. Dengan mengungkapkan penggunaan layanan, Arsitektur Layanan menyiratkan beberapa tingkat pengetahuan tentang ketergantungan antara peserta dalam beberapa konteks. Setiap penggunaan layanan dalam Arsitektur Layanan diwakili oleh penggunaan Kontrak Layanan yang terikat pada peran peserta dalam arsitektur itu.
Internal Participant		Peserta yang bekerja sama dalam sebuah arsitektur.
External Participant		Peserta eksternal menggunakan fitur "agregasi bersama" dari UML untuk menunjukkan bahwa peran-peran ini berada di luar arsitektur sedangkan peran lainnya bersifat internal.
Service Contract Use (CollaborationUse)		CollaborationUse secara eksplisit menunjukkan kemampuan Classifier yang memiliki untuk memenuhi ServiceContract atau mematuhi ServicesArchitecture. Sebuah Classifier dapat berisi sejumlah CollaborationUses yang menunjukkan apa yang dipenuhinya. CollaborationUse memiliki roleBindings yang menunjukkan peran apa yang dimainkan setiap bagian dalam Classifier pemilik. Jika CollaborationUse ketat, maka bagian-bagiannya harus kompatibel dengan peran yang mereka ikat, dan Pengklasifikasi pemilik harus memiliki perilaku yang sesuai dengan perilaku yang dimiliki dari tipe Kolaborasi CollaborationUse.
Role Binding		Sebuah CollaborationUse berisi roleBindings yang mengikat setiap peran Kolaborasinya ke bagian dari Pengklasifikasi yang berisi.

Gambar 4.26. Notasi *Service Architecture Diagram*

Contoh *service architecture diagram*:

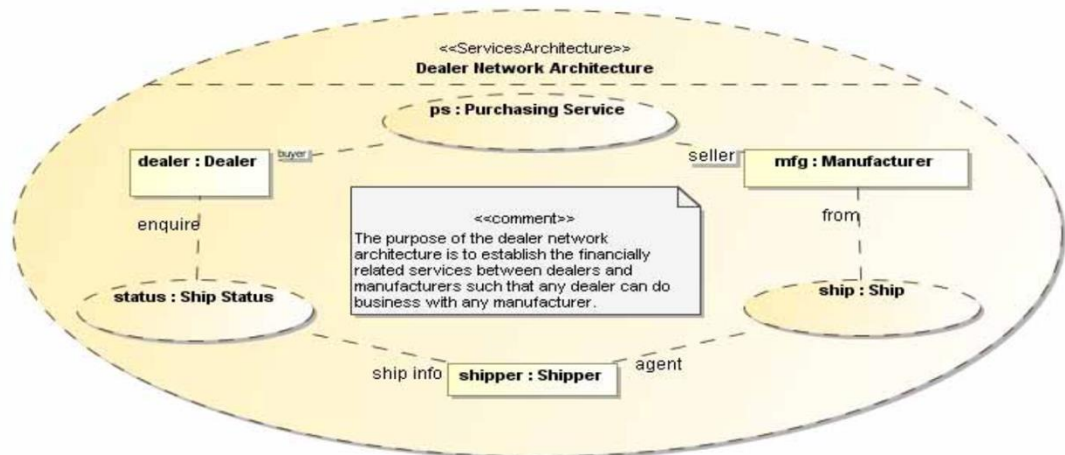
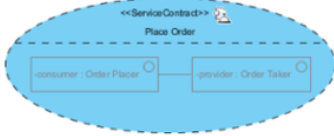
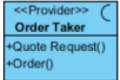
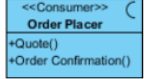
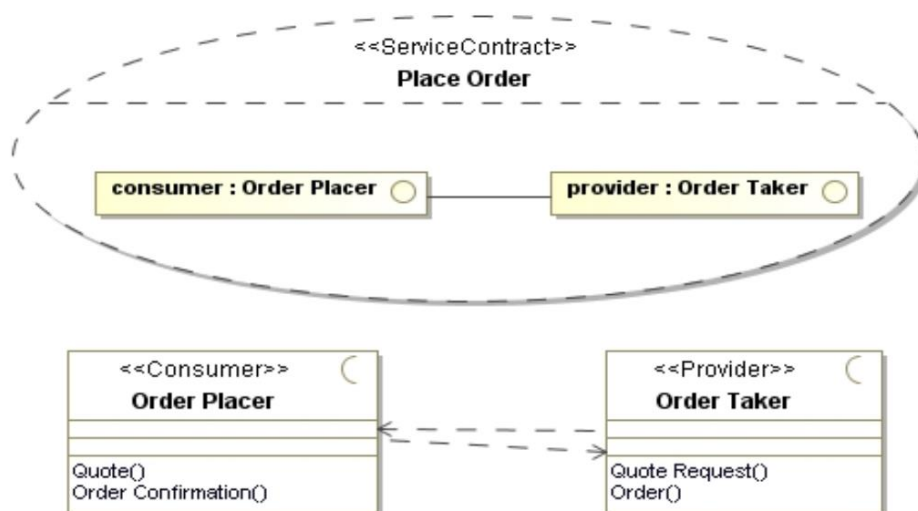
Gambar 4.27. Contoh *Service Architecture Diagram*d) *Service Contract Diagram*

Diagram kontrak layanan dirancang untuk spesifikasi kontrak layanan. Pemodel biasanya menggabungkan penggunaan *sequence diagram* dan *activity diagram* dalam mewakili koreografi kontrak layanan. Notasi pada *service contract diagram* adalah sebagai berikut:

Nama	Perwakilan	Keterangan
Service Contract		ServiceContract adalah spesifikasi kesepakatan antara penyedia dan konsumen layanan untuk mengetahui informasi, produk, aset, nilai, dan kewajiban apa yang akan mengalir antara penyedia dan konsumen layanan tersebut. Ini menentukan layanan tanpa memperhatikan realisasi, kemampuan, atau implementasi. ServiceContract tidak memerlukan spesifikasi tentang siapa, bagaimana, atau mengapa pihak mana pun akan memenuhi kewajiban mereka berdasarkan ServiceContract itu, sehingga menyediakan kopling longgar dari paradigma SOA. Dalam kebanyakan kasus, ServiceContract akan menentukan dua peran (penyedia dan konsumen) tetapi peran layanan lainnya juga dapat ditentukan. ServiceContract juga dapat memiliki perilaku yang menentukan urutan pertukaran antara para pihak serta status dan pengiriman kemampuan yang dihasilkan.
Role		ServiceContract menangkap kesepakatan antara peran yang dimainkan oleh konsumen dan penyedia layanan, kemampuan dan kebutuhan mereka dan aturan tentang bagaimana konsumen dan penyedia harus berinteraksi. Peran dalam ServiceContract diketik oleh Antarmuka yang menentukan Operasi dan peristiwa yang terdiri dari interaksi koreografi layanan.
Provider		Sebuah "Penyedia" memodelkan antarmuka yang disediakan oleh penyedia layanan. Penyedia layanan memberikan hasil interaksi layanan. Penyedia biasanya akan menjadi orang yang merespons interaksi layanan. Antarmuka penyedia digunakan sebagai jenis "Kontrak Layanan" dan terikat oleh syarat dan ketentuan kontrak layanan tersebut.
Consumer		Sebuah «Konsumen» memodelkan antarmuka yang disediakan oleh konsumen layanan. Konsumen jasa menerima hasil interaksi jasa. Konsumen biasanya akan menjadi orang yang memulai interaksi layanan. Antarmuka konsumen digunakan sebagai jenis «Kontrak Layanan» dan terikat oleh syarat dan ketentuan kontrak layanan tersebut.
Connector		Hubungkan peran, jika ada, dalam peserta.
Dependency		Penyedia mungkin juga memiliki ketergantungan penggunaan pada antarmuka konsumen, yang menunjukkan fakta bahwa penyedia dapat memanggil konsumen sebagai bagian dari interaksi dua arah. Ini juga dikenal sebagai "panggilan balik" di banyak teknologi.

Gambar 4.28. Notasi *Service Contract Diagram*Contoh *service contract diagram*:

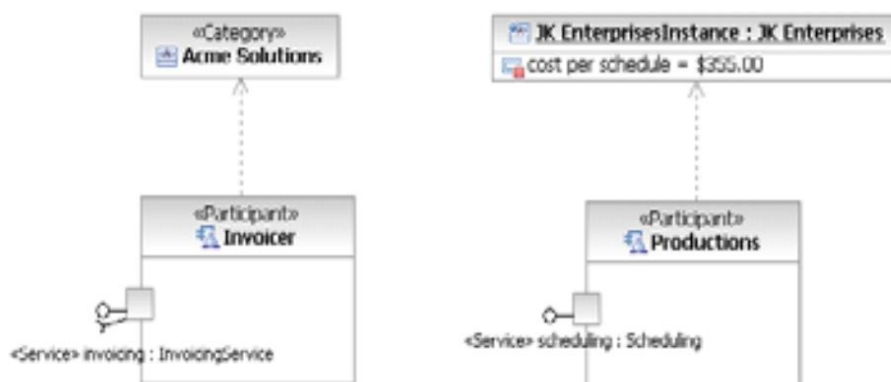
Gambar 4.29. Contoh *Service Contract Diagram*e) *Service Categorization Diagram*

Kategorisasi diperlukan untuk mengatur konten model agar elemen model dapat digunakan untuk berbagai tujuan dan dilihat dari perspektif yang berbeda. Diagram kategorisasi layanan memungkinkan kategorisasi elemen SoaML dengan katalog, kategori, dan nilai kategori. Notasi pada *service categorization diagram* adalah sebagai berikut:

Nama	Perwakilan	Keterangan
Catalog	 	Kumpulan elemen terkait bernama, termasuk katalog lain yang dicirikan oleh serangkaian kategori tertentu. Menerapkan Kategori ke Elemen menggunakan Kategorisasi menempatkan Elemen di Katalog.
Category	 	Kategori adalah sepotong informasi tentang suatu elemen. Kategori memiliki nama yang menunjukkan tentang informasi tersebut dan seperangkat atribut dan batasan yang menjadi ciri Kategori. Sebuah Elemen mungkin memiliki banyak Kategori dan Kategori yang sama dapat diterapkan ke banyak Elemen. Kategori dapat diatur ke dalam hierarki Katalog.
Category Value	 	Sebuah CategoryValue memberikan nilai untuk atribut dari sebuah Kategori. Ini juga dapat digunakan untuk mengkategorikan elemen model yang memberikan informasi rinci untuk kategori tersebut.

Gambar 4.30. Notasi *Service Categorization Diagram*

Contoh *service categorization diagram*:

Gambar 4.31. Contoh *Service Categorization Diagram*

b. Perancangan Antarmuka Pengguna

Desain antarmuka pengguna adalah bagian penting dari proses desain perangkat lunak. Desain antarmuka pengguna harus memastikan bahwa interaksi antara manusia dan mesin menyediakan operasi dan kontrol mesin yang efektif. Agar perangkat lunak mencapai potensi penuhnya, antarmuka pengguna harus dirancang agar sesuai dengan keterampilan, pengalaman, dan harapan pengguna (IEEE Computer Society, 2014).

Perancangan antarmuka pengguna secara umum dibagi dua yaitu rancangan masukan dan rancangan keluaran.

1. Rancangan Masukan (Input)

Input merupakan awal dimulainya proses pengolahan data. Bahan mentah dari informasi merupakan data yang dihasilkan dari transaksi tersebut akan menjadi input bagi sistem informasi. Hasil yang diperoleh tentunya tidak akan menyimpang jauh dari input tersebut. Kualitas input sangat menentukan kualitas *output*. Kalau yang masuk adalah sampah, maka yang keluar pun nantinya adalah sampah. Supaya bukan sampah yang dihasilkan, maka data input yang melalui formulir harus dirancang sedemikian rupa sehingga sampah tersebut bisa dihindari.

Pedoman perancangan formulir input adalah sebagai berikut:

- Mempertimbangkan media pemasukan
- *Layout* formulir harus sesuai dengan yang diinginkan
- Formulir harus akurat
- Tampilan formulir harus atraktif
- Mudah diisi:
 - ✓ Alur pengisian formulir teratur, dari kiri ke kanan atau dari atas ke bawah
 - ✓ Komponen formulir jelas: judul, identifikasi dan akses, instruksi, isian, tanda tangan dan pengesahan.
 - ✓ Menggunakan *caption*

Layar masukan bisa dirancang berdasarkan dokumen formulir. Dalam hal ini layar masukan dirancang semirip mungkin dengan dokumen formulir yang sudah ada. Jika tampilan pertanyaan pada formulir terlalu banyak, kita bisa menggunakan pendekatan *parent-child (header-detail)*. Tampilan yang seimbang akan mudah dibaca. Ada beberapa cara yang dapat digunakan untuk mengurangi jumlah masukan, cara yang dapat dilakukan yaitu menggunakan kode, data yang relatif konstan disimpan di *file* induk, jam dan tanggal dapat diambil dari sistem, rutin perhitungan dilakukan oleh sistem.

Empat garis pedoman untuk merancang formulir:

- Membuat formulir mudah diisi, yaitu dengan memperhatikan aliran formulir, pengelompokan bagian sebuah formulir, pembuatan judul.
- Memastikan bahwa formulir akan memenuhi tujuan yang telah dibuat.
- Membuat formulir yang memastikan penyelesaian tepat.
- Buatlah formulir yang menarik.

Contoh rancangan masukan:

Tambah Petugas

Provinsi

---PILIH PROVINSI---

Kab/Kota

---PILIH KAB/KOTA---

Kode Petugas

Kode Petugas

Nama

Nama

Status

☒ Organik

☐ Mitra

Jabatan

☒ Pencacah

☐ Pengawas

Simpan

Gambar 4.32. Contoh Rancangan Masukan

2. Rancangan Keluaran (*Output*)

Perancangan *output* atau keluaran merupakan hal yang tidak dapat diabaikan, karena laporan atau keluaran yang dihasilkan harus memudahkan bagi setiap unsur manusia yang membutuhkannya. Tujuan yang harus dicapai analisis sistem saat merancang *output*:

- Merancang *output* untuk tujuan tertentu
- Membuat *output* bermanfaat bagi para pengguna
- Menampilkan jumlah *output* yang tepat
- Menyediakan distribusi *output* yang tepat
- Menyediakan *output* yang tepat waktu
- Memilih metode *output* yang paling efektif.

Berdasarkan tujuannya, *output* dapat dibedakan menjadi 2 tipe:

a) Eksternal

External Output bersifat keluar organisasi. *Output* ini ditujukan kepada konsumen, pemasok, mitra bisnis dan badan pemerintahan. *Output external* menyimpulkan dan melaporkan transaksi bisnis. Contoh dari *external output* ini antara lain faktur, nota pembelian barang, jadwal kursus, tiket pesawat, tagihan telepon dan lain sebagainya. Tujuan *output* untuk informasi di luar organisasi pemakai. Contoh: faktur, *check*, tanda terima pembayaran, dan sebagainya.

b) Internal

Internal output digunakan untuk para pemilik dan pengguna sistem dalam sebuah perusahaan. *Internal output* mendukung operasi bisnis sehari-hari atau pengawasan manajemen dan pengambilan keputusan.

Contoh: laporan-laporan terinci, laporan-laporan ringkasan, dan sebagainya.

Tiga jenis *internal output* adalah sebagai berikut:

- 1) *Detailed Report*, menyajikan informasi dengan sedikit atau tanpa dilakukan penyaringan atau pembatasan. Contoh: Daftar seluruh tagihan pelanggan.
- 2) *Summary Report*, berisi informasi dari manajer yang tidak perlu diperlihatkan keseluruhan laporan secara detail. Contoh: Laporan ringkasan total penjualan dalam hitungan bulanan dan grafik penjualan per-tahun.
- 3) *Exception Report*, menyaring data sebelum ditunjukkan kepada manajer sebagai sebuah informasi. Contoh: Laporan persediaan barang yang hampir habis.

Macam-Macam Bentuk Laporan

a) Laporan Berbentuk Tabel

- *Notice Report*

Notice report merupakan bentuk laporan yang memerlukan perhatian khusus. Laporan ini harus dibuat sesederhana mungkin, tetapi jelas, karena dimaksudkan agar permasalahan-permasalahan yang terjadi tampak dengan jelas, sehingga dapat langsung ditangani.

- *Equipoised Report*

Isi dari *equipoised report* adalah hal-hal yang bertentangan. Laporan ini biasanya digunakan untuk maksud perencanaan. Dengan disajikan informasi yang berisi hal-hal bertentangan, maka dapat dijadikan sebagai dasar di dalam pengambilan keputusan.

- *Variance Report*

Laporan ini menunjukkan selisih (*variance*) antara standar yang sudah ditetapkan dengan hasil kenyataannya atau sesungguhnya.

- *Comparative Report*

Isi laporan ini adalah membandingkan antara satu hal dengan hal yang lainnya. Misalnya pada laporan rugi/laba atau neraca dapat dibandingkan antara nilai-nilai elemen tahun berjalan dengan tahun-tahun sebelumnya.

b) Laporan Berbentuk Grafik

- Bagan Garis

Pada bagan garis (*line chart*), variasi dari data ditunjukkan dengan suatu garis atau kurva. Bagan garis mempunyai beberapa kebaikan, yaitu:

- ✓ Dapat menunjukkan hubungan antara nilai dengan baik.
- ✓ Dapat menunjukkan beberapa titik.
- ✓ Tingkat ketepatannya dapat diatur sesuai dengan skalanya.
- ✓ Mudah dimengerti.

Disamping kebaikannya, bagan garis mempunyai beberapa kelemahan, yaitu:

- ✗ Bila terlalu banyak garis atau kurva (sekitar lebih dari 4 buah garis atau kurva), maka akan tampak ruwet.
- ✗ Hanya terbatas pada 2 dimensi.
- ✗ Spasi dapat menyesatkan.

- Bagan Batang

Nilai-nilai data dalam bagan batang (*bar chart*) digambarkan dalam bentuk batang-batang vertikal ataupun batang-batang horisontal. Kebaikan dari bagan batang adalah sebagai berikut:

- ✓ Baik untuk perbandingan.
- ✓ Dapat menunjukkan nilai dengan tepat.
- ✓ Mudah dimengerti.

Disamping kebaikannya, bagan batang mempunyai beberapa kelemahan, yaitu:

- ✖ Terbatas hanya pada satu titik saja.
- ✖ Spasi dapat menyesatkan.
- Bagan Pai

Bagan pai (*pie chart*) merupakan bagan yang berbentuk lingkaran menyerupai kue pai (*pie*). Tiap-tiap potong dari *pie* dapat menunjukkan bagian dari data. Kebaikan dari bagan pai adalah sebagai berikut ini.

- ✓ Baik untuk perbandingan sebagian dengan keseluruhannya.
- ✓ Mudah dimengerti.

Disamping kebaikannya, bagan pai mempunyai beberapa kelemahan, yaitu:

- ✖ Penggunaannya terbatas
- ✖ Ketepatannya kurang
- ✖ Tidak dapat menunjukkan hubungan beberapa titik

c) Laporan Untuk Level Manajemen yang Berbeda

- Laporan Berhirarki

Laporan yang dibuat untuk masing-masing level manajemen untuk menerima informasi sesuai dengan permintaan khusus, tanpa memberikan detail yang tidak relevan. Para eksekutif akan melihat *trend*, kecenderungan, dan pola-pola dari laporan tersebut. Mereka ingin mengetahui apakah masing-masing bagian sudah mencapai tujuan.

Ada dua macam laporan berhirarki:

- 1) *Filter Report*: laporan yang dirancang untuk memfilter elemen-elemen data yang dipilih dari *database*, sehingga pengambilan keputusan akan memperoleh laporan yang sesuai dengan kebutuhannya. Biasanya data difilter pada level atas.

2) *Responsibility Report*: laporan yang dibuat untuk memutuskan siapa yang bertanggung jawab terhadap suatu laporan, apakah CEO, manajer pemasaran, atau spesialis media, dll.

- Laporan Yang Membandingkan Data

Laporan ini dibuat untuk membantu manajer dan *user* lain dalam memilih dua atau lebih item untuk menyusun kesamaan atau ketidaksamaan (perbedaan). Dengan perbandingan ini, *user* berada pada posisi terbaik untuk membuat keputusan yang rasional.

Ada 3 macam laporan yang membandingkan data:

- 1) *Horizontal Report*: neraca dan laporan rugi laba menunjukkan laporan keuangan periodik yang meringkas ribuan transaksi dan elemen data menjadi *output* untuk beragam *user*. *User* akan memperoleh gambaran yang jelas dengan melihat perbandingan pada laporan. Hal ini dapat dilakukan dengan merancang *horizontal report*. Jumlah setiap item dibandingkan dengan item yang berhubungan pada satu atau lebih laporan sebelumnya.
- 2) *Vertical Report*: laporan yang membandingkan suatu bagian komponen dengan totalnya.
- 3) *Counterbalance Report*: setiap situasi dibandingkan dalam laporan. Contohnya, skenario yang terburuk, layak, dan terbaik dapat membantu para perencana menilai proyek-proyek yang berisiko, juga informasi berharga bagi para eksekutif dalam pengambilan keputusan.

- Laporan Untuk Monitor Variansi Data

- 1) *Variance Report*: laporan yang dibuat untuk membandingkan standar dengan hasil aktual yang diperoleh. Biasanya laporan ini dibuat sesuai dengan waktu atau selesainya suatu proses.

- 2) *Exception Report*: laporan ini seperti *variance report*, tetapi beberapa kuota atau batasan dibuat untuk suatu proses atau aktivitas. Laporan ini dibuat hanya ketika beberapa proses atau aktivitas tidak sesuai dengan batasan atau kuota. Contoh: Daftar pelanggan yang sering menunggak pembayaran.

Contoh rancangan keluaran:

**Tabel Rekapitulasi Pengeluaran Rumah Tangga
Hasil Pencacahan VBH22-BL Bulan 1 dan 2**

Provinsi	: [11] ACEH	Triwulan	: 1
Kabupaten/Kota	: [06] ACEH TENGAH	Status BL-1	: Clean
Kecamatan	: [031] LUT TAWAR	Status BL-2	: Belum
Desa/Kelurahan	: [017] ONE-ONE		
No. Blok Sensus	: 001B		
NKS	: 10123		
No. Urut Sampel	: 001		
Nama KRT	: M HUSIN		

No	Kode Komoditas	Jenis Barang dan Jasa	Nilai Konsumsi		Jumlah (Rp)
			Bulan 1	Bulan 2	
(1)	(2)	(3)	(4)	(5)	(6)
1	0212007	JAKET PRIA	250.000	0	250.000
2	0221002	SANDAL KULIT PRIA	100.000	0	100.000
3	0313001	PERKIRAAN SEWA RUMAH MILIK SENDIRI	350.000	0	350.000
4	0341001	TARIF LISTRIK	37.000	0	37.000
5	0342002	BAHAN BAKAR RUMAHTANGGA	50.000	0	50.000
6	0461010	SABUN DETERGEN BUBUK	30.000	0	30.000
7	0621002	BAN LUAR MOBIL	2.400.000	0	2.400.000
8	0622003	BENSIN ECERAN/K5	190.000	0	190.000

Gambar 4.33. Contoh Rancangan Keluaran

c. Perancangan Arsitektur

Desain arsitektur berkaitan dengan pemahaman bagaimana sistem harus diatur dan merancang struktur keseluruhan sistem itu. Ini adalah hubungan penting antara desain dan rekayasa persyaratan, karena mengidentifikasi komponen struktural utama dalam suatu sistem dan hubungan di antara mereka. *Output* dari proses desain arsitektur adalah

model arsitektur yang menggambarkan bagaimana sistem diatur sebagai satu set komponen yang berkomunikasi (Sommerville, 2011).

Desain arsitektur perangkat lunak dibagi dua yaitu arsitektur dalam skala kecil dan arsitektur dalam skala besar:

- Arsitektur dalam skala kecil berkaitan dengan arsitektur program individu. Pada tingkat ini, kita berfokus dengan cara program individu didekomposisi menjadi komponen-komponen. Subbab ini sebagian besar berkaitan dengan arsitektur program.
- Arsitektur secara luas berkaitan dengan arsitektur sistem organisasi yang kompleks yang mencakup sistem lain, program, dan komponen program. Sistem organisasi ini didistribusikan melalui komputer yang berbeda, yang mungkin dimiliki dan dikelola oleh organisasi yang berbeda.

1. Keputusan Desain Arsitektur

Desain arsitektur adalah proses kreatif di mana Anda merancang organisasi sistem yang akan memenuhi persyaratan fungsional dan non-fungsional suatu sistem. Karena merupakan proses kreatif, aktivitas dalam proses bergantung pada jenis sistem yang dikembangkan, latar belakang dan pengalaman arsitek sistem, dan persyaratan khusus untuk sistem. Oleh karena itu, berguna untuk memikirkan desain arsitektur sebagai serangkaian keputusan yang harus dibuat daripada urutan kegiatan.

Selama proses desain arsitektur, arsitek sistem harus membuat sejumlah keputusan struktural yang sangat mempengaruhi sistem dan proses pengembangannya. Berdasarkan pengetahuan dan pengalaman mereka, mereka harus mempertimbangkan pertanyaan mendasar berikut tentang sistem:

- a) Apakah ada arsitektur aplikasi generik yang dapat bertindak sebagai *template* untuk sistem yang sedang dirancang?
- b) Bagaimana sistem akan didistribusikan ke sejumlah inti atau prosesor?

- c) Pola atau gaya arsitektur apa yang mungkin digunakan?
- d) Apa pendekatan mendasar yang digunakan untuk menyusun sistem?
- e) Bagaimana komponen struktural dalam sistem didekomposisi menjadi subkomponen?
- f) Strategi apa yang akan digunakan untuk mengendalikan operasi komponen-komponen dalam sistem?
- g) Organisasi arsitektur apa yang terbaik untuk memenuhi kebutuhan non-fungsional dari sistem?
- h) Bagaimana desain arsitektur dievaluasi?
- i) Bagaimana seharusnya arsitektur sistem didokumentasikan?

Untuk sistem dan sistem tertanam (*embedded system*) yang dirancang untuk komputer pribadi, biasanya hanya ada satu prosesor dan kita tidak perlu mendesain arsitektur terdistribusi untuk sistem tersebut. Namun, sebagian besar sistem sekarang adalah sistem terdistribusi di mana perangkat lunak sistem didistribusikan di banyak komputer yang berbeda. Pilihan arsitektur distribusi adalah keputusan kunci yang mempengaruhi kinerja dan keandalan sistem.

2. Pandangan Arsitektur

Di bagian ini, dibahas mengenai pandangan atau perspektif apa yang berguna saat merancang dan mendokumentasikan arsitektur sistem dan notasi apa yang harus digunakan untuk menggambarkan model arsitektur.

Ada empat pandangan yang dapat digunakan untuk merancang arsitektur, yaitu:

- a) Pandangan logis, yang menunjukkan abstraksi kunci dalam sistem sebagai objek atau kelas objek. Seharusnya dimungkinkan untuk menghubungkan persyaratan sistem dengan entitas dalam tampilan logis ini.
- b) Pandangan proses, yang menunjukkan bagaimana, pada saat *run-time*, sistem terdiri dari proses-proses yang berinteraksi. Pandangan ini

berguna untuk membuat penilaian tentang karakteristik sistem non-fungsional seperti kinerja dan ketersediaan.

- c) Pandangan pengembangan, yang menunjukkan bagaimana perangkat lunak didekomposisi untuk pengembangan, yaitu, menunjukkan perincian perangkat lunak menjadi komponen-komponen yang diimplementasikan oleh satu pengembang atau tim pengembangan. Tampilan ini berguna untuk manajer dan pemrogram perangkat lunak.
- d) Pandangan fisik, yang menunjukkan perangkat keras sistem dan bagaimana komponen perangkat lunak didistribusikan ke seluruh prosesor dalam sistem. Pandangan ini berguna untuk *system engineer* yang merencanakan penyebaran sistem.

Ada perbedaan pandangan tentang apakah arsitek perangkat lunak harus menggunakan UML untuk deskripsi arsitektur. Sebuah survei pada tahun 2006 menunjukkan bahwa, ketika UML digunakan, sebagian besar diterapkan secara longgar dan informal. Oleh karena itu, sejumlah peneliti telah mengusulkan penggunaan bahasa deskripsi arsitektur (*Architectural Description Languages/ADL*) yang lebih khusus untuk menggambarkan arsitektur sistem. Elemen dasar ADL adalah komponen dan konektor, dan mereka termasuk aturan dan pedoman untuk arsitektur yang terbentuk dengan baik. Namun, karena sifatnya yang khusus, spesialis domain dan aplikasi merasa sulit untuk memahami dan menggunakan ADL. Hal ini membuat sulit untuk menilai kegunaannya untuk rekayasa perangkat lunak praktis. Namun, pada penerapannya model dan notasi informal, seperti UML, akan tetap menjadi cara yang paling umum digunakan untuk mendokumentasikan arsitektur sistem.

3. Pola Arsitektur

Pada bagian ini, diperkenalkan pola arsitektur dan dijelaskan secara singkat pilihan pola arsitektur yang umum digunakan dalam berbagai jenis sistem. Pola arsitektur dapat dianggap sebagai deskripsi abstrak bergaya praktik yang baik, yang telah dicoba dan diuji dalam sistem dan lingkungan

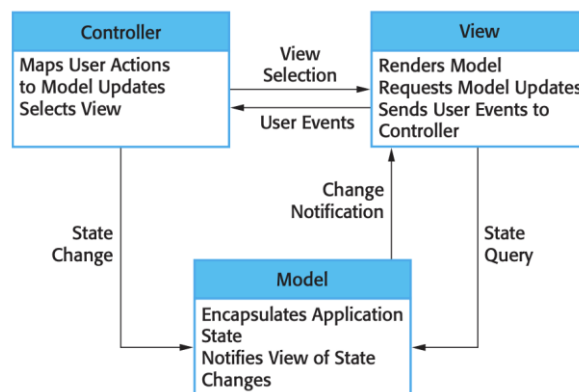
yang berbeda. Jadi, pola arsitektur harus menggambarkan organisasi sistem yang telah berhasil dalam sistem sebelumnya. Itu harus mencakup informasi tentang kondisi yang cocok untuk menggunakan pola itu, serta kekuatan dan kelemahan pola itu.

Tabel 4.1. Pola *Model-View-Controller* (MVC)

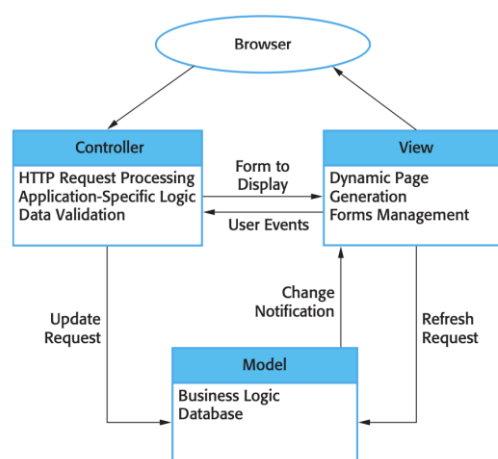
Nama	MVC (<i>Model-View-Controller</i>)
Deskripsi	Memisahkan presentasi dan interaksi dari data sistem. Sistem ini disusun menjadi tiga komponen logis yang berinteraksi satu sama lain. Komponen Model mengelola data sistem dan operasi terkait pada data tersebut. Komponen View mendefinisikan dan mengelola bagaimana data disajikan kepada pengguna. Komponen Controller mengelola interaksi pengguna (misalnya, penekanan tombol, klik mouse, dll.) dan meneruskan interaksi ini ke View dan Model. Lihat Gambar 4.34.
Contoh	Gambar 4.35 menunjukkan arsitektur sistem aplikasi berbasis web yang diatur menggunakan pola MVC.
Kapan Digunakan	Digunakan ketika ada beberapa cara untuk melihat dan berinteraksi dengan data. Juga digunakan ketika persyaratan masa depan untuk interaksi dan penyajian data tidak diketahui.
Kekuatan	Mengizinkan data berubah secara independen dari representasinya dan sebaliknya. Mendukung penyajian data yang sama dengan cara yang berbeda dengan perubahan yang dibuat dalam satu representasi yang ditampilkan pada keseluruhan.
Kelemahan	Dapat melibatkan kode tambahan dan kompleksitas kode ketika model data dan interaksinya sederhana.

Sebagai contoh, Tabel 4.1 menjelaskan pola *Model-View-Controller* yang terkenal. Pola ini adalah dasar dari manajemen interaksi di banyak sistem berbasis web. Deskripsi pola mencakup nama pola, deskripsi singkat (dengan model grafis terkait), dan contoh jenis sistem dimana pola digunakan (sekali lagi, mungkin dengan model grafis). Anda juga harus memasukkan informasi tentang kapan pola harus digunakan dan kelebihan dan kekurangannya. Model

grafis arsitektur yang terkait dengan pola MVC ditunjukkan pada Gambar 4.34 dan Gambar 4.35. Ini menyajikan arsitektur dari tampilan yang berbeda. Gambar 4.34 adalah tampilan konseptual dan Gambar 4.35 menunjukkan kemungkinan arsitektur *run-time* ketika pola ini digunakan untuk manajemen interaksi dalam sistem berbasis web.



Gambar 4.34. Organisasi MVC



Gambar 4.35. Arsitektur Aplikasi Web Menggunakan Pola MVC

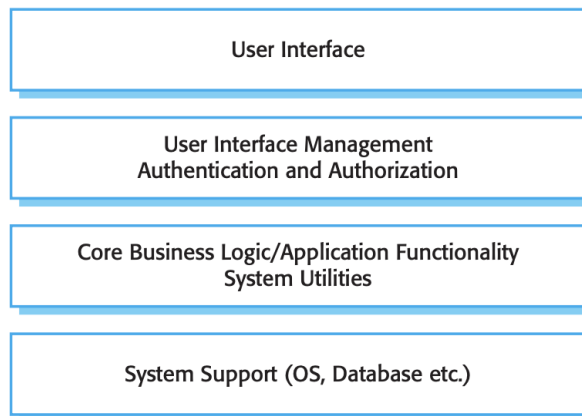
a) Arsitektur Berlapis (*Layered Architecture*)

Gagasan pemisahan dan kemandirian merupakan dasar desain arsitektur karena memungkinkan perubahan dilokalisasi. Pola MVC, ditunjukkan pada Tabel 4.1, memisahkan elemen sistem, memungkinkan

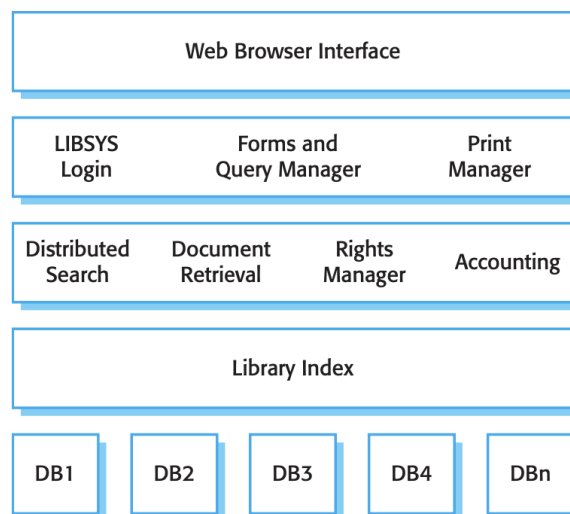
mereka untuk berubah secara independen. Misalnya, menambahkan tampilan baru atau mengubah tampilan yang ada dapat dilakukan tanpa perubahan apa pun pada data dasar dalam model. Pola arsitektur berlapis adalah cara lain untuk mencapai pemisahan dan kemandirian. Pola ini ditunjukkan pada Tabel 4.2. Di sini, fungsionalitas sistem diatur ke dalam lapisan yang terpisah, dan setiap lapisan hanya bergantung pada fasilitas dan layanan yang ditawarkan oleh lapisan tepat di bawahnya.

Tabel 4.2. Pola Arsitektur Berlapis

Nama	Arsitektur Berlapis
Deskripsi	Mengatur sistem ke dalam lapisan dengan fungsionalitas terkait yang berasosiasi dengan setiap lapisan. Lapisan menyediakan layanan ke lapisan di atasnya sehingga lapisan tingkat terendah mewakili layanan inti yang mungkin digunakan di seluruh sistem. Lihat Gambar 4.36.
Contoh	Model berlapis dari sistem untuk berbagi dokumen hak cipta yang disimpan di perpustakaan yang berbeda, seperti yang ditunjukkan pada Gambar 4.37.
Kapan Digunakan	Digunakan saat membangun fasilitas baru di atas sistem yang ada; ketika pengembangan tersebar di beberapa tim dengan tanggung jawab masing-masing tim untuk lapisan fungsionalitas; ketika ada persyaratan untuk keamanan multi-level.
Kekuatan	Memungkinkan penggantian seluruh lapisan selama antarmuka dipertahankan. Fasilitas redundan (misalnya, otentikasi) dapat disediakan di setiap lapisan untuk meningkatkan ketergantungan sistem.
Kelemahan	Dalam praktiknya, memberikan pemisahan yang bersih antara lapisan seringkali sulit dan lapisan tingkat tinggi mungkin harus berinteraksi langsung dengan lapisan tingkat yang lebih rendah daripada melalui lapisan langsung di bawahnya. Performa dapat menjadi masalah karena beberapa tingkat interpretasi permintaan layanan saat diproses di setiap lapisan.



Gambar 4.36. Arsitektur Berlapis Generik



Gambar 4.37. Arsitektur Sistem LIBSYS

b) Arsitektur *Repository* (*Repository Architecture*)

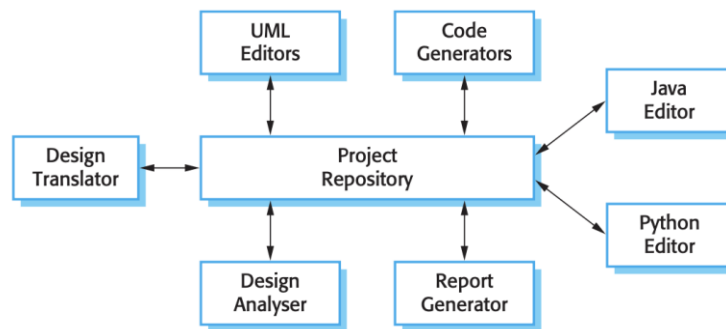
Arsitektur berlapis dan pola MVC adalah contoh pola dimana tampilan yang disajikan adalah organisasi konseptual dari suatu sistem. Contoh berikutnya, pola *repository* (Tabel 4.3), menjelaskan bagaimana sekumpulan komponen yang berinteraksi dapat berbagi data.

Mayoritas sistem yang menggunakan data dalam jumlah besar diatur di sekitar *database* atau *repository* bersama. Oleh karena itu, model

ini cocok untuk aplikasi dimana data dihasilkan oleh satu komponen dan digunakan oleh komponen lain. Contoh jenis sistem ini termasuk sistem komando dan kontrol, sistem informasi manajemen, sistem CAD, dan lingkungan pengembangan interaktif untuk perangkat lunak.

Tabel 4.3. Pola *Repository*

Nama	<i>Repository</i>
Deskripsi	Semua data dalam suatu sistem dikelola dalam <i>repository</i> pusat yang dapat diakses oleh semua komponen sistem. Komponen tidak berinteraksi secara langsung, hanya melalui <i>repository</i> .
Contoh	Gambar 4.38 adalah contoh IDE yang menggunakan komponen <i>repository</i> dari informasi desain sistem. Setiap alat perangkat lunak menghasilkan informasi yang kemudian tersedia untuk digunakan oleh alat lain.
Kapan Digunakan	Pola ini harus digunakan ketika kita memiliki sistem di mana sejumlah besar informasi dihasilkan yang harus disimpan untuk waktu yang lama. Kita juga dapat menggunakannya dalam sistem berbasis data di mana penyertaan data dalam <i>repository</i> memicu tindakan atau alat.
Kekuatan	Komponen dapat berdiri sendiri, tidak perlu mengetahui keberadaan komponen lain. Perubahan yang dilakukan oleh satu komponen dapat disebarkan ke semua komponen. Semua data dapat dikelola secara konsisten (misalnya, pencadangan dilakukan pada waktu yang sama) karena semuanya ada di satu tempat.
Kelemahan	<i>Repository</i> adalah satu titik kegagalan sehingga masalah dalam <i>repository</i> mempengaruhi keseluruhan sistem. Mungkin inefisiensi dalam mengatur semua komunikasi melalui <i>repository</i> . Mendistribusikan <i>repository</i> di beberapa komputer mungkin sulit.



Gambar 4.38. Arsitektur *Repository* untuk IDE

c) Arsitektur *Client-Server* (*Client-Server Architecture*)

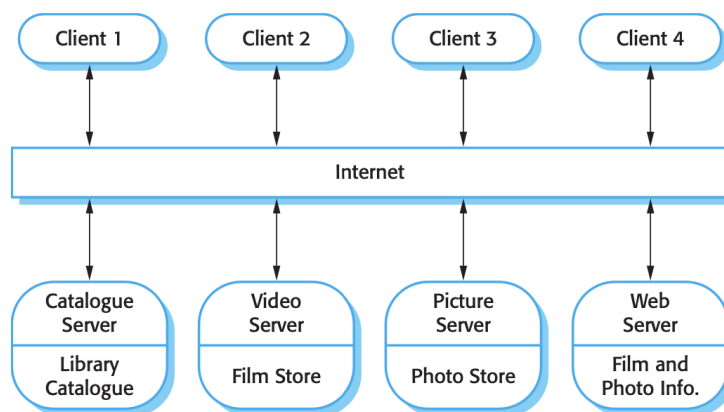
Pola *repository* berkaitan dengan struktur statis suatu sistem dan tidak menunjukkan organisasi *run-time*-nya. Contoh berikutnya menggambarkan organisasi *run-time* yang sangat umum digunakan untuk sistem terdistribusi. Pola *client-server* dijelaskan pada Tabel 4.4.

Sebuah sistem yang mengikuti pola *client-server* diatur sebagai satu set layanan dan server terkait, dan klien yang mengakses dan menggunakan layanan. Komponen utama dari model ini adalah:

- 1) Seperangkat server yang menawarkan layanan ke komponen lain. Contoh server termasuk server cetak yang menawarkan layanan pencetakan, server *file* yang menawarkan layanan manajemen *file*, dan server kompilasi, yang menawarkan layanan kompilasi bahasa pemrograman.
- 2) Sekumpulan klien yang memanggil layanan yang ditawarkan oleh server. Biasanya akan ada beberapa contoh program klien yang dijalankan secara bersamaan pada komputer yang berbeda.
- 3) Jaringan yang memungkinkan klien untuk mengakses layanan ini. Sebagian besar sistem *client-server* diimplementasikan sebagai sistem terdistribusi, terhubung menggunakan protokol Internet.

Tabel 4.4. Pola *Client-Server*

Nama	<i>Client-Server</i>
Deskripsi	Dalam arsitektur <i>client-server</i> , fungsionalitas sistem diatur ke dalam layanan, dengan setiap layanan dikirimkan dari <i>server</i> terpisah. Klien adalah pengguna layanan ini dan mengakses <i>server</i> untuk menggunakannya.
Contoh	Gambar 4.39 adalah contoh perpustakaan film dan video/DVD yang diatur sebagai sistem <i>client-server</i> .
Kapan Digunakan	Digunakan ketika data dalam <i>database</i> bersama harus diakses dari berbagai lokasi. Karena server dapat direplikasi, <i>server</i> juga dapat digunakan ketika beban pada sistem bervariasi.
Kekuatan	Keuntungan utama dari model ini adalah bahwa <i>server</i> dapat didistribusikan di seluruh jaringan. Fungsionalitas umum (misalnya, layanan pencetakan) dapat tersedia untuk semua klien dan tidak perlu diimplementasikan oleh semua layanan.
Kelemahan	Setiap layanan adalah satu titik kegagalan sehingga rentan terhadap serangan penolakan layanan atau kegagalan server. Kinerja mungkin tidak dapat diprediksi karena tergantung pada jaringan dan juga sistem. Mungkin masalah manajemen jika <i>server</i> dimiliki oleh organisasi yang berbeda.

Gambar 4.39. Arsitektur *Client-Server* untuk Perpustakaan Filmd) Arsitektur Pipa dan Filter (*Pipe and filter architecture*)

Contoh terakhir dari pola arsitektur adalah pola pipa dan filter. Ini adalah model organisasi *run-time* dari suatu sistem di mana transformasi

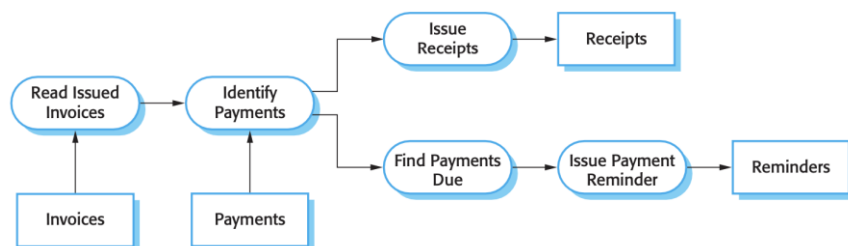
fungsional memproses input mereka dan menghasilkan *output*. Data mengalir dari satu ke yang lain dan ditransformasikan saat bergerak melalui urutan. Setiap langkah pemrosesan diimplementasikan sebagai transformasi. Data input mengalir melalui transformasi ini sampai dikonversi menjadi *output*. Transformasi dapat dijalankan secara berurutan atau paralel. Data dapat diproses oleh setiap transformasi item demi item atau dalam satu *batch*.

Nama '*pipe* dan filter' berasal dari sistem Unix asli di mana dimungkinkan untuk menghubungkan proses menggunakan '*pipe*'. Ini melewati aliran teks dari satu proses ke proses lainnya. Sistem yang sesuai dengan model ini dapat diimplementasikan dengan menggabungkan perintah Unix, menggunakan *pipe* dan fasilitas kontrol dari shell Unix. Istilah '*filter*' digunakan karena transformasi '*menyaring*' data yang dapat diproses dari aliran data inputnya.

Tabel 4.5. Pola Pipa dan Filter

Nama	Pipa dan Filter
Deskripsi	Pemrosesan data dalam suatu sistem diatur sedemikian rupa sehingga setiap komponen pemrosesan (<i>filter</i>) bersifat diskrit dan melakukan satu jenis transformasi data. Data mengalir (seperti dalam <i>pipe</i>) dari satu komponen ke komponen lain untuk diproses.
Contoh	Gambar 6.13 adalah contoh sistem <i>pipe</i> dan filter yang digunakan untuk memproses faktur.
Kapan Digunakan	Umumnya digunakan dalam aplikasi pemrosesan data (baik berbasis <i>batch</i> maupun transaksi) di mana input diproses dalam tahapan terpisah untuk menghasilkan <i>output</i> terkait.
Kekuatan	Mudah dipahami dan mendukung transformasi penggunaan kembali. Gaya alur kerja cocok dengan struktur banyak proses bisnis. Evolusi dengan menambahkan transformasi sangatlah mudah. Dapat diimplementasikan sebagai sistem sekuensial atau bersamaan.

Nama	Pipa dan Filter
Kelemahan	Format untuk transfer data harus disepakati antara transformasi komunikasi. Setiap transformasi harus mengurai inputnya dan mengurai <i>output</i> -nya ke bentuk yang disepakati. Ini meningkatkan <i>overhead</i> sistem dan mungkin berarti bahwa tidak mungkin untuk menggunakan kembali transformasi fungsional yang menggunakan struktur data yang tidak kompatibel.



Gambar 4.40. Contoh Arsitektur Pipa dan Filter

4. Arsitektur Aplikasi

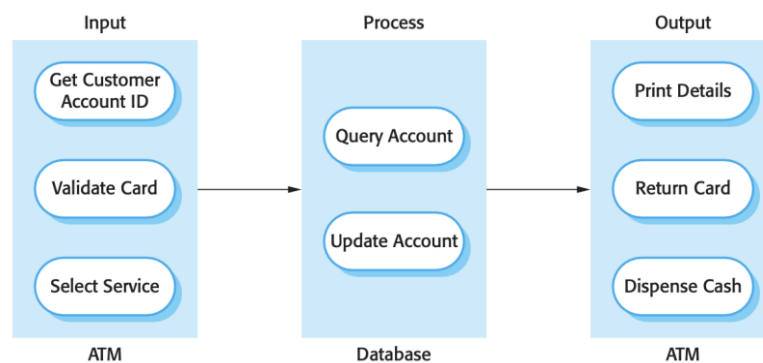
Sistem aplikasi dimaksudkan untuk memenuhi kebutuhan bisnis atau organisasi. Semua bisnis memiliki banyak kesamaan, mereka perlu mempekerjakan orang, menerbitkan faktur, menyimpan rekening, dan sebagainya. Bisnis yang beroperasi di sektor yang sama menggunakan aplikasi khusus sektor yang sama. Oleh karena itu, seperti halnya fungsi bisnis umum, semua perusahaan telepon memerlukan sistem untuk menghubungkan panggilan, mengelola jaringan mereka, mengeluarkan tagihan kepada pelanggan, dll. Akibatnya, sistem aplikasi yang digunakan oleh bisnis ini juga memiliki banyak kesamaan.

Arsitektur aplikasi dapat diimplementasikan kembali ketika mengembangkan sistem baru tetapi, untuk banyak sistem bisnis, penggunaan kembali aplikasi dimungkinkan tanpa implementasi ulang. Ini dilihat dalam pertumbuhan sistem *Enterprise Resource Planning* (ERP) dari perusahaan seperti SAP dan Oracle, dan paket perangkat lunak vertikal (COTS) untuk

aplikasi khusus di berbagai bidang bisnis. Dalam sistem ini, sistem generik dikonfigurasi dan disesuaikan untuk membuat aplikasi bisnis tertentu.

Ada banyak jenis sistem aplikasi dan dalam beberapa kasus mungkin tampak sangat berbeda. Namun, banyak dari aplikasi yang sangat berbeda ini saat ini memiliki banyak kesamaan dan dengan demikian dapat diwakili oleh arsitektur aplikasi abstrak tunggal. Hal ini diilustrasikan dengan penjelasan arsitektur berikut dari dua jenis aplikasi:

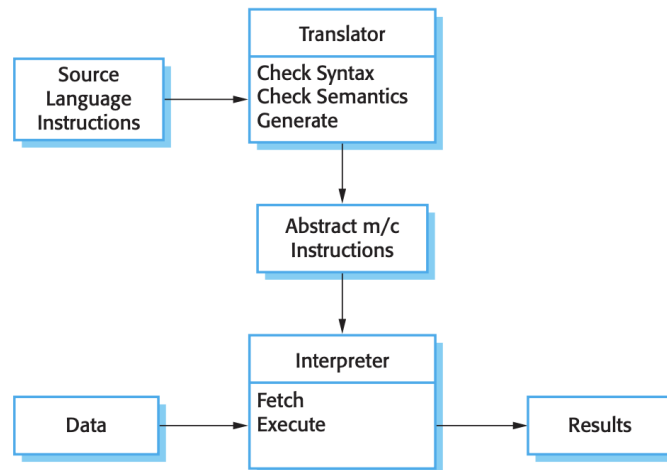
- a) Aplikasi pemrosesan transaksi (*transaction processing applications*) adalah aplikasi yang berpusat pada basis data yang memproses permintaan pengguna untuk informasi dan memperbarui informasi dalam basis data. Ini adalah jenis sistem bisnis interaktif yang paling umum. Semua diatur sedemikian rupa sehingga tindakan pengguna tidak dapat saling mengganggu dan integritas *database* tetap terjaga. Kelas sistem ini mencakup sistem perbankan interaktif, sistem *e-commerce*, sistem informasi, dan sistem pemesanan.



Gambar 4.41. Arsitektur Perangkat Lunak dari Sistem ATM

- b) Sistem pemrosesan bahasa (*language processing systems*) adalah sistem dimana maksud pengguna diekspresikan dalam bahasa formal (seperti Java). Sistem pemrosesan bahasa memproses bahasa ini ke dalam format internal dan kemudian menafsirkan representasi internal ini. Sistem pemrosesan bahasa yang paling terkenal adalah *compiler*,

yang menerjemahkan program bahasa tingkat tinggi ke dalam kode mesin. Namun, sistem pemrosesan bahasa juga digunakan untuk menginterpretasikan bahasa perintah untuk *database* dan sistem informasi, dan bahasa *markup* seperti XML.



Gambar 4.42. Arsitektur Sistem Pemrosesan Bahasa

4.2. Rangkuman

Pemodelan sistem umumnya berarti mewakili sistem menggunakan beberapa jenis notasi grafis, yang sekarang hampir selalu digunakan adalah notasi dalam *Unified Modeling Language* (UML).

Pemodelan proses sistem informasi dapat mengadopsi pendekatan berorientasi pada prosedur, berorientasi pada objek, atau berorientasi pada layanan.

Desain antarmuka pengguna harus memastikan bahwa interaksi antara manusia dan mesin menyediakan operasi dan kontrol mesin yang efektif. Perancangan antarmuka pengguna secara umum dibagi dua yaitu rancangan masukan dan rancangan keluaran.

Desain arsitektur berkaitan dengan pemahaman bagaimana sistem harus diatur dan merancang struktur keseluruhan sistem itu. Ada empat

pandangan yang dapat digunakan untuk merancang arsitektur, yaitu pandangan logis, proses, pengembangan, dan fisik. Selain itu, dalam merancang arsitektur perlu memilih pola yang cocok digunakan sesuai dengan kondisi yang dihadapi dengan mempelajari kekuatan dan kelemahan masing-masing pola arsitektur.

4.3. Soal Latihan

Buatlah Diagram *Use Case* berdasarkan *stakeholder request* “Diperlukan mekanisme *monitoring* dan evaluasi yang *real-time* dan komprehensif pada kegiatan pendataan Survei Biaya Hidup yang dilakukan BPS yang mampu mendeteksi wilayah mana yang belum dan sudah selesai dilakukan pendataan, mampu memberikan informasi berapa jumlah rumah tangga yang berhasil didata dan non respons, dan mampu menyajikan laporan data statistik konsumsi rumah tangga”.

4.4. Contoh Kasus

BPS Provinsi Y membutuhkan suatu sistem yang dapat mengelola pertukaran data registrasi penduduk dengan sistem yang ada di Dinas Dukcapil setempat. Pertukaran data dilakukan dengan memanfaatkan jaringan internet dikarenakan tidak tersedia jalur VPN khusus. Data yang diperoleh akan ditampilkan dalam sebuah *dashboard*. Berdasarkan kebutuhan tersebut, pola arsitektur apa yang menurut Anda paling cocok diterapkan dan sertakan hasil rancangan arsitekturnya. Anda dapat menambahkan asumsi jika diperlukan.

BAB V PEMBANGUNAN, PENGEMBANGAN, DAN UJI COBA SISTEM INFORMASI

5.1. Uraian Materi

Rekayasa perangkat lunak mencakup semua aktivitas yang terlibat dalam pengembangan perangkat lunak mulai dari persyaratan awal sistem hingga pemeliharaan dan pengelolaan sistem yang digunakan. Tahap kritis dari proses ini, tentu saja, adalah implementasi sistem, di mana Anda membuat versi perangkat lunak yang dapat dieksekusi. Implementasi mungkin melibatkan pengembangan program dalam bahasa pemrograman tingkat tinggi atau rendah atau menyesuaikan dan mengadaptasi sistem generik untuk memenuhi persyaratan spesifik organisasi. Pada bagian ini akan dibahas mengenai pembangunan, pengembangan, dan uji coba sistem informasi.

a. Algoritma Pemrograman

1. Pengertian Algoritma Pemrograman

Algoritma adalah langkah-langkah yang disusun secara tertulis dan berurutan untuk menyelesaikan suatu masalah. Sedangkan Algoritma Pemrograman adalah langkah-langkah yang ditulis secara berurutan untuk menyelesaikan masalah pemrograman komputer. Dalam pemrograman yang sederhana, algoritma merupakan langkah pertama yang harus ditulis sebelum menuliskan program. Masalah yang dapat diselesaikan dengan pemrograman komputer adalah masalah-masalah yang berhubungan dengan perhitungan matematik.

- Program adalah serangkaian instruksi untuk melakukan suatu fungsi spesifik pada komputer.
- Pemrograman adalah proses menulis, menguji, dan memperbaiki (*debug*), serta memelihara kode yang membangun suatu program komputer.
- Unsur-unsur pemrograman: Input → Proses → Output.
- Beda Algoritma dan Program:

Program = Algoritma + Bahasa (Struktur Data)

2. Paradigma Pemrograman

Paradigma pemrograman merupakan cara pandang untuk menyelesaikan suatu masalah dengan cara pemrograman. Paradigma pemrograman penting bagi seorang *programmer* untuk dapat mengidentifikasi sebuah masalah sebelum mempersiapkan solusinya dengan sebuah program komputer.

Saat ini ada beberapa paradigma pemrograman yang banyak diimplementasikan oleh para *programmer*, antara lain:

- Paradigma Prosedural atau Imperatif, bersumber pada konsep mesin Von Newman (*stored program concept*) dimana tempat penyimpanan (memori) dibedakan menjadi memori instruksi dan memori data, dan masing-masing memori tersebut dapat diberi nama dan nilai untuk kemudian dieksekusi satu persatu. Kelebihan dari paradigma ini adalah sangat dekat dengan bahasa mesin, sedangkan kekurangannya adalah banyaknya batasan-batasan yang terkadang menyulitkan seorang *programmer*. Contoh bahasa pemrograman yang menggunakan paradigma prosedural atau imperatif adalah bahasa pemrograman Fortran dan Cobol.

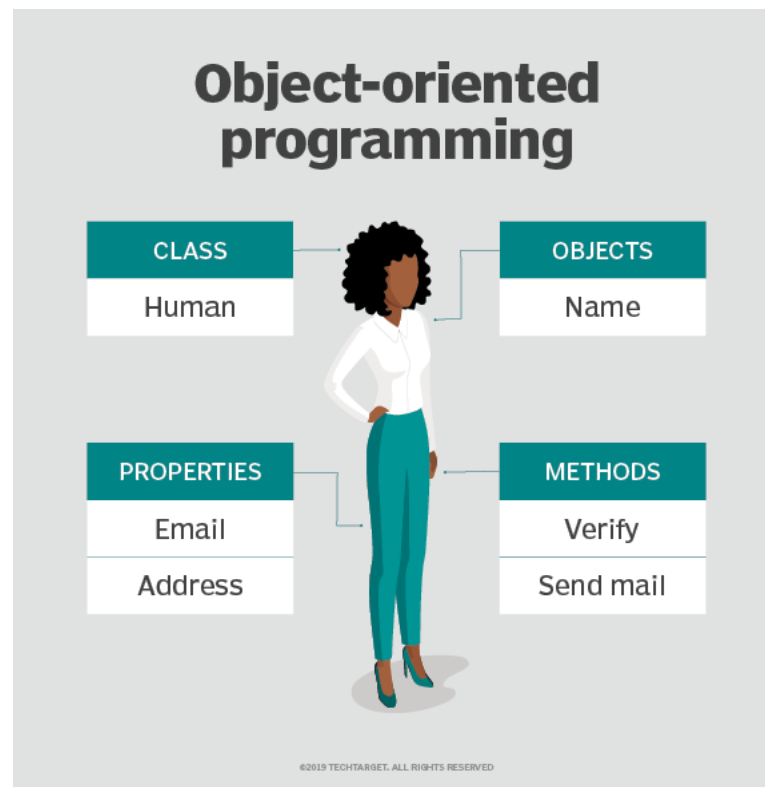
```

000024
000025  PROCEDURE DIVISION.
000026  0001-MAIN.
000027      INSPECT FUNCTION REVERSE(STR-1)
000028          TALLYING WS-LEN1 FOR LEADING SPACES.
000029      COMPUTE WS-LEN = LENGTH OF STR-1 - WS-LEN1.
000030      DISPLAY WS-LEN.
000031      MOVE 1 TO I.
000032      MOVE WS-LEN TO J.
000033      PERFORM REV-PARA WS-LEN TIMES.
000034      DISPLAY STR-1.
000035      DISPLAY STR-2.
000036      GOBACK.
000037  REV-PARA.
000038      MOVE STR-1(J:1) TO STR-2(I:1).
000039      SUBTRACT 1 FROM J.
000040      ADD 1 TO I.
000041      EXIT.
***** ***** Bottom of Data *****

```

Gambar 5.1. Contoh Paradigma Prosedural atau Imperatif

- Paradigma Fungsional, didasari oleh konsep pemetaan dan fungsi matematika. Dalam paradigma ini, diasumsikan bahwa akan selalu ada fungsi-fungsi dasar yang dapat digunakan, sehingga penyelesaian masalah berdasarkan pada fungsi-fungsi yang telah tersedia tersebut. Contoh bahasa pemrograman yang menggunakan paradigma fungsional adalah bahasa pemrograman LOGO, APL dan LISP.
- Paradigma *Logical Declarative*, didasari oleh konsep pendefinisian relasi antar individu yang dinyatakan sebagai predikat. Dalam paradigma ini, seorang *programmer* diminta menguraikan sekumpulan fakta dan aturan-aturan yang akan menjadi panduan ketika program dieksekusi. Contoh bahasa pemrograman yang menggunakan paradigma ini adalah bahasa pemrograman Prolog.
- Paradigma *Object Oriented*, menggunakan konsep *class* dan *object* untuk menyelesaikan permasalahan-permasalahan. *Object* adalah proses instansiasi atau proses membuat objek dari sebuah *class*, dimana setiap *object* akan mempunyai *attribute* dan *method*, dan masing-masing *object* dapat berinteraksi dengan *object* lainnya meskipun berasal dari *class* yang berbeda. Contoh bahasa pemrograman yang menggunakan paradigma ini adalah bahasa pemrograman Delphi, Java, C++, PHP, Python.



Gambar 5.2. Contoh Paradigma *Object Oriented*

- Paradigma *Concurrent*, didasari oleh kenyataan bahwa sebuah sistem komputer harus menangani beberapa program (*task*) yang harus dieksekusi secara bersamaan dalam suatu waktu. Dalam paradigma ini, seorang *programmer* harus mampu menangani komunikasi dan sinkronisasi antar *task* yang dijalankan oleh komputer.
- Paradigma *Multi Programming*, didasari kebutuhan bahwa bahasa pemrograman yang dapat mendukung lebih dari satu paradigma pemrograman. Gagasan utama dari paradigma *multi programming* adalah untuk menyediakan kerangka kerja dimana para *programmer* dapat bekerja dalam berbagai gaya, secara bebas mencampurkan konstruksi dari paradigma yang berbeda, dan sebagai solusi bahwa pada beberapa tahap satu paradigma tidak mampu menyelesaikan semua masalah dengan cara termudah atau paling efisien. Contoh

bahasa pemrograman yang menggunakan paradigma ini adalah bahasa pemrograman Wolfram.

3. Program *Flowchart*

Flowchart adalah sebuah diagram yang menjelaskan alur proses dari sebuah program. Dalam membangun sebuah program, *flowchart* berperan penting untuk menerjemahkan proses berjalannya sebuah program agar lebih mudah untuk dipahami.

Fungsi utama dari *flowchart* adalah memberi gambaran jalannya sebuah program dari satu proses ke proses lainnya. Sehingga, alur program menjadi mudah dipahami oleh semua orang. Selain itu, fungsi lain dari *flowchart* adalah untuk menyederhanakan rangkaian prosedur agar memudahkan pemahaman terhadap informasi tersebut. *Flowchart* sendiri terdiri dari lima jenis, masing-masing jenis memiliki karakteristik dalam penggunaannya. Berikut adalah jenis-jenisnya:

a) *Flowchart* Dokumen

Pertama ada *flowchart* dokumen (*document flowchart*) atau bisa juga disebut dengan *paperwork flowchart*. *Flowchart* dokumen berfungsi untuk menelusuri alur *form* dari satu bagian ke bagian yang lain, termasuk bagaimana laporan diproses, dicatat, dan disimpan.

b) *Flowchart* Program

Flowchart program menggambarkan secara rinci prosedur dari proses program. *Flowchart* program terdiri dari dua macam, antara lain: *flowchart* logika program (*program logic flowchart*) dan *flowchart* program komputer terinci (*detailed computer program flowchart*).

c) *Flowchart* Proses

Flowchart proses adalah cara penggambaran rekayasa industrial dengan cara merinci dan menganalisis langkah-langkah selanjutnya dalam suatu prosedur atau sistem.

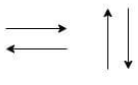
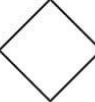
d) *Flowchart* Sistem

Flowchart sistem adalah *flowchart* yang menampilkan tahapan atau proses kerja yang sedang berlangsung di dalam sistem secara menyeluruh. Selain itu *flowchart* sistem juga menguraikan urutan dari setiap prosedur yang ada di dalam sistem.

e) *Flowchart* Skematik

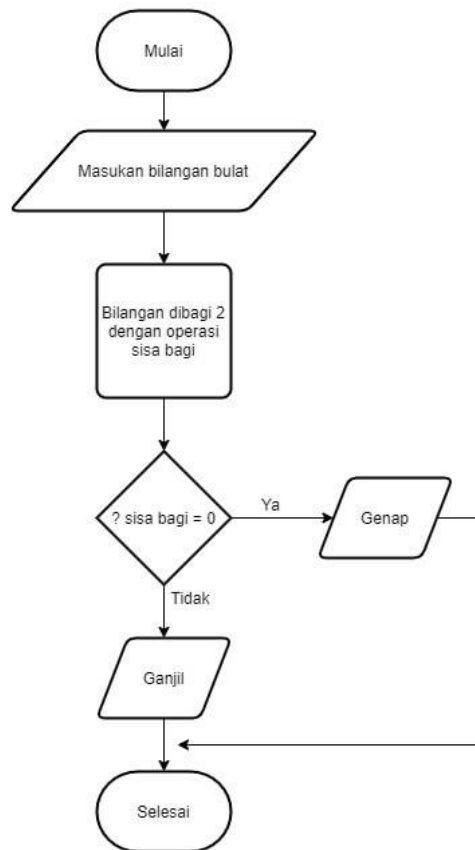
Flowchart skematik ini menampilkan alur prosedur suatu sistem, hampir sama dengan *flowchart* sistem. Namun, ada perbedaan dalam penggunaan simbol-simbol dalam menggambarkan alur. Selain simbol-simbol, *flowchart* skematik juga menggunakan gambar-gambar komputer serta peralatan lainnya untuk mempermudah dalam pembacaan *flowchart* untuk orang awam.

Pada dasarnya simbol-simbol dalam *flowchart* memiliki arti yang berbeda-beda. Simbol-simbol tersebut memiliki jenis dan fungsi yang berbeda-beda. Ada yang berfungsi untuk menghubungkan satu simbol dengan simbol lainnya seperti simbol *flow*, *on-page* dan *off-page reference*. Selain itu ada juga simbol yang berfungsi untuk menunjukkan suatu proses yang sedang berjalan, dan yang terakhir terdapat simbol yang berfungsi untuk memasukkan input dan menampilkan *output*. Berikut adalah simbol-simbol yang sering digunakan dalam proses pembuatan *flowchart*.

	Flow Simbol yang digunakan untuk menggabungkan antara simbol yang satu dengan simbol yang lain. Simbol ini disebut juga dengan Connecting Line.		Input/output Simbol yang menyatakan proses input atau output tanpa tergantung peralatan.
	On-Page Reference Simbol untuk keluar - masuk atau penyambungan proses dalam lembar kerja yang sama.		Manual Operation Simbol yang menyatakan suatu proses yang tidak dilakukan oleh komputer.
	Off-Page Reference Simbol untuk keluar - masuk atau penyambungan proses dalam lembar kerja yang berbeda.		Document Simbol yang menyatakan bahwa input berasal dari dokumen dalam bentuk fisik, atau output yang perlu dicetak.
	Terminator Simbol yang menyatakan awal atau akhir suatu program.		Predefine Proses Simbol untuk pelaksanaan suatu bagian (sub-program) atau prosedur.
	Process Simbol yang menyatakan suatu proses yang dilakukan komputer.		Display Simbol yang menyatakan peralatan output yang digunakan.
	Decision Simbol yang menunjukan kondisi tertentu yang akan menghasilkan dua kemungkinan jawaban, yaitu ya dan tidak.		Preparation Simbol yang menyatakan penyediaan tempat penyimpanan suatu pengolahan untuk memberikan nilai awal.

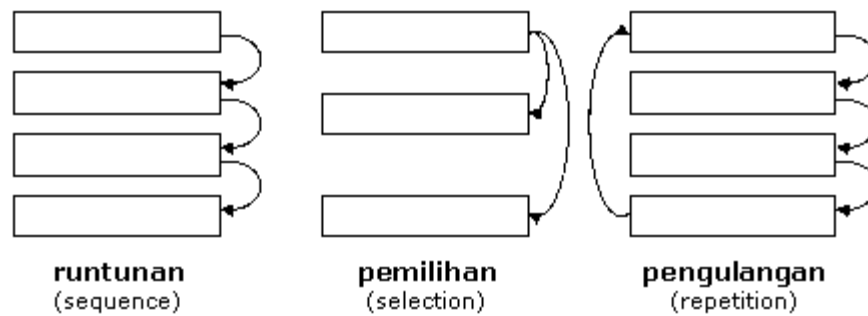
Gambar 5.3. Notasi *Flowchart*

Untuk membuat sebuah *flowchart* harus mengetahui setiap jenis simbol dan juga fungsinya dari masing-masing simbol tersebut. Untuk memperjelas dan membangun pemahaman dari penggunaan simbol dari *flowchart*, berikut ini sebuah contoh *flowchart* sederhana untuk menentukan apakah bilangan yang dimasukkan ganjil atau genap, berikut contohnya:

Gambar 5.4. Contoh *Flowchart*

4. Struktur Alur Algoritma

Secara umum, struktur dasar algoritma terdiri dari sekuensial (*sequential*), percabangan (*branching*), dan perulangan (*looping*). Ketiga struktur dasar ini sering ditemukan saat pembuat program menuliskan algoritmanya, entah itu dengan menggunakan *pseudocode* ataupun dengan menggunakan *flowchart* atau diagram alir. Berikut ini adalah ilustrasi untuk ketiga struktur dasar algoritma yang akan kita pelajari kali ini.



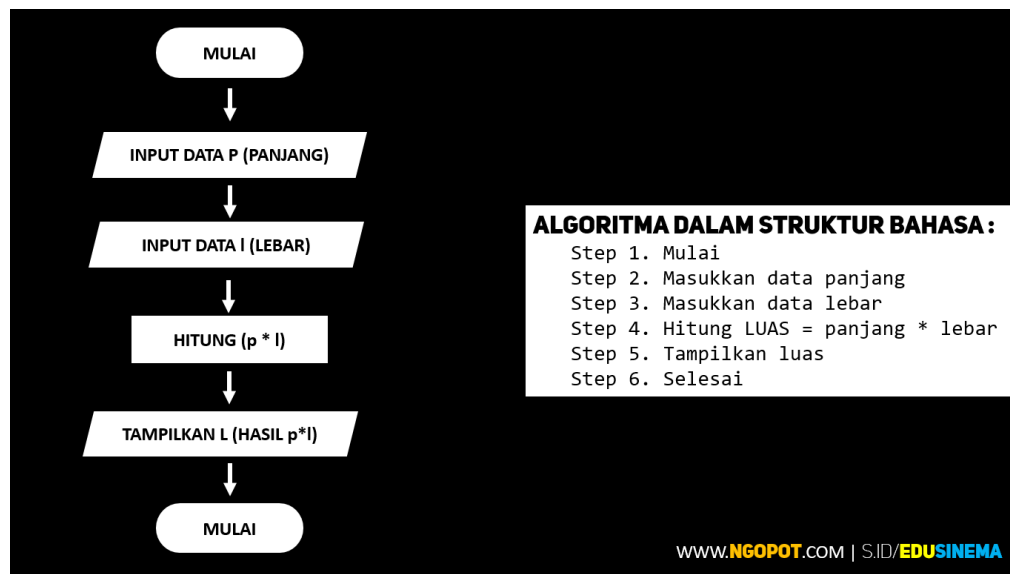
Gambar 5.5. Struktur Alur Algoritma

a) Struktur Dasar Berurutan (*Sequence*)

Struktur urut adalah struktur yang digunakan untuk mengerjakan jenis program yang pernyataannya *sequential* atau berurutan. Pada struktur ini, perintah yang diberikan secara beruntun atau berurutan baris per baris mulai dari awal hingga akhir. Struktur urutan tidak memuat lompatan atau pengulangan di dalamnya. Instruksi dalam struktur urut memiliki karakteristik seperti:

- 1) Tiap perintah dikerjakan satu persatu sebanyak sekali
- 2) Pelaksanaan perintah dilakukan secara berurutan
- 3) Perintah terakhir merupakan akhir dari algoritma
- 4) Perubahan urutan dapat menyebabkan hasil yang berbeda

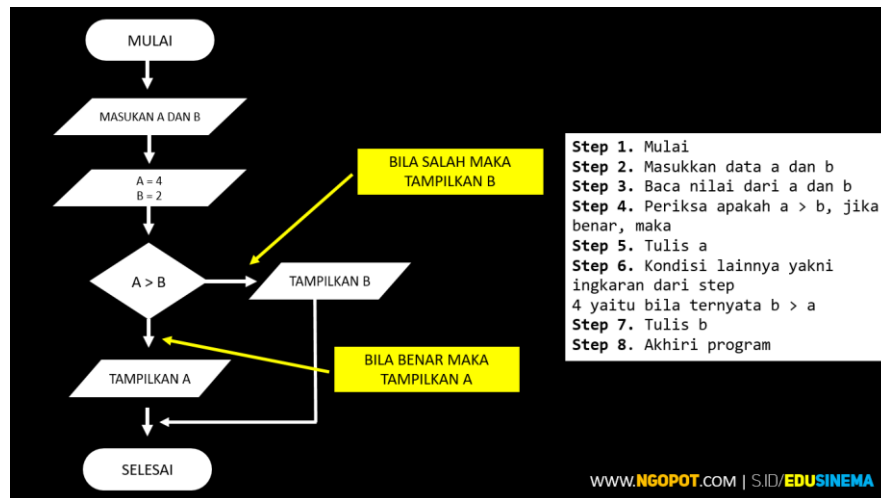
Struktur urut dalam sains biasanya digunakan untuk melakukan perhitungan pada kasus yang melibatkan rumus-rumus sederhana dengan melibatkan operator penjumlahan, pengurangan, dan perkalian. Beberapa contoh kasus yang dapat menerapkan logika dengan struktur urut adalah perhitungan suatu besaran dengan rumus sederhana seperti jarak tempuh, luas persegi panjang, luas lingkaran, perhitungan upah pegawai, dan sejenisnya.



Gambar 5.6. Contoh Struktur Dasar Berurutan

b) Struktur Dasar Pemilihan (*Selection*)

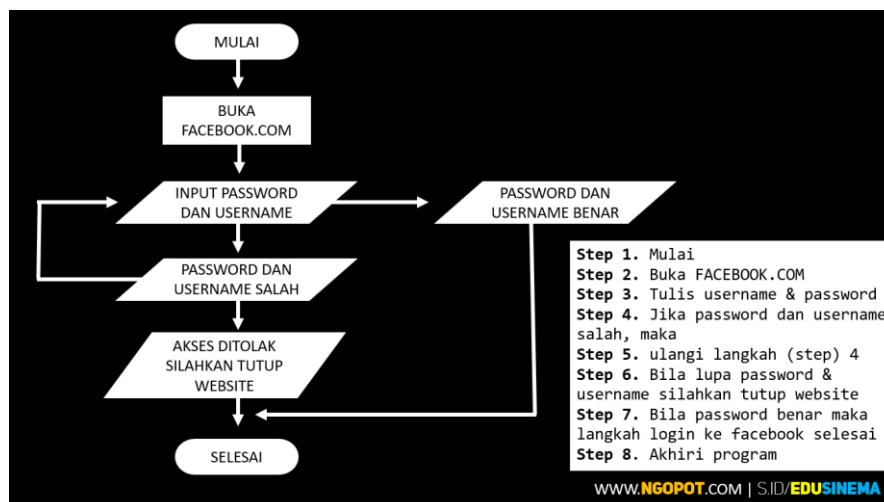
Struktur Pemilihan adalah struktur yang digunakan pada program yang memerlukan proses pengujian kondisi untuk mengambil suatu keputusan apakah suatu baris perintah akan diproses atau tidak. Pengujian kondisi ini dilakukan untuk memilih salah satu dari beberapa alternatif yang tersedia. Tidak semua baris program akan dikerjakan pada struktur ini, melainkan hanya baris yang memenuhi syarat saja. Secara umum, perintah dalam struktur ini berjalan secara runtun atau berurut mulai dari perintah pertama hingga akhir, namun perintah dapat dibuat berpindah ke perintah lain atau berhenti jika syarat yang ditentukan terpenuhi.



Gambar 5.7. Contoh Struktur Dasar Pemilihan

c) Struktur Dasar Pengulangan (*Repetition*)

Salah satu kelebihan komputer adalah kemampuannya untuk mengerjakan perintah secara berulang kali tanpa lelah. Ini berbeda dengan manusia yang cepat lelah bila mengerjakan pekerjaan yang sama berulang-ulang dan bisa jadi membosankan. Struktur Pengulangan adalah struktur yang melakukan pengulangan beberapa kali terhadap satu baris atau satu blok baris program. Pengulangan akan dilakukan sesuai dengan persyaratan yang diberikan.



Gambar 5.8. Contoh Struktur Dasar Pengulangan

b. Pemrograman Berorientasi Objek (PBO)**1. Pengertian PBO**

PBO (*Object Oriented Programming/OOP*) merupakan paradigma pemrograman berdasarkan konsep objek yang dapat berisi data, dalam bentuk *field* atau dikenal juga sebagai atribut serta kode, dalam bentuk fungsi/prosedur atau dikenal juga sebagai *method*. Semua data dan fungsi di dalam paradigma ini dibungkus dalam kelas-kelas atau objek-objek. Bandingkan dengan logika pemrograman terstruktur. Setiap objek dapat menerima pesan, memproses data, dan mengirim pesan ke objek lainnya.

Model data berorientasi objek dikatakan dapat memberi fleksibilitas yang lebih, kemudahan mengubah program, dan digunakan luas dalam teknik peranti lunak skala besar. Tujuan dari PBO diciptakan adalah untuk mempermudah pengembangan program dengan cara mengikuti model yang telah ada di kehidupan sehari-hari. Jadi setiap bagian dari suatu permasalahan adalah objek, nah suatu objek itu sendiri merupakan gabungan dari beberapa objek yang lebih kecil lagi.

2. Konsep Dasar PBO

PBO sendiri memiliki beberapa konsep dasar seperti berikut ini:

a) Kelas (*Class*)

Class bertugas untuk mengumpulkan prosedur/fungsi dan variabel dalam satu tempat. *Class* merupakan *blueprint* dari sebuah objek atau cetakan untuk membuat objek. *Class* akan merepresentasikan objek yang mau dibuat. Jadi dalam membuat nama kelas harus disesuaikan dengan objek yang akan dibuat. Penulisan nama *class* memiliki aturan. Yakni dengan format Pascal Case. Apa itu? Penulisannya diawali dengan huruf kapital. Jika nama variabel tersusun dari dua kata atau lebih maka tidak perlu diberi spasi di antaranya dan diawali dengan huruf kapital pula.

Misal: `class MakananKucing, class Senjata, dan`
`class SignIn`

- ▶ Bagian atas menunjukkan nama kelas
- ▶ Bagian kedua diisi dengan field/atributnya → tanda (-) menunjukkan field tersebut hanya berlaku dalam kelas (*private*).
- ▶ Bagian ketiga berisi daftar fungsi dari kelas ini → tanda (+) menunjukkan fungsi tersebut bisa diakses oleh obyek lain (*public*).

DOSEN
-nama
-NIP
-tanggalMasuk
-status
-spesialisasi
+MasukkanNilai()
+Cuti()
+JumlahMataKuliah()

Gambar 5.9. Contoh Kelas

b) Objek (*Object*)

Objek adalah sebuah variabel *instance* yang merupakan wujud dari *class*. *Instance* merupakan wujud dari sebuah kelas. Sebuah objek digambarkan dengan *variable* dan *method*. Objek memiliki batasan (*boundary*) dan identitas (*identity*) yang terdefinisi dengan jelas. Sebuah objek memiliki karakteristik sebagai berikut :

1) Kondisi (State)

Kondisi dinyatakan sebagai sekelompok properti yang disebut atribut, berikut juga nilai dari setiap properti ini. Contoh:

- ✓ Mobil adalah sebuah objek
- ✓ Memiliki properti warna, jumlah roda, kapasitas mesin, jumlah pintu dll.
- ✓ Setiap properti ini memiliki nilai

Tabel 5.1. Contoh *Property* Objek

Warna	Hijau
Jumlah roda	4 roda
Kapasitas mesin	2000 cc
Jumlah pintu	4 pintu

2) Perilaku (*Behavior*)

Perilaku dinyatakan dengan operasi-operasi yang dapat dilakukan oleh objek untuk manipulasi data yang dibutuhkan, fungsi-fungsi pendukung, atau interaksi yang diperbolehkan oleh objek, berikut contohnya :

- ✓ Objek *Software*
- ✓ Atribut sebuah *software* didefinisikan dengan variabel.
- ✓ Sebuah *software* juga mengimplementasikan perilakunya ke dalam sebuah metode (sebuah fungsi yang berhubungan dengan objek tertentu)

Hubungan antar objek ditentukan berdasarkan skenario (gambaran tentang proses yang akan menggunakan objek). Skenario dibutuhkan untuk mengetahui sebuah objek dalam konteks tertentu dan menentukan properti atau atribut yang tepat. Ada 3 tipe dasar hubungan objek yaitu subkelas (hubungan “adalah”), kontainer (hubungan “memiliki”), dan kolaborator (hubungan “menggunakan”).

3. Prinsip Dasar PBO

a) Abstraksi (*Abstraction*)

Kemampuan sebuah program untuk melewati aspek informasi yang diolah adalah kemampuan untuk fokus pada inti permasalahan. Setiap objek dalam sistem melayani berbagai model dari pelaku abstrak yang dapat melakukan kerja, laporan dan perubahan serta berkomunikasi dengan objek lain dalam sistem, tanpa harus menampilkan kelebihan diterapkan.

b) Enkapsulasi (Pembungkus)

Enkapsulasi adalah sebuah konsep pembungkusan data bersama metodenya, dimana hal ini bertujuan untuk menyembunyikan rincian implementasi dari pemakai. Tujuan dari enkapsulasi ini adalah untuk

menjaga suatu proses program agar tidak dapat diakses secara sembarangan atau diintervensi oleh program lain.

Penerapan konsep enkapsulasi dalam kehidupan sehari-hari dapat ditemukan pada penggunaan arus listrik pada generator, dan sistem perputaran generator untuk menghasilkan arus listrik. Kerja arus listrik tidak mempengaruhi kerja dari sistem perputaran generator, begitu pula sebaliknya. Karena di dalam arus listrik tersebut, kita tidak perlu mengetahui bagaimana kinerja sistem perputaran generator, apakah generator berputar ke belakang atau ke depan atau bahkan serong. Begitu pula dalam sistem perputaran generator, kita tidak perlu tahu bagaimana arus listrik, apakah menyala atau tidak. Keuntungan dari enkapsulasi ini adalah memberikan kemudahan dalam perawatan program dan dalam hal pencarian kesalahan.

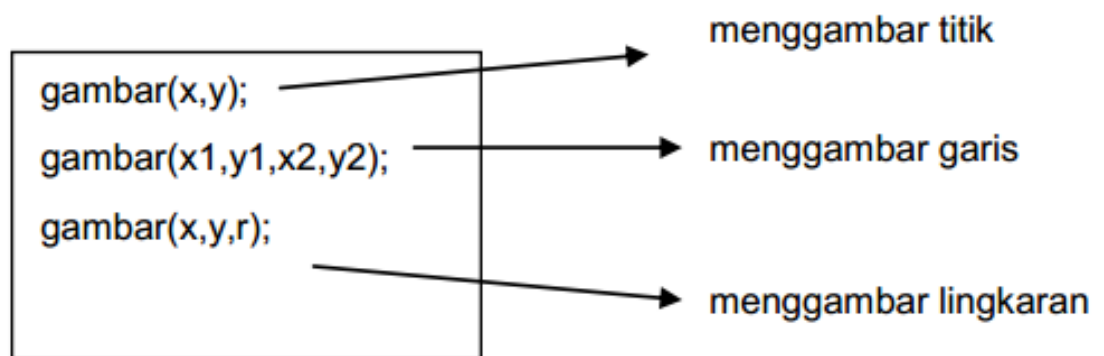
```
1. <?php
2. class a {
3.     const C = 10;
4.     var $aVariable ;
5.     function __construct() {
6.     }
7.     function __destruct() {
8.     }
9.     function doSomething() {
10.    }
11. }
12. ?>
```

Gambar 5.10. Contoh Enkapsulasi

c) Polimorfisme

Polymorphism secara umum adalah penggunaan suatu item baik *interface*, fungsi, dan lain-lain pada berbagai macam jenis objek maupun *entity* yang berbeda-beda dengan syarat suatu objek atau *entity* tersebut memiliki relasi yang menjembatani agar akses ke item tersebut dapat didapatkan.

Dalam konteks *Object Oriented Programming* (OOP), *polymorphism* terjadi ketika kita memiliki banyak *class* yang memiliki relasi satu sama lain menggunakan *Inheritance*. Seperti yang kita ketahui, *inheritance* memungkinkan kita untuk menurunkan *attribute*, *method* atau fungsi pada *class* lainnya. Namun selain itu, dilihat dari sisi bahasa pemrograman pada umumnya, *polymorphism* juga dapat diimplementasikan menggunakan baik *inheritance* maupun *interface*.



Gambar 5.11. Contoh Polimorfisme

d) *Inheritance* (Pewarisan)

Konsep pewarisan mempunyai fungsi mengatur polimorfisme dan enkapsulasi dengan mengizinkan objek didefinisikan dan diciptakan dengan jenis khusus dari objek yang sudah ada. Objek-objek ini dapat membagi dan memperluas perilaku mereka tanpa mengimplementasikan perilaku tersebut.

c. Pemrograman *Database*

1. Pengertian Program *Database*

Database atau basis data adalah sekumpulan data yang dikelola dengan sedemikian rupa berdasarkan ketentuan tertentu yang saling berkaitan sehingga memudahkan dalam pengelolaannya. Lewat pengelolaan itulah

pengguna bisa mendapatkan kemudahan dalam mencari sebuah informasi, membuang informasi, maupun menyimpan informasi.

Database berwujud tabel yang terdiri dari kolom dan baris yang memuat atribut dan nilai tertentu. Adapun jumlah kolom dan baris dalam suatu *database* tergantung pada jumlah kategori atau jenis informasi yang perlu disimpan. Fungsi *database* adalah untuk menghindari data ganda yang tersimpan. Suatu *database management system* (DBMS) dapat diatur supaya bisa mengenali duplikasi data ketika diinput. Namun selain untuk menghindari data ganda, *database* memiliki fungsi lainnya, antara lain:

a) Kecepatan dan kemudahan:

Sistem *database* memberikan kemampuan dalam seleksi data menjadi satu kelompok yang terurut dengan cepat. Instrumen tersebut menghasilkan pencarian informasi yang dibutuhkan ditemukan dengan cepat. Kecepatannya juga dipengaruhi oleh jenis *database* yang digunakan. Setiap jenis *database* memberikan kemampuan yang berbeda-beda.

b) *Multi-user*:

Database memberikan kemudahan akses bagi banyak pengguna dalam waktu yang bersamaan. Sistem tersebut memungkinkan akses suatu dokumen ke lebih dari satu pengguna. Sehingga kinerja mesin dan jaringan dimudahkan melalui *multi-user* karena penyimpanan hanya terdiri satu unit yang dapat diakses secara bersamaan.

c) Keamanan data:

Sistem *database* melalui bahasa pemrogramannya telah dibuat secara *safety*. Melalui instrumen *password* membuat data tersebut hanya bisa diakses kepada pihak yang diizinkan. Manajemen tersebut telah diterapkan pada hampir seluruh jenis sistem *database*. Sehingga menjadikan keamanan data merupakan hal prioritas bagi layanan sistem *database*.

d) Penghematan biaya perangkat:

Memiliki satu *database* terpusat sudah cukup bagi perusahaan besar yang membutuhkan pengumpulan data secara ringkas. Hal ini membuat perusahaan tidak memerlukan ruang penyimpanan di tiap tempat yang berbeda. Melalui jaringan internet, cabang perusahaan di daerah terpencil pun bisa melakukan akses data yang ada di pusat.

e) Kontrol data terpusat:

Database tidak memerlukan *server* lebih dari satu dalam penggunaannya. Cukup satu *server* terpusat untuk menyimpan data sehingga data tersebut bisa diakses oleh banyak pengguna. Hal ini memberikan harga yang murah bagi perusahaan untuk investasi ruang penyimpanan data penting perusahaan. Seperti kantor perusahaan tidak perlu membuat suatu data di tiap divisi pekerjaannya. Setiap divisi bisa mengumpulkan data khusus melalui satu *server* yang ditentukan sehingga laporan untuk atasan menjadi ringkas.

f) Mudah membuat aplikasi:

Melalui kaitannya terhadap perusahaan jika perusahaan membutuhkan aplikasi input data yang baru, *programmer* tidak perlu membuat ulang struktur *database*. Menggunakan struktur *database* yang dibuat sebelumnya sudah cukup untuk mengenali aplikasi input data yang baru.

2. SQL (*Statement Query Language*)

SQL adalah bahasa *query* yang dirancang untuk membantu dalam pengambilan dan mengelola informasi pada sebuah *database*. Untuk yang masih pemula dalam dunia IT, biasanya diartikan sebagai bahasa yang digunakan untuk mengakses sebuah data dalam basis relasional. Terkait dengan standarisasi SQL sudah ada sejak tahun 1986 dan diinisialisasi oleh ANSI (American National Standard Institute). Hingga sekarang, banyak sekali *server* yang dapat mengartikan SQL, baik dari *database* maupun *software*.

Terdapat beberapa fungsi yang dimiliki oleh bahasa *query* SQL. Berikut merupakan beberapa fungsi dari bahasa pemrograman ini:

a) Dapat Memanipulasi dan Mengakses *Database*

Fungsi yang pertama adalah dengan menggunakan SQL, maka kita dapat mengakses *database* dengan menuliskan beberapa perintah sesuai dengan *query* yang telah ditetapkan. Misalnya saja, anda dapat membuat, menambahkan, mengubah, dan menghapus basis data, tabel, dan beberapa informasi yang tidak dibutuhkan sistem.

b) Mampu untuk Mengeksekusi *Query*

Fungsi yang kedua adalah mampu untuk mengeksekusi berbagai *query* yang ada. Penggunaan dari masukan *query* tersebut bertujuan untuk memberikan perintah langsung kepada sistem untuk dapat mengelola sebuah sistem *database*. Contoh dari beberapa eksekusi *query* adalah fungsi *trigger*, *alter*, *grant*, dan lain sebagainya.

c) Dapat Mengatur Hak Akses User

Dan fungsi yang terakhir adalah untuk mengatur dan mengelola kebutuhan hak akses tabel, pandangan, dan prosedur pada *database*. Tujuan dari adanya hak akses ini adalah untuk membatasi akses pengguna sesuai dengan kebutuhan sistem yang diterapkan.

Setelah memahami beberapa fungsi dari SQL, kini saatnya untuk lebih mengetahui apa saja perintah-perintah dasar yang terdapat dalam SQL tersebut. Setidaknya ada tiga jenis perintah dasar dalam SQL yang penjabarannya akan diulas dalam poin-poin berikut ini:

a) *Data Definition Language* (DDL)

Data Definition Language (DDL) adalah perintah yang digunakan untuk mendefinisikan data seperti membuat tabel *database* baru, mengubah *dataset*, dan menghapus data. Kemudian, perintah dasar DDL masih dibedakan lagi ke dalam setidaknya lima jenis perintah yakni bisa kamu lihat di bawah ini.

- Perintah *Create*: perintah untuk membuat tabel baru di dalam sebuah *database* adalah *create*. Tak cuma untuk tabel baru, tapi juga *database* maupun kolom baru. Kamu bisa membuat sebuah *query* dengan contoh “CREATE DATABASE nama_database”.
- Perintah *Alter*: biasa digunakan ketika seseorang ingin mengubah struktur tabel yang sebelumnya sudah ada. Bisa jadi dalam hal ini adalah seperti nama tabel, penambahan kolom, mengubah, maupun menghapus kolom serta menambahkan atribut lainnya.
- Perintah *Rename*: dapat kamu gunakan untuk mengubah sebuah nama di sebuah tabel ataupun kolom yang ada. Bila kamu menggunakan perintah ini maka *query*-nya menjadi “RENAME TABLE nama_tabel_lama TO nama_tabel_baru”
- Perintah *Drop*: Bisa kamu gunakan dalam menghapus baik itu berupa *database*, *table* maupun kolom hingga *index*.
- Perintah *Show*: perintah DDL ini digunakan untuk menampilkan sebuah tabel yang ada.

b) *Data Manipulation Language (DML)*

Pada *database* SQL, perintah yang digunakan untuk memanipulasi data adalah DML. Perintah dalam DML juga terbagi ke dalam empat jenis, beberapa di antaranya adalah *insert*, *select*, *update*, dan *delete*:

- Perintah *Insert*: kita bisa menggunakan perintah ini untuk memasukkan sebuah *record* baru di dalam sebuah tabel *database*.

```
INSERT INTO pegawai (nip,nama, umur, alamat, kota)
VALUES ('34005320', 'Dudi Barmana', 35, 'Jl. Angsoka jaya No 8', 'Bogor');
```

- Perintah *Select*: *Select* digunakan untuk memanipulasi data dengan tujuan menampilkan maupun mengambil sebuah data pada tabel. Data yang diambil pun tidak hanya terbatas pada satu jenis saja melainkan lebih dari satu tabel dengan memakai relasi.

```
SELECT Kontak>NamaDepan, Kontak>NamaKeluarga, Sales.* FROM Kontak, Sales WHERE
Kontak.ID = Sales.ID
```


- Perintah *update*: Digunakan ketika ingin melakukan pembaruan data di sebuah tabel. Contohnya saja jika ada kesalahan ketika memasukkan sebuah *record*. Kita tidak perlu menghapusnya dan bisa diperbaiki menggunakan perintah ini.

```
UPDATE pegawai  
SET nama='Miswar', umur=26  
WHERE nip='340053210';
```

- Perintah *Delete*: Perintah DML ini dapat digunakan ketika kita ingin menghapus sebuah *record* yang ada dalam sebuah tabel.

```
DELETE FROM pegawai  
WHERE nip='340053210';
```

c) Data Control Language (DCL)

Perintah dasar berikutnya adalah DCL. Perintah SQL ini digunakan khususnya untuk mengatur hak apa saja yang dimiliki oleh pengguna. Baik itu hak terhadap sebuah *database* ataupun pada tabel maupun *field* yang ada. Melalui perintah ini, seorang admin *database* bisa menjaga kerahasiaan sebuah *database*. Terutama untuk data yang penting. DCL berdasarkan perintah dasarnya terbagi dalam dua perintah utama yakni:

- Perintah *Grant*: Perintah ini biasanya digunakan ketika admin *database* ingin memberikan hak akses ke *user* lainnya. Tentu pemberian hak akses ini dapat dibatasi atau diatur. Dalam hal ini admin pun dapat memberikan akses mengenai perintah dalam DML di atas.
- Perintah *Revoke*: Kebalikannya dari *Grant*, *Revoke* terkadang sering digunakan untuk mencabut maupun menghapus hak akses seorang pengguna yang awalnya diberikan akses oleh admin *database* melalui perintah *Grant* sebelumnya.

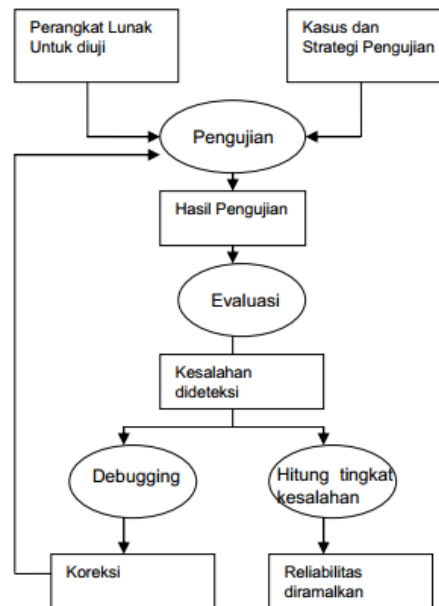
d. Strategi dan Teknis Pengujian Program

1. Pengertian dan Tahapan Pengujian Program

Pengujian sistem adalah pengujian program perangkat lunak yang lengkap dan terintegrasi yang harus mengikuti suatu pola dan rencana yang harus ditetapkan sebelumnya. Pengujian sangat dianjurkan dilakukan oleh pihak yang independen dengan pembuat perangkat lunak. Perangkat lunak yang dihasilkan perlu dilakukan *test-case* dan *test-strategy*. *Test-case* adalah prosedur untuk memeriksa perangkat lunak dan memberikan hasil yang menentukan penerimaan, pemodifikasian, atau bahkan penilaian terhadap perangkat lunak tersebut. Kualitas dan efisiensi pengujian sangat tergantung pada pengembangan dan aplikasi *test-case* tersebut.

Daftar area di mana *test-case* biasanya dilakukan yaitu:

- *Field*, pengujian dilakukan pada atribut dari setiap *field* yang ada.
- *Record*, pengujian dilakukan pada tahap pemasukan, penyimpanan, pemanggilan, penghapusan, dan pemrosesan *record*.
- *File*, pengujian dilakukan pada pembukaan, pemanggilan, penggunaan, dan penutupan *file*.
- Kendali, pengujian dilakukan pada semua kendali program untuk memastikan keakuratan, kelengkapan, dan kewenangan pemrosesan transaksi.
- Entri Data, pengujian dilakukan untuk mengetahui adanya ketidakakuratan, ketidaklengkapan, atau kekinian data. Selain itu juga dilakukan validasi pada fungsi-fungsi yang diperlukan dalam memasukkan data yang benar sesuai atau menurut spesifikasi rancangan.
- Alur Program: pengujian untuk memastikan bahwa program yang diuji tersebut menjalankan konsepsi rangkaian, seleksi, dan repetisi (pengulangan) secara benar.



Gambar 5.12. Langkah Pengujian Program

Sebelum melakukan pengujian program, perlu dilakukan terlebih dahulu hal-hal berikut agar pengujian dapat berjalan lancar dan mendapatkan hasil sesuai yang diharapkan yaitu:

a) Penyusunan Definisi *Rule* Validasi Program

Kegiatan membuat rangkaian *rule* validasi yang diterapkan pada sistem untuk menjaga kevalidan data yang direkam pada sistem informasi. *Rule* validasi merupakan batasan atau aturan terhadap data yang direkam ke dalam sistem agar keluaran data dari sistem sesuai dengan aturan atau batasan-batasan dari kebutuhan di sisi bisnis.

Rule validasi dapat berupa pengecekan mengenai rentang nilai suatu data, tipe data, batasan karakter yang boleh direkam ke dalam sistem atau aturan hubungan antar *fields* dari suatu data. Penulisan *rule* validasi yang diterapkan pada sistem informasi dapat berupa penulisan ekspresi logika yang melibatkan suatu *fields* atau beberapa *fields* data pada kode program, fungsi atau tabel di *database* atau standar *template* yang sudah disediakan pada sistem aplikasi.

Tahapan kegiatan ini mencakup pada:

- Membuat aturan struktur data seperti rentang nilai, tipe data, batasan karakter yang diperbolehkan untuk disimpan;
- Membuat aturan pada hubungan antar *fields* dari data yang direkam ke dalam sistem informasi;
- Membuat *rule validation* yang diimplementasikan pada kode program;
- Membuat *rule validation* yang diimplementasikan pada fungsi atau tabel di *database*;
- Membuat *rule validation* yang diimplementasikan pada standar *template* dari aplikasi yang digunakan, seperti file excel, csv, txt;
- Membuat *rule validation* yang diimplementasikan pada modul atau menu khusus di aplikasi yang digunakan.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Field	Description	Data Type	Type	Max Len	Max Len	Allowed Char	Blank	Not Blank	Range	Key Down	Tab Index	Page	Version
2			Char	master, alpha, numeric, c, datetime, Tinyint, Text, Float, comma				blank, notblank		prevtab, nexttab, tab, none				
3	b1_r108	R Nama satuan lingkungan setempat	Varchar	alphanumeric	250	250	'\./\,- _'	notblank		tab		7	1	bs
4	b2_r201_k2	R Nama Petugas (pencacah)	Varchar	alphanumeric	150	150	'\./\,- _'	notblank		tab		8	1	bs
5	b2_r201_k3	R Nama petugas (pemeriksa)	Varchar	alphanumeric	150	150	'\./\,- _'	notblank		tab		9	1	bs
6	b2_r202_k2tg1	R Tanggal Pencacahan Awal	Varchar	Numeric	2	2		notblank	1-31	tab		10	1	bs
7	b2_r202_k2b1n1	R Bulan Pencacahan Awal	Varchar	Numeric	2	2		notblank	1-12	tab		11	1	bs
8	b2_r202_k2tg2	R Tanggal Pencacahan Akhir	Varchar	Numeric	2	2		notblank	1-31	tab		12	1	bs
9	b2_r202_k2b1n2	R Bulan Pencacahan Akhir	Varchar	Numeric	2	2		notblank	1-12	tab		13	1	bs
10	b2_r202_k3tg1	R Tanggal Pemeriksaan Awal	Varchar	Numeric	2	2		notblank	1-31	tab		14	1	bs
11	b2_r202_k3b1n1	R Bulan Pemeriksaan Awal	Varchar	Numeric	2	2		notblank	1-12	tab		15	1	bs
12	b2_r202_k3tg2	R Tanggal Pemeriksaan Akhir	Varchar	Numeric	2	2		notblank	1-31	tab		16	1	bs
13	b2_r202_k3b1n2	R Bulan Pemeriksaan Akhir	Varchar	Numeric	2	2		notblank	1-12	tab		17	1	bs
14	bs_r301	R jumlah rumah tangga hasil	int	Numeric	3	3		blank	1-999	tab		18	1	bs
15	bs_r302	R jumlah rumah tangga pemutakhiran	int	Numeric	3	3		blank	1-999	tab		19	1	bs
16	bs_r303	R jumlah rumah tangga budidaya	int	Numeric	3	3		blank	1-999	tab		20	1	bs
17	bs_r304	R jumlah rumah tangga budidaya	int	Numeric	3	3		blank	1-999	tab		21	1	bs
18	bs_r305	R jumlah rumah tangga budidaya	int	Numeric	3	3		blank	1-999	tab		22	1	bs
19	bs_r306	R jumlah rumah tangga budidaya kopi	int	Numeric	3	3		blank	1-999	tab		23	1	bs
metadata														
1	Field	Order	Page	relFields	rule	Message	Perlakuan	Type	Version					
2	b2_r202_k2b1n1	1	1	b2_r202_k2tg1,b2_r202_k2b1n1	(b2_r202_k2tg1 > 28 AND b2_r202_k2b1n1 = 2) OR (b2_r202_k2tg1 > 30 AND (b2_r202_k2b1n1 = 4 OR b2_r202_k2b1n1 = 6 OR b2_r202_k2b1n1 = 9 OR b2_r202_k2b1n1 = 11)) OR (b2_r202_k2tg1 > 31 AND (b2_r202_k2b1n1 = 1 OR b2_r202_k2b1n1 = 3 OR b2_r202_k2b1n1 = 5 OR b2_r202_k2b1n1 = 7 OR b2_r202_k2b1n1 = 8 OR b2_r202_k2b1n1 = 10 OR (b2_r202_k3tg1 > 28 AND b2_r202_k3b1n1 = 2) OR (b2_r202_k3tg1 > 30 AND (b2_r202_k3b1n1 = 4 OR b2_r202_k3b1n1 = 6 OR b2_r202_k3b1n1 = 9 OR b2_r202_k3b1n1 = 11)) OR (b2_r202_k3tg1 > 31 AND (b2_r202_k3b1n1 = 1 OR b2_r202_k3b1n1 = 3 OR b2_r202_k3b1n1 = 5 OR b2_r202_k3b1n1 = 7 OR b2_r202_k3b1n1 = 8 OR b2_r202_k3b1n1 = 10 OR (b2_r202_k2b1n1 = b2_r202_k3b1n1 AND (b5_k6 < 5 OR b5_k6 = 6 OR b5_k6 = 7) AND (b5_k7 = b5_k6,b5_k7,b5_k8; b5_k9,b5_k10,b5_k11 1;b5_k12,b5_k13;b5_k14,b5_k15	b2_r202_k2tg1 tidak sesuai dengan b2_r202_k2b1n1	Manual cek	bs						
3	b2_r202_k3tg1	1	1	b2_r202_k2tg1,b2_r202_k2b1n1,b2_r202_k3tg1,b2_r202_k3b1n1	(b2_r202_k2b1n1 = b2_r202_k3b1n1 AND (b5_k6 < 5 OR b5_k6 = 6 OR b5_k6 = 7) AND (b5_k7 = b5_k6,b5_k7,b5_k8; b5_k9,b5_k10,b5_k11 1;b5_k12,b5_k13;b5_k14,b5_k15	b2_r202_k3tg1 tidak sesuai dengan b2_r202_k3b1n1	Manual cek	bs						
4	b2_r202_k2tg1	1	1	b2_r202_k2tg1,b2_r202_k2b1n1,b2_r202_k2b1n1,b2_r202_k2b1n1	(b5_k6 < 5 OR b5_k6 = 6 OR b5_k6 = 7) AND (b5_k7 = b5_k6,b5_k7,b5_k8; b5_k9,b5_k10,b5_k11 1;b5_k12,b5_k13;b5_k14,b5_k15	b2_r202_k2tg1 terisi lebih dari b5_k2 berkode 1 sampai 4 tetapi b5_k6 berkode 4	Manual cek	bs						
5	b5_k2	1	2	b5_k1,b5_k2,b5_k6	b5_k2 < b5_k1 AND b5_k6 = 4	b5_k2 berkode 1 sampai 4 tetapi b5_k6 berkode 4	Manual cek	rt						
6	b5_k6	1	2	b5_k6,b5_k7	(b5_k6 = '1' OR b5_k6 = '2' OR b5_k6 = '3' OR b5_k6 = '4') AND b5_k7 = ''	b5_k6 berkode 1 sampai 4 tetapi b5_k6 berkode 4	Manual cek	rt						
7	b5_k6	2	2	b5_k6,is_preprinted	(b5_k6 = '1' OR b5_k6 = '2' OR b5_k6 = '3' OR b5_k6 = '4') AND (b5_k7 = b5_k6,b5_k7,b5_k8; b5_k9,b5_k10,b5_k11 1;b5_k12,b5_k13;b5_k14,b5_k15	b5_k6 berkode 5 sampai 7 tetapi b5_k7 sampai b5_k6 berkode 2	Manual cek	rt						
8	b5_k6	3	2	b5_k6,b5_k4,nama	b5_k6 = '2' AND b5_k4 = nama_krt	b5_k6 berkode 2	Manual cek	rt						

Gambar 5.13. Contoh Penyusunan Definisi *Rule Validation* Program

b) Penyiapan Data Untuk Uji Coba Program

Kegiatan merancang dan menyiapkan masukan sistem untuk uji coba, merupakan salah satu tahapan dari rangkaian proses pengujian sistem. Masukan sistem yang sudah disiapkan tersebut digunakan untuk menguji coba beberapa kondisi atau skenario sistem untuk menentukan apakah keluaran dari sistem sudah sesuai dengan kebutuhan, dapat diandalkan sesuai tujuan, atau masih terdapat kesalahan. Data yang digunakan untuk uji coba dapat dalam bentuk data simulasi maupun data nyata. Penyiapan data dapat dilakukan dengan cara manual maupun dengan bantuan *tools*.

Tahapan kegiatan ini mencakup pada:

- Menyiapkan data simulasi untuk digunakan pengujian saat pembangunan sistem (*development testing*) secara manual;
- Menyiapkan data nyata, seperti data dari sistem lama atau data dari pengguna akhir (pihak bisnis), untuk digunakan pengujian tahap akhir sebelum sistem mulai digunakan atau dioperasikan oleh pengguna (*acceptance testing*); dan
- Menyiapkan sekumpulan data simulasi dengan menggunakan *tools* bantuan seperti *Test Data Generator*, untuk menguji kehandalan sistem.

c) Penyusunan Skenario Uji Coba Program

Kegiatan perancangan berbagai skenario *test* yang akan dilakukan untuk menguji fungsi pada sistem informasi.

Tahapan kegiatan ini mencakup pada:

- Analisis *Requirement*, pada fase ini dilakukan kegiatan pengumpulan data dengan metode wawancara/konsultasi kepada beberapa narasumber terkait program aplikasi, dilanjutkan dengan

menganalisis hasil dari wawancara, mendefinisikan pengguna program aplikasi yang akan diuji, merangkum daftar kebutuhan pengguna, dan diakhiri dengan membuat *Requirement Traceability Matrix*.

- *Test Planning*, pada fase ini dilakukan perancangan metode pengujian yang paling tepat dan perencanaan upaya pengujian mulai dari menentukan waktu serta menentukan bagian sistem yang akan diuji.
- *Test Case Development*, pada fase ini dilakukan perancangan berbagai skenario *test* yang akan dilakukan untuk menguji fungsi pada sistem sesuai dengan RTM dan *planning* yang sudah dibuat. Langkah awal yang diperlukan yaitu membuat tabel deskripsi awal yang berisikan *test id* dan deskripsi *test* sebagai acuan yang nantinya akan didetailkan lagi berupa sebuah tabel skenario.

Requirement Traceability	Business Requirement						
	BR1	BR2	BR3	BR4	BR5	BR6	BR7
	Login	Dashboard	Master	Entri Data	Laporan	Revalidasi Data	Export Data
Test Case	TC1						
	TC2						
	TC3						
	TC4						
	TC5						
	TC6						

Gambar 5.14. Contoh *Requirement Traceability Matrix*

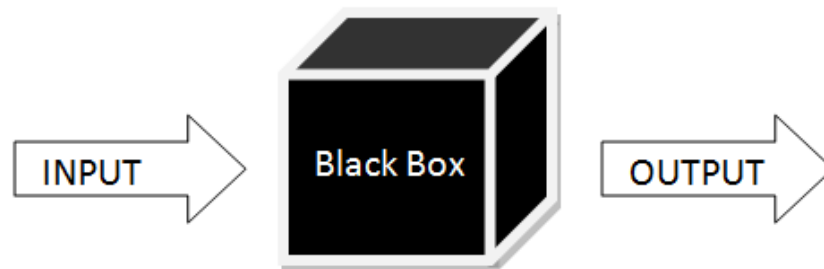
Test Case ID	Test Case Description
TC1	Login menggunakan kombinasi username dan password salah atau sebaliknya
TC2	Login menggunakan username yang tidak terdaftar
TC3	Login menggunakan password dengan menggunakan spasi
TC4	Input data melebihi kapasitas/length
TC5	Input data tidak pada tempatnya dengan menggunakan special character/symbol/number
TC6	Input blank data pada field
TC7	View function berdasarkan hak akses
TC8	View data inputan

Gambar 5.15. Contoh *Test Case*

2. Jenis Pengujian Program

Pengujian perangkat lunak secara umum dapat dibedakan menjadi dua yaitu *Black Box Testing* dan *White Box Testing*.

a) *Black Box Testing*

Gambar 5.16. *Black Box Testing*

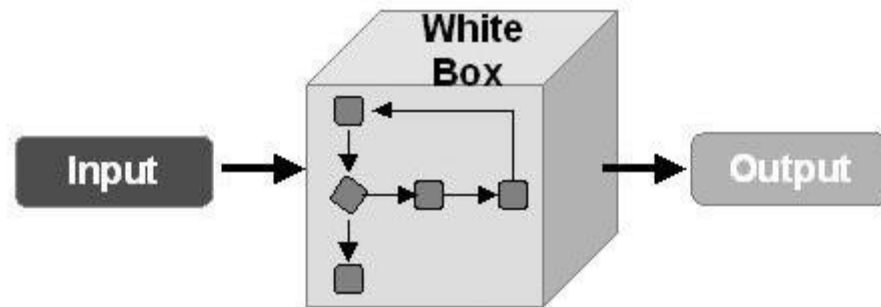
Black Box Testing atau yang sering dikenal dengan sebutan pengujian fungsional merupakan metode pengujian perangkat lunak yang digunakan untuk menguji perangkat lunak tanpa mengetahui struktur internal kode atau program. Dalam pengujian ini, *tester* menyadari apa yang harus dilakukan oleh program tetapi tidak memiliki pengetahuan tentang bagaimana melakukannya.

Kelebihan *Black Box Testing* yaitu:

- ✓ Efisien untuk segmen kode besar
- ✓ Akses kode tidak diperlukan
- ✓ Pemisahan antara perspektif pengguna dan pengembang

Kelemahan *Black Box Testing* yaitu:

- ✗ Cakupan terbatas karena hanya sebagian kecil dari skenario pengujian yang dilakukan
- ✗ Pengujian tidak efisien karena keberuntungan *tester* dari pengetahuan tentang perangkat lunak internal

b) *White Box Testing*Gambar 5.17. *White Box Testing*

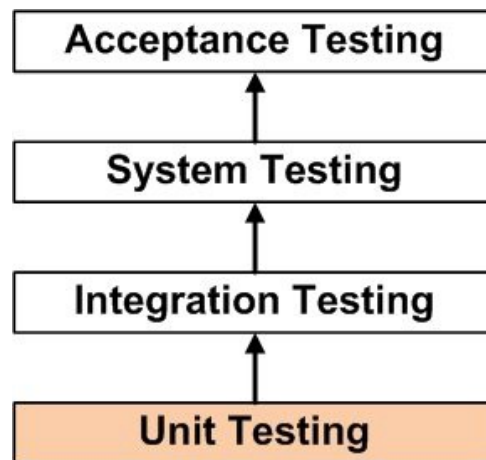
White Box Testing merupakan metode pengujian perangkat lunak di mana struktur internal diketahui untuk menguji siapa yang akan menguji perangkat lunak. Pengujian ini membutuhkan pengetahuan internal tentang kemampuan sistem dan pemrograman.

Kelebihan *White Box Testing* yaitu:

- ✓ Efisien dalam menemukan kesalahan dan masalah
- ✓ Diperlukan pengetahuan tentang internal perangkat lunak yang sedang diuji bermanfaat untuk pengujian menyeluruh
- ✓ Memungkinkan menemukan kesalahan tersembunyi
- ✓ Membantu mengoptimalkan kode

Kelemahan *White Box Testing* yaitu:

- ✗ Membutuhkan pengetahuan tingkat tinggi dari perangkat lunak internal yang sedang diuji
- ✗ Membutuhkan akses kode



Gambar 5.18. Kategori Pengujian berdasarkan Kronologis

Pengujian perangkat lunak memiliki urutan-urutan mengenai beberapa hal yang perlu dilakukan. Berikut adalah kategori pengujian perangkat lunak yang disusun secara kronologis seperti terlihat pada Gambar 5.18:

- *Unit Testing*: Pengujian dilakukan pada setiap modul atau blok kode selama pengembangan. Pengujian ini biasanya dilakukan oleh *developer* yang menulis kode.
- *Integration Testing*: Pengujian yang dilakukan sebelum, selama, dan setelah integrasi modul baru ke dalam paket perangkat lunak utama. Pengujian ini melibatkan pengujian setiap modul kode dari masing-masing individu. Satu perangkat lunak dapat berisi beberapa modul yang sering dibuat oleh beberapa *developer* yang berbeda.
- *System Testing*: Pengujian yang dilakukan oleh agen pengujian profesional pada produk perangkat lunak yang telah selesai sebelum perangkat lunak tersebut diperkenalkan secara umum.
- *Acceptance Testing*: Pengujian beta dari produk yang dilakukan oleh pengguna akhir yang sebenarnya.

Selain jenis pengujian di atas, terdapat beberapa pengujian sistem yang biasanya digunakan oleh perusahaan pengembang *software* atau perangkat lunak besar. Beberapa jenis tersebut diantaranya yaitu:

a) *Performance Testing*

Performance test adalah *integration* dan *usability test* yang menentukan apakah sistem atau subsistem dapat memenuhi kriteria kinerja berbasis waktu seperti *response time* atau *throughput*. *Response time* menentukan batas waktu maksimum yang diizinkan dari respons *software* untuk *query* dan *update*. *Throughput* menentukan jumlah minimum *query* dan transaksi yang harus diproses per menit atau per jam.

b) *System Testing*

System test adalah *integration test* dari *behavior* seluruh sistem atau *independent subsystem*. *System testing* biasanya dilakukan pertama kali oleh pengembang atau personil pengujian untuk memastikan bahwa keseluruhan sistem berfungsi dan bahwa sistem telah memenuhi persyaratan pengguna (*user requirement*). *System testing* biasanya dilakukan di akhir setiap iterasi untuk mengidentifikasi isu – isu penting, seperti masalah *performance* yang perlu ditangani pada iterasi berikutnya. Biasanya *test* ini harus dilakukan sesering mungkin.

c) *Unit Testing*

Unit testing adalah proses metode pengujian individual, *class*, atau komponen sebelum mereka terintegrasi dengan perangkat lunak lainnya. Tujuan dari *unit testing* adalah untuk mengidentifikasi dan memperbaiki kesalahan sebanyak mungkin sebelum modul – modul digabungkan menjadi unit perangkat lunak yang lebih besar, seperti program, *class* dan subsistem. Kesalahan menjadi lebih sulit dan mahal untuk ditemukan dan diperbaiki ketika banyak unit telah digabungkan. *Unit testing* memerlukan implementasi dari *driver* dan/atau *stub*. *Stub* adalah *dummy class* atau *method* yang dapat dipanggil namun biasanya tidak melakukan apapun kecuali mengembalikan tipe yang diperlukan. Modul *driver* adalah program yang menjalankan *method* atau fungsi dari *class* yang akan diuji. Berikut adalah langkah yang harus dilakukan:

- ✓ Menentukan nilai dari parameter input

- ✓ Memanggil unit yang dites, melewatkannya dengan parameter input
- ✓ Menerima parameter kembalian dari unit yang diuji dan mencetaknya, menampilkannya, atau menguji hasilnya terhadap hasil yang diharapkan.

d) *Integration Testing*

Integration test adalah mengevaluasi *behavior* dari kelompok *method* atau *class*. Tujuan dari *integration test* adalah untuk mengidentifikasi kesalahan yang tidak dapat dideteksi oleh *unit testing*. Kesalahan tersebut mungkin disebabkan oleh beberapa masalah, diantaranya:

- ✓ *Interface incompatibility*, misalnya sebuah *method* melewati parameter dengan tipe data yang salah ke *method* lainnya
- ✓ *Parameter values*, misalnya sebuah *method* mengembalikan nilai yang tidak terduga seperti nomor negatif untuk harga
- ✓ *Run-time exceptions*, misalnya *method* menyebabkan kesalahan seperti *out of memory* atau *file already in use* karena ada konflik kebutuhan sumber daya
- ✓ *Unexpected state interactions*, misalnya *state* dari dua atau lebih objek yang berinteraksi menyebabkan kesalahan yang kompleks seperti ketika *method* pada *class Order* menjalankan satu kesalahan dari semua kemungkinan *state* pada objek *Customer*.

Beberapa masalah di atas merupakan kesalahan paling umum yang sering ditemui dalam *integration testing*, tetapi sebenarnya masih banyak masalah lainnya yang dapat menjadi penyebab kesalahan (*error*).

e) *Usability Testing*

Usability test adalah *test* untuk menentukan apakah *method*, *class*, subsistem, atau sistem telah memenuhi persyaratan pengguna. Oleh karena banyaknya tipe persyaratan sistem baik yang fungsional maupun

non-fungsional, maka banyak tipe dari *usability test* yang harus dilakukan di waktu yang berbeda. Umumnya *usability test* mengevaluasi persyaratan fungsional dan kualitas dari *user interface*. *User* berinteraksi dengan sistem untuk menentukan apakah fungsi telah seperti yang diharapkan dan apakah *user interface* membuat sistem dapat mudah digunakan. Pengujian ini sering dilakukan untuk mendapatkan *feedback* yang cepat dalam meningkatkan *interface* dan mengoreksi kesalahan dalam komponen perangkat lunak.

f) *Smoke Testing*

Smoke testing adalah *system test* yang biasanya dilakukan setiap hari atau beberapa kali per minggu. *Build and smoke test* sangat penting karena menyediakan *feedback* yang cepat dalam masalah yang signifikan.

g) *Stress Testing*

Stress Testing adalah pengujian yang biasanya dilakukan dalam membuat sebuah *website*, dimana *stress testing* dilakukan untuk mengetahui sekuat apa *server website* kita menampung *visitor* dalam *website* tersebut, dengan cara melakukan *hit dummy* ke *website* menggunakan *tools*.

h) *User Acceptance Test (UAT)*

User acceptance test digunakan untuk menentukan apakah sistem yang dikembangkan telah memenuhi kebutuhan pengguna. Dalam beberapa proyek, *acceptance testing* dilakukan pada putaran terakhir proses pengujian yaitu sebelum sistem diserahkan kepada user. *Acceptance testing* biasanya dilakukan setelah rangkaian *testing* seperti *Unit Testing*, *Integration Testing*, dan *System Testing* selesai dan menggunakan metode *Black Box Testing*, dengan menggunakan dokumen *test case* untuk dipresentasikan di akhir ke *user* atau *client*.

Dari berbagai jenis pengujian di atas, dapat dihasilkan dokumen hasil uji coba dari setiap skenario *Test Case* yang sudah dibuat yang dapat

dilengkapi dengan keterangan mencakup (1) Aksi; (2) Input; (3) Hasil yang diharapkan; dan (4) Hasil Akhir.

Test Skenario 5				
Test Case 1				
No	Action	Input	Expected Outcome	Result
1	Login menggunakan kombinasi username dan password salah atau sebaliknya	User : adam Pass : demo	User / password salah	Pass
			Actual Outcome	
2		User : demo Pass : 1234	User / password benar	Pass
			Actual Outcome	

Gambar 5.19. Contoh Dokumen Hasil Uji Coba (*Test Execution*)

3. Pemeriksaan dan Analisis Hasil Pengujian Program

Kegiatan pemeriksaan dan analisis dilakukan dengan mendiskusikan hasil dari siklus pengujian dan dianalisis bagaimana cara untuk meningkatkan strategi tes yang digunakan, menghilangkan kemacetan proses untuk siklus pengujian, dan berbagi cara/metode terbaik untuk program aplikasi serupa di masa depan.

Dokumen hasil pemeriksaan dan analisis hasil uji coba dilengkapi dengan keterangan *Requirement Traceability Matrix* setelah pengujian yang berisi informasi hubungan antara *Test Case* dengan *Business Requirement*.

Requirement Traceability		Business Requirement						
		BR1	BR2	BR3	BR4	BR5	BR6	BR7
		Login	Dashboard	Master	Entri Data	Laporan	Revalidasi Data	Export Data
Test Case	TC1	✓						
	TC2	✓						
	TC3	✓						
	TC4		✓					
	TC5			✓				
	TC6				✓			
	TC7					✓		
	TC8						✓	
	TC9							✓
	TC10							✓

Gambar 5.20. Contoh *Requirement Traceability Matrix* setelah Pengujian

5.2. Rangkuman

Algoritma pemrograman adalah langkah-langkah yang ditulis secara berurutan untuk menyelesaikan masalah pemrograman komputer dan merupakan langkah pertama yang harus ditulis sebelum menuliskan program. Algoritma pemrograman dapat dituliskan dalam bentuk *flowchart* agar lebih mudah dipahami.

Paradigma pemrograman penting bagi seorang *programmer* untuk dapat mengidentifikasi sebuah masalah sebelum mempersiapkan solusinya dengan sebuah program komputer. Pemrograman Berorientasi Objek (PBO) merupakan paradigma pemrograman yang populer digunakan saat ini dengan konsep kelas dan objek. Prinsip Dasar PBO yaitu abstraksi, enkapsulasi, polimorfisme, dan *inheritance*.

Dalam membuat program dibutuhkan juga pemahaman terkait pemrograman *database* dengan menggunakan *Structured Query Language* (SQL) yaitu bahasa *query* yang dirancang untuk membantu dalam pengambilan dan pengelolaan informasi pada sebuah *database*.

Perangkat lunak yang dihasilkan perlu dilakukan *test-case* yaitu prosedur untuk memeriksa perangkat lunak dan memberikan hasil yang

menentukan penerimaan, pemodifikasian, atau bahkan penilaian terhadap perangkat lunak tersebut. Hasil uji coba perlu diperiksa dan dianalisis untuk meningkatkan strategi tes yang digunakan, menghilangkan kemacetan proses untuk siklus pengujian, dan berbagi cara/metode terbaik untuk program aplikasi serupa di masa depan. Dokumen hasil pemeriksaan dan analisis ini berupa *Requirement Traceability Matrix* setelah pengujian yang berisi informasi hubungan antara *Test Case* dengan *Business Requirement*.

5.3. Soal Latihan

BPS RI akan membuat sebuah sistem manajemen mitra. Mitra yang akan ditugaskan untuk survei harus didaftarkan terlebih dahulu pada sistem tersebut. Buatlah *flowchart* untuk proses registrasi mitra.

5.4. Contoh Kasus

Direktorat Z baru saja membuat suatu sistem *Agile IT Project Management System* yang merupakan sistem untuk mengoptimalkan pengelolaan proyek TI. Sistem ini memiliki empat modul yaitu modul pengelolaan proyek (untuk mengelola proyek, tim, task, milestone, kanban board), modul dashboard (untuk menampilkan progress proyek), modul notifikasi (untuk menginformasikan progress proyek), dan modul panduan interaktif (untuk panduan penggunaan sistem). Berdasarkan penjelasan sistem tersebut, Buatlah dokumen untuk kebutuhan pengujian mulai dari penyusunan skenario uji coba sampai dengan pemeriksaan dan analisis hasil uji coba meliputi dokumen *Requirement Traceability Matrix*, *Test Case*, *Test Execution*, dan *Requirement Traceability Matrix* setelah Pengujian. Anda dapat menambahkan asumsi jika diperlukan.

BAB VI PEMANTAUAN KINERJA SISTEM INFORMASI

6.1. Uraian Materi

Monitoring (pemantauan) merupakan sebuah proses penaksiran atau penilaian kualitas kinerja sistem dari waktu ke waktu. Pemantauan ini dilakukan secara berkelanjutan sejalan dengan kegiatan usaha yang mencakup kegiatan sehari-hari. Pemantauan kinerja sistem informasi erat kaitannya dengan pemeliharaan perangkat lunak. Pemeliharaan perangkat lunak adalah proses umum mengubah sistem setelah dikirimkan. Perubahan yang dilakukan pada perangkat lunak dapat berupa perubahan sederhana untuk memperbaiki kesalahan pengkodean, perubahan yang lebih luas untuk memperbaiki kesalahan desain, atau peningkatan yang signifikan untuk memperbaiki kesalahan spesifikasi atau mengakomodasi persyaratan baru. Perubahan diimplementasikan dengan memodifikasi komponen sistem yang ada dan, jika perlu, dengan menambahkan komponen baru ke sistem.

a. Tujuan Pemantauan

Tujuan utama pemantauan adalah memastikan sistem melakukan suatu proses sesuai prosedur yang berlaku sehingga proses berjalan sesuai jalur yang disediakan (*on the track*). Dengan demikian dapat diidentifikasi hasil yang tidak diinginkan pada suatu proses dengan cepat tanpa menunggu proses selesai untuk selanjutnya dilakukan pemeliharaan terhadap perangkat lunak yang digunakan jika diperlukan.

Ada tiga motif yang mendorong perlu dilakukannya pemeliharaan perangkat lunak:

1. Perbaikan Kesalahan

Kesalahan pengkodean biasanya relatif murah untuk diperbaiki dibandingkan kesalahan desain yang lebih mahal karena mungkin melibatkan penulisan ulang beberapa komponen program. Kesalahan

persyaratan adalah yang paling mahal untuk diperbaiki karena desain ulang sistem yang ekstensif yang mungkin diperlukan.

2. Adaptasi Lingkungan

Jenis pemeliharaan ini diperlukan ketika beberapa aspek lingkungan sistem seperti perangkat keras, sistem operasi platform, atau perangkat lunak pendukung lainnya berubah. Sistem aplikasi harus dimodifikasi untuk menyesuaikannya dengan perubahan lingkungan ini.

3. Penambahan Fungsionalitas

Jenis pemeliharaan ini diperlukan ketika persyaratan sistem berubah sebagai respons terhadap perubahan organisasi atau bisnis. Skala perubahan yang diperlukan untuk perangkat lunak sering kali jauh lebih besar daripada jenis pemeliharaan lainnya.

b. Hasil Pemantauan Kinerja Sistem

Hasil yang diperoleh dari pemantauan kinerja sistem yaitu kondisi terkini sistem yang digunakan, apakah masih relevan dan berjalan lancar dalam menangani proses bisnis berjalan ataukah ditemukan permasalahan yang mengganggu proses atau terdapat perubahan/penambahan *requirement* baru yang diperlukan oleh bisnis yang belum diakomodir oleh sistem saat ini. Hasil dari pemantauan ini perlu dikumpulkan dan dievaluasi sebagai bahan masukan untuk menentukan pemeliharaan seperti apa yang diperlukan terhadap sistem berjalan agar proses bisnis dapat kembali lancar.

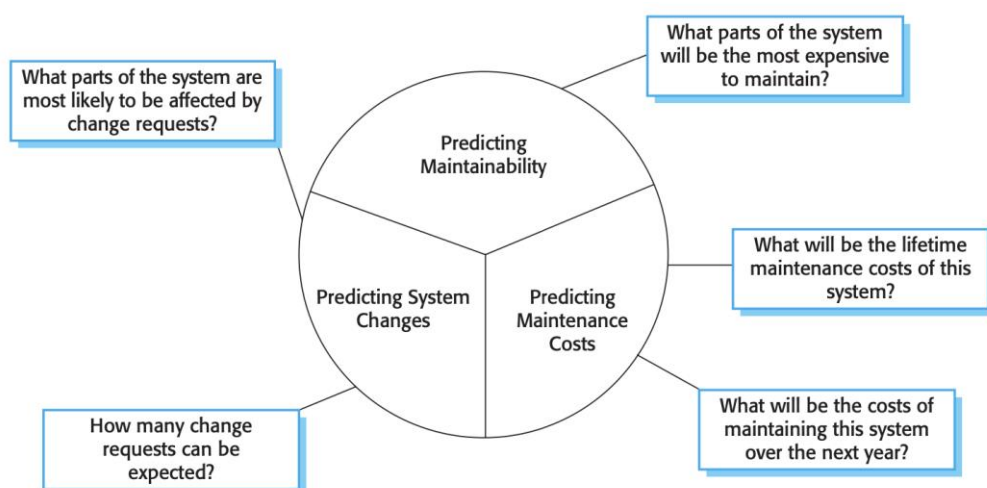
Selama kegiatan pemantauan, jika ditemukan adanya permasalahan yang terjadi, maka perlu segera dicari solusi yang dapat menyelesaikan permasalahan tersebut baik itu solusi sementara maupun solusi permanen. Hal ini diperlukan agar permasalahan tersebut dapat segera teratasi sehingga tidak sampai mengganggu proses bisnis secara keseluruhan.

Tabel 6.1. Contoh Hasil Pemantauan Kinerja Sistem

Tujuan	Memantau performa dan workload aplikasi selama pengelolaan proyek
Nama aplikasi	Agile Development Project Management System
Deskripsi aplikasi	Sistem untuk mengoptimalkan pengelolaan proyek pada Fungsi IPD yang mempunyai empat modul yaitu modul pengelolaan proyek (tim, task, milestone, kanban board), modul dashboard, modul notifikasi, dan modul panduan interaktif.
Hasil pantauan kinerja aplikasi	Secara umum aplikasi berjalan lancar meskipun digunakan secara bersama-sama oleh seluruh personel namun untuk notifikasi email status perubahan task terkadang terlambat dikirim.
Permasalahan dan solusi selama pemantauan	Notifikasi terkadang terlambat dikirim dikarenakan keterbatasan jumlah pengiriman email hosting dalam satu hari. Solusinya menggunakan SMTP Premium untuk pengiriman email.
Periode pemantauan	Minggu IV Januari 2021

c. Prediksi Pemeliharaan

Pemimpin organisasi tidak menyukai kejutan, terutama jika kejutan tersebut mengakibatkan biaya tinggi yang tidak terduga. Oleh karena itu, kita harus mencoba memprediksi perubahan sistem apa yang mungkin diusulkan dan bagian sistem mana yang paling sulit untuk dirawat. Kita juga harus mencoba memperkirakan biaya perawatan keseluruhan untuk suatu sistem dalam jangka waktu tertentu. Gambar 6.1 menunjukkan prediksi ini dan pertanyaan terkait.



Gambar 6.1. Prediksi Pemeliharaan

Memprediksi jumlah permintaan perubahan untuk suatu sistem memerlukan pemahaman tentang hubungan antara sistem dan lingkungan eksternalnya. Beberapa sistem memiliki hubungan yang sangat kompleks dengan lingkungan eksternalnya dan perubahan pada lingkungan itu pasti menghasilkan perubahan pada sistem. Untuk mengevaluasi hubungan antara sistem dan lingkungannya, hal berikut harus dinilai:

1. Jumlah dan kompleksitas antarmuka sistem. Semakin besar jumlah antarmuka dan semakin kompleks antarmuka ini, semakin besar kemungkinan perubahan antarmuka akan diperlukan saat persyaratan baru diusulkan.
2. Jumlah kebutuhan sistem yang secara inheren mudah berubah. Persyaratan yang mencerminkan kebijakan dan prosedur organisasi cenderung lebih mudah berubah daripada persyaratan yang didasarkan pada karakteristik domain yang stabil.
3. Proses bisnis di mana sistem digunakan. Seiring berkembangnya proses bisnis, mereka menghasilkan permintaan perubahan sistem. Semakin banyak proses bisnis yang menggunakan suatu sistem, semakin banyak tuntutan untuk perubahan sistem.

Setelah sistem dimasukkan ke dalam layanan, kita mungkin dapat menggunakan data proses untuk membantu memprediksi kemampuan perawatan. Contoh metrik proses yang dapat digunakan untuk menilai perawatan adalah sebagai berikut:

1. Jumlah permintaan untuk pemeliharaan korektif. Peningkatan jumlah laporan *bug* dan kegagalan dapat mengindikasikan bahwa lebih banyak kesalahan yang dimasukkan ke dalam program daripada yang diperbaiki selama proses pemeliharaan. Ini mungkin menunjukkan penurunan dalam pemeliharaan.
2. Rata-rata waktu yang dibutuhkan untuk analisis dampak. Ini mencerminkan jumlah komponen program yang terpengaruh oleh

- permintaan perubahan. Jika waktu ini meningkat, itu berarti semakin banyak komponen yang terpengaruh dan perawatan menurun.
3. Rata-rata waktu yang dibutuhkan untuk mengimplementasikan permintaan perubahan. Ini tidak sama dengan waktu untuk analisis dampak meskipun mungkin berkorelasi dengannya. Ini adalah jumlah waktu yang kita perlukan untuk memodifikasi sistem dan dokumentasinya, setelah kita menilai komponen mana yang terpengaruh. peningkatan waktu yang dibutuhkan untuk mengimplementasikan perubahan dapat mengindikasikan penurunan kemampuan pemeliharaan.
 4. Jumlah permintaan perubahan yang belum diselesaikan. Peningkatan jumlah ini dari waktu ke waktu dapat menyiratkan penurunan kemampuan pemeliharaan.

6.2. Rangkuman

Monitoring (pemantauan) merupakan sebuah proses penaksiran atau penilaian kualitas kinerja sistem dari waktu ke waktu. Pemantauan kinerja sistem informasi erat kaitannya dengan pemeliharaan perangkat lunak yang merupakan proses umum mengubah sistem setelah dikirimkan.

Tujuan utama pemantauan adalah memastikan sistem melakukan suatu proses sesuai prosedur yang berlaku sehingga proses berjalan sesuai jalur yang disediakan (*on the track*). Ada tiga motif yang mendorong perlu dilakukannya pemeliharaan perangkat lunak yaitu perbaikan kesalahan, adaptasi lingkungan, dan penambahan fungsionalitas.

Hasil dari pemantauan kinerja sistem yaitu kondisi terkini sistem yang digunakan, apakah masih relevan ataukah ditemukan permasalahan atau terdapat perubahan/penambahan *requirement* baru yang diperlukan oleh bisnis yang belum diakomodir oleh sistem. Hasil dari pemantauan ini perlu dikumpulkan dan dievaluasi sebagai bahan masukan untuk menentukan

pemeliharaan seperti apa yang diperlukan terhadap sistem agar proses bisnis dapat kembali lancar.

6.3. Soal Latihan

Lakukan pemantauan terhadap kinerja salah satu sistem informasi yang digunakan di Instansi Anda dan Buatlah laporan hasil pemantauan yang sudah Anda lakukan selama seminggu.

BAB VII KESIMPULAN

Informasi merupakan sesuatu yang sangat penting dalam kehidupan manusia yang tidak boleh terlambat, tidak boleh bias, harus bebas dari kesalahan-kesalahan, dan relevan dengan penggunaannya, sehingga informasi tersebut menjadi informasi yang berkualitas dan berguna bagi pemakainya. Untuk mendapatkan informasi yang berkualitas, perlu dibangun sebuah sistem informasi sebagai media pembangkitnya. Sistem informasi merupakan cara untuk menghasilkan informasi yang berguna untuk mendukung sebuah pengambilan keputusan bagi pemakainya.

Untuk dapat menghasilkan sistem informasi yang berkualitas, sebuah organisasi perlu menjalankan siklus hidup sistem (*system life cycle*) mulai dari analisis kebutuhan, perancangan, implementasi, pengujian, sampai dengan pengoperasian dan pemeliharaan sistem. Jika pengelolaan *requirements* pada tahap analisis kebutuhan tidak efektif maka risiko kegagalan pembangunan atau perbaikan sistem sangat tinggi. Begitu pula tahapan yang lainnya perlu disusun dan dijalankan sesuai standar yang telah ditetapkan. Dengan berjalannya proses pembangunan atau perbaikan sistem yang baik, diharapkan dapat menghasilkan sistem informasi yang berkualitas yang dapat mendukung kelancaran proses bisnis organisasi.

DAFTAR PUSTAKA

- Badan Pusat Statistik. Peraturan Badan Pusat Statistik Nomor 2 Tahun 2021 tentang Petunjuk Teknis Penilaian Angka Kredit Jabatan Fungsional Pranata Komputer (2021).
- IEEE Computer Society. 2014. *SWEBOK: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society.
- Object Management Group. 2012. *Service oriented architecture Modeling Language (SoaML) Specification*. Object Management Group.
- Pressman, R. S. (2010). *Software Engineering: practitioner approach* (7th ed.). New York: McGraw-Hill.
- Sommerville, I. (2011). *Software Engineering: A Practitioner's Approach* (9th ed.). New York: Addison-Wesley.
- Visual Paradigm User's Guide. (2020, February 21). Retrieved February 24, 2022, from https://www.visual-paradigm.com/support/documents/vpuserguide/2535_soamlmodelin.html
- Whitten, J.L., Bentley, L.D., dan Dittman, K.C., 2004, Metode Desain dan Analisis Sistem, McGraw-Hill, disadur oleh Andi Offset.
- O'Brien, J.A., dan Marakas, G.M., 2011, Management Information Systems 10 edition, McGrawHill Education.
- Leitch and Robert A, 1983, Accounting Information System, Prentice Hall
- Davis, G dan Olson, M., 2012, Management Information Systems 2nd edition, Tata Mcgraw Hill.
- Stair, R dan Reynolds, G., 2017, Fundamentals of Information Systems 9th Edition, Cengage Learning.

Laudon, K.C dan Laudon, J.P., 2015, Management Information Systems:
Managing the Digital Firm, Student Value Edition (14th Edition),
Pearson.